Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерного проектирования
Кафедра экономической информатики
Дисциплина «Программирование сетевых приложений»

«К ЗАЩИТЕ ДОПУСТИТЬ»
Руководитель курсового проекта
Старший преподаватель кафедры ЭИ
_____ Д.А. Сторожев
___.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему:

«ПРОГРАММНОЕ СРЕДСТВО ДЛЯ ПОДДЕРЖКИ ПРОЦЕССА ОКАЗАНИЯ СТРАХОВЫХ УСЛУГ НАСЕЛЕНИЮ»

БГУИР КП 1-40 01 02-08 015 ПЗ

Выполнила студентка группы 414351
Балащенко Ксения Руслановна

_____
(подпись студента)

Курсовой проект представлен на проверку
___.____.2026

_____
(подпись студента)

Минск 2026

РЕФЕРАТ

ПРОГРАММНОЕ СРЕДСТВО ДЛЯ ПОДДЕРЖКИ ПРОЦЕССА ОКАЗАНИЯ СТРАХОВЫХ УСЛУГ НАСЕЛЕНИЮ : курсовой проект / К. Р. Балащенко. – Минск : БГУИР, 2026. – п.з. – 115 с., чертежей (плакатов) – 5 л. формата А4.

Предметом проектирования является процесс оказания страховых услуг физическим лицам: расчёт стоимости страхования, заключение и сопровождение договоров, учёт взносов и урегулирование страховых случаев.

Цель работы – разработка клиент-серверного программного средства на языке Java с многопоточным сервером и базой данных PostgreSQL, автоматизирующего работу страховой компании с клиентами.

При проектировании построены модели процессов в нотациях BPMN 2.0 и IDEF0, информационная модель IDEF1X и UML-модели, разработаны алгоритмы расчёта страховой премии, страховой выплаты и возврата части премии при расторжении договора.

Результатом является программное средство «Страховой сервис» для трёх ролей пользователей, реализующее семнадцать вариантов использования. Работоспособность подтверждена модульным, сквозным и нагрузочным тестированием.

Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»
Факультет компьютерного проектирования
Кафедра экономической информатики

«УТВЕРЖДАЮ»
Заведующий кафедрой ЭИ
_____А.А. Ефремов
«01» сентября 2026

ЗАДАНИЕ
на курсовой проект по дисциплине
«Программирование сетевых приложений»

Группа 414351

Студенту *Балащенко Ксении Руслановне*
(указать полностью фамилию, имя, отчество)

1. Тема курсового проекта: «Программное средство для поддержки процесса оказания страховых услуг населению»

2. Сроки сдачи студентом законченной работы: 05.01.2027.

3. Исходные данные к курсовому проекту:

3.1. Общие требования. *Приложение должно быть выполнено в архитектуре клиент-сервер с многопоточным сервером с организацией взаимодействия с базой данных на объектно-ориентированном языке Java*.

Уровни архитектуры: Серверное приложение может быть реализовано в виде консольного приложения или GUI-приложения. Настройки сервера должны меняться без изменения исходного кода (аргументы командной строки, конфиг-файлы и т.д.). Клиентское приложение: оконное приложение с использованием стандартных библиотек пользовательского интерфейса (JavaFX, SWING, AWT).

В рамках работы над курсовым проектом должны быть использованы следующие техники:

1) разработка и использование собственной иерархии классов (не менее 5), расширение базовых классов, предоставляемых JDK 8 и выше;
2) реализация не менее 2-х паттернов проектирования на свой выбор (по желанию).
3) использовать сокрытие данных (инкапсуляция), перегрузку методов, переопределение методов, сериализацию, абстрактные типы данных (интерфейсы, абстрактные классы), статические методы, обработку исключительных ситуаций.

Бизнес-логика. Бизнес-логика системы должна быть реализована только на серверной части приложения. На сервере должна быть предусмотрена возможность параллельной обработки запросов. Пользовательский интерфейс должен обеспечивать доступ ко всем функциям. Функциональные возможности серверной части должны насчитывать не менее 12 высокоуровневых вариантов использования, исключая тривиальные операции работы с БД (добавление, удаление, редактирование записей в БД). Необходимо предусмотреть механизм авторизации всех пользователей. Количество ролей – не менее 2-3 в зависимости от специфики предметной области (администратор, сотрудник компании, гость). Проектирование и разработку реляционной модели (не менее 5 связанных таблиц) предметной области осуществить с использованием процедуры нормализации отношений (до 3 НФ). Связи между таблицами должны быть определены как идентифицирующие и не идентифицирующие, с указанием мощности связи. Схема базы данных должна обеспечивать ссылочную целостность данных. Доступ к данным в СУБД должен осуществляться через драйвер, предоставляемый производителем СУБД или через использование специальных технологий.

Конкретные версии фреймворков и технологий, применяемых для реализации программного средства, должны быть актуальными на начало 2025 года.

3.2. Разработанное программное обеспечение должно выполняться в системе Windows 7 и выше с возможной предустановкой библиотек или пакетов выбранной среды программирования.

3.3. Нормативные источники: Положение о курсовом проектировании БГУИР. СТП 01-2024. Стандарт предприятия. Дипломные проекты (работы). Проектирование программного

средства выполнять с учетом положений, изложенных в руководящих документах методологий IDEF0 и IDEF1.X, нотации BPMN 2.0, стандарта UML 2.0 и выше.

3.4. Курсовой проект должен храниться в публичном репозитории на GitHub. Все коммиты, которые были сделаны в течение каждой календарной недели, должны быть загружены в репозиторий на GitHub.

3.5. Остальные данные и требования уточняются у руководителя в процессе проектирования и разработки.

4. Содержание пояснительной записки (перечень подлежащих разработке вопросов):

Титульный лист. Реферат. Задание по курсовому проекту. Содержание. Перечень условных обозначений, символов и терминов.

Введение.

4.1. Анализ и моделирование предметной области программного средства. 4.1.1. Описание предметной области. 4.1.2. Разработка функциональной модели предметной области. 4.1.3. Анализ требований к разрабатываемому программному средству. Спецификация функциональных требований. 4.1.4. Разработка информационной модели предметной области. 4.1.5. UML-модели представления программного средства и их описание.

4.2. Проектирование и конструирование программного средства. 4.2.1. Постановка задачи. 4.2.2. Архитектурные решения. 4.2.3. Описание алгоритмов, реализующих ключевую бизнес-логику разрабатываемого программного средства. 4.2.4. Проектирование пользовательского интерфейса. 4.2.5. Обоснование выбора компонентов и технологий для реализации программного средства.

4.3. Тестирование и проверка работоспособности программного средства.

4.4. Инструкция по развертыванию приложения и сквозной тестовый пример, начиная от авторизации, демонстрируя реализацию всех вариантов использования.

Заключение. Список использованных источников. Приложения (обязательные): отчет о проверке на заимствования в системе «Антиплагиат»; листинг кода алгоритмов, реализующих основную бизнес-логику; листинг скрипта генерации базы данных. Ведомость документов курсового проекта.

5. Перечень графического материала (с указанием обязательных чертежей и графиков):

5.1. IDEF0-модель процессов предметной области (чертеж, 1 лист формата А4).

5.2. Схема алгоритма, реализующая основную бизнес-логику программного средства (чертеж, 1 лист формата А4).

5.3. Плакаты, отражающие результаты проектирования программного средства (3 листа формата А4):

5.3.1. UML диаграмма классов (плакат, 1 лист формата А4).

5.3.2. Модели представления программного средства (плакат, 1 лист формата А4).

5.3.3. Скриншоты рабочих окон программного средства (плакат, 1 лист формата А4).

6. Консультант по курсовому проекту: ст. преподаватель СТОРОЖЕВ Дмитрий Алексеевич (ауд. 802а – 5 корп.).

7. Дата выдачи задания: 01.09.2026.

8. Календарный график работы над курсовым проектом на весь период проектирования (с указанием сроков выполнения и трудоемкости отдельных этапов):

№ п/п	Наименование этапов курсового проекта	Срок выполнения этапов курсового проекта	Примечание
1.	1-я опроцентовка (введение, 4.1, 5.1)	10-15.10.2026	30%
2.	2-я опроцентовка (4.2, 4.3, 5.2)	10-15.11.2026	60%
3.	3-я опроцентовка (введение, 4.4, 5.3 заключение)	20-24.12.2026	90%
4.	Сдача курсового проекта на проверку	05.01.2027	100%
5.	Защита курсового проекта	12-25.01.2027	Согласно графику

Руководитель _____ (Д.А. Сторожев)

Задание принял к исполнению 01.09.2026 _____ (К.Р. Балащенко)

(*подпись студента*) *(расшифровка подписи)*

СОДЕРЖАНИЕ

Перечень условных обозначений, символов и терминов.....6
Введение.....8
1 Анализ и моделирование предметной области программного средства……10
1.1 Описание процесса оказания страховых услуг населению.....10
1.2 Разработка функциональной модели предметной области.....15
1.3 Анализ требований к разрабатываемому программному средству. Спецификация функциональных требований.....21
1.4 Разработка информационной модели предметной области.....23
1.5 UML-модели представления программного средства и их описание….27
2 Проектирование и конструирование программного средства.....34
2.1 Постановка задачи.....34
2.2 Архитектурные решения.....36
2.3 Описание алгоритмов, реализующих ключевую бизнес-логику разрабатываемого программного средства.....45
2.4 Проектирование пользовательского интерфейса.....51
2.5 Обоснование выбора компонентов и технологий для реализации программного средства.....56
3 Тестирование и проверка работоспособности программного средства.....59
4 Инструкция по развертыванию приложения и сквозной тестовый пример. 66
4.1 Требования к программному и аппаратному обеспечению.....66
4.2 Развертывание базы данных.....66
4.3 Сборка, настройка и запуск сервера.....67
4.4 Запуск клиентского приложения.....68
4.5 Сквозной тестовый пример.....68
Заключение.....87
Список использованных источников.....89
Приложение А (обязательное) Отчет о проверке на заимствования в системе «Антиплагиат».....91
Приложение Б (обязательное) Листинг программного кода алгоритмов, реализующих основную бизнес-логику.....92
Приложение В (обязательное) Листинг скрипта генерации базы данных….111
Ведомость документов.....115

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СИМВОЛОВ И ТЕРМИНОВ

ЗНФ – третья нормальная форма отношений реляционной базы данных.

БД – база данных.

КАСКО – добровольное страхование наземного транспортного средства от ущерба и угона.

Коэффициент бонус-малус ($К_{бм}$) – поправочный коэффициент к премии, зависящий от числа выплат клиенту за предыдущие два года.

Коэффициент убыточности – отношение суммы страховых выплат к сумме собранных страховых премий за период.

ПС – программное средство.

Страхователь – физическое лицо, заключившее со страховщиком договор страхования (в ПС – клиент).

Страховая выплата (В) – денежная сумма, выплачиваемая страховщиком в возмещение ущерба.

Страховая премия (П) – плата за страхование, уплачиваемая страхователем единовременно или страховыми взносами.

Страховая сумма (С) – денежная сумма, в пределах которой страховщик обязан произвести страховую выплату.

Страховой агент – работник страховой организации, оформляющий договоры и урегулирующий страховые случаи.

Страховой взнос – часть страховой премии, уплачиваемая в рассрочку в соответствии с графиком.

Страховой случай – событие, предусмотренное договором, с наступлением которого возникает обязанность страховщика произвести страховую выплату.

Страховой тариф (Т) – годовая ставка страховой премии в долях страховой суммы.

СУБД – система управления базами данных.

ФИО – фамилия, имя, отчество.

Франшиза (Ф) – часть ущерба, не возмещаемая страховщиком; задаётся в процентах от страховой суммы.

ФТ – функциональное требование.

API – Application Programming Interface, программный интерфейс приложения.

BPMN – Business Process Model and Notation, нотация моделирования бизнес-процессов.

BYN – белорусский рубль.

CSS – Cascading Style Sheets, язык описания внешнего вида элементов интерфейса.

DAO – Data Access Object, объект доступа к данным, инкапсулирующий SQL-запросы к одной таблице или группе таблиц.

FK – Foreign Key, внешний ключ таблицы базы данных.

GPL – General Public License, лицензия свободного программного обеспечения.

HTTP – HyperText Transfer Protocol, протокол передачи гипертекста.

IDEF0 – методология функционального моделирования процессов.

IDEF1X – методология информационного моделирования (проектирования реляционных баз данных).

JDBC – Java Database Connectivity, стандартный программный интерфейс Java для доступа к реляционным СУБД.

JDK, JRE – Java Development Kit и Java Runtime Environment, комплект разработчика и среда выполнения Java.

JSON – JavaScript Object Notation, текстовый формат обмена данными.

LTS – Long-Term Support, выпуск с долгосрочной поддержкой.

ORM – Object-Relational Mapping, технология отображения объектов на таблицы реляционной базы данных.

PK – Primary Key, первичный ключ таблицы базы данных.

REST – Representational State Transfer, архитектурный стиль веб-сервисов.

RMI – Remote Method Invocation, механизм удалённого вызова методов Java.

SHA-256 – криптографическая хеш-функция с длиной результата 256 бит.

SQL – Structured Query Language, язык структурированных запросов к реляционным базам данных.

TCP/IP – стек сетевых протоколов, используемый для обмена клиентского приложения с сервером.

UC – Use Case, вариант использования; UC1–UC17 – номера вариантов использования ПС.

UML – Unified Modeling Language, унифицированный язык моделирования.

ВВЕДЕНИЕ

Страхование является одним из основных механизмов защиты имущественных интересов граждан от последствий случайных неблагоприятных событий: повреждения жилья и автомобиля, несчастных случаев, болезни во время поездки за рубеж. Добровольные виды страхования физических лиц относятся к массовым услугам: страховая компания одновременно сопровождает тысячи договоров, по каждому из которых нужно рассчитать стоимость, оформить полис, проконтролировать уплату взносов, а при наступлении страхового случая – проверить обоснованность требования и рассчитать сумму возмещения.

При ручном ведении этих операций или при использовании разрозненных электронных таблиц возникают типичные проблемы: ошибки при расчёте страховой премии и выплат, несогласованность сведений о договорах между сотрудниками, потеря информации о просроченных взносах, длительное ожидание клиентом решения по заявке, трудоёмкость подготовки отчётности для руководства. Поэтому автоматизация процесса оказания страховых услуг населению является актуальной задачей.

Целью курсового проекта является разработка программного средства для поддержки процесса оказания страховых услуг населению, построенного по архитектуре «клиент-сервер» с многопоточным сервером и хранением данных в реляционной базе данных.

Для достижения поставленной цели необходимо решить следующие задачи:

– проанализировать предметную область и построить её функциональную модель в нотации IDEF0;

– сформулировать функциональные и нефункциональные требования к программному средству;

– разработать информационную модель предметной области в нотации IDEF1X и спроектировать базу данных, приведённую к третьей нормальной форме;

– построить UML-модели программного средства: диаграммы вариантов использования, состояний, последовательности, классов и развёртывания;

– спроектировать архитектуру программного средства, алгоритмы расчёта страховой премии и страховой выплаты, пользовательский интерфейс;

– реализовать серверную и клиентскую части программного средства на языке Java;

– провести тестирование программного средства и подготовить инструкцию по его развёртыванию.

Программное средство предназначено для трёх категорий пользователей: клиентов страховой компании, страховых агентов и администратора. Оно позволяет клиенту самостоятельно рассчитать стоимость страхования, подать заявку, вносить взносы и заявлять о страховых случаях, агенту – оформлять договоры и урегулировать убытки, а администратору – управлять тарифами и учётными записями и анализировать результаты деятельности компании.

Пояснительная записка состоит из четырёх разделов. В первом разделе выполнен анализ предметной области и построены её модели, во втором описаны архитектура, алгоритмы, интерфейс и выбор технологий, в третьем – тестирование, в четвёртом – развёртывание и сквозной пример работы. В приложениях приведены отчёт о проверке на заимствования (приложение А), листинг программного кода бизнес-логики (приложение Б) и скрипт генерации базы данных (приложение В). Оформление выполнено в соответствии со стандартом предприятия СТП 01-2024 [1].

Данный курсовой проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет ___ % (отчет о проверке на заимствования прилагается).

1 АНАЛИЗ И МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ ПРОГРАММНОГО СРЕДСТВА

1.1 Описание процесса оказания страховых услуг населению

Страхование – это отношения по защите имущественных интересов физических и юридических лиц при наступлении определённых событий (страховых случаев) за счёт денежных фондов, формируемых из уплачиваемых страховых взносов. В Республике Беларусь страховая деятельность регулируется главой 48 Гражданского кодекса [2] и Положением о страховой деятельности, утверждённым Указом Президента Республики Беларусь № 530 [3]. Добровольное страхование осуществляется на основании правил страхования, которые разрабатывает и утверждает страховая организация.

Основными участниками процесса являются страховщик (страховая организация), страхователь – физическое лицо, заключившее договор, и страховой агент – работник страховщика, который консультирует клиентов, оформляет договоры и рассматривает заявления о страховых случаях. Руководство компании устанавливает тарифы и анализирует результаты продаж и выплат.

Ключевыми понятиями предметной области являются:

– **страховая сумма** – денежная сумма, в пределах которой страховщик обязан произвести выплату;

– **страховой тариф** – ставка страховой премии с единицы страховой суммы за период страхования;

– **страховая премия** – плата за страхование, которую страхователь обязан уплатить страховщику; может уплачиваться единовременно или в рассрочку страховыми взносами;

– **франшиза** – часть убытка, не возмещаемая страховщиком; устанавливается в процентах от страховой суммы;

– **страховой случай** – совершившееся событие, предусмотренное договором, с наступлением которого возникает обязанность страховщика произвести страховую выплату;

– **страховая выплата** – денежная сумма, выплачиваемая страховщиком в возмещение ущерба;

– **коэффициент убыточности** – отношение суммы страховых выплат к сумме собранных страховых премий за период.

В рамках курсового проекта рассматриваются четыре массовых вида добровольного страхования физических лиц: страхование имущества (квартиры), страхование наземных транспортных средств (КАСКО),

страхование от несчастных случаев и страхование медицинских расходов граждан, выезжающих за рубеж.

Процесс оказания страховой услуги включает следующие этапы:

1 Консультирование и расчёт стоимости. Клиент обращается в компанию, выбирает вид страхования, страховую сумму и срок; агент по тарифам рассчитывает размер премии с учётом поправочных коэффициентов (возраст страхователя, срок, история выплат, размер франшизы).

2 Заключение договора. Клиент подаёт заявление (заявку) с данными об объекте страхования; агент проверяет заявку и при положительном решении оформляет полис, определяя график уплаты взносов.

3 Сопровождение договора. Учитываются поступившие взносы, контролируются просрочки, договор может быть продлён на новый срок или досрочно расторгнут с возвратом части премии.

4 Урегулирование страхового случая. Клиент сообщает о событии, агент проверяет действие договора и уплату взносов, оценивает ущерб, рассчитывает и оформляет выплату или мотивированный отказ.

5 Отчётность и анализ. Руководство получает сведения о продажах агентов и анализирует убыточность видов страхования, на основании которой корректируются тарифы.

Анализ существующих решений показывает, что крупные страховые организации предоставляют клиентам личные кабинеты на своих сайтах [4], а внутренний учёт ведут в отраслевых учётных системах. Такие решения дороги, ориентированы на конкретную компанию и не подходят для небольших страховых организаций и учебных целей. Разрабатываемое программное средство объединяет в одном приложении функции клиента, агента и администратора, автоматизирует расчёты премии, выплат и возвратов и предоставляет аналитику в наглядном виде.

Взаимодействие участников процесса описано моделями бизнес-процессов в нотации BPMN 2.0 [5]. Модели построены в виде диаграмм взаимодействия: клиент и страховая компания представлены отдельными пулами, внутри пула страховой компании выделены дорожки страхового агента и разрабатываемого программного средства. Потоки управления внутри пула показаны сплошными линиями со стрелками, потоки сообщений между пулами – штриховыми линиями. Ромбы с крестом обозначают исключающие шлюзы (выбор одного из вариантов), окружности – начальные, промежуточные и конечные события; конверт внутри события означает получение или отправку сообщения.

Процесс заключения договора страхования приведён на рисунке 1.1. Клиент запрашивает расчёт стоимости страхования, который программное средство выполняет по тарифу вида страхования и поправочным

коэффициентам. Если условия устраивают клиента, он подаёт заявку. Поступившую заявку рассматривает страховой агент: при положительном решении программное средство оформляет договор и график взносов, при отрицательном агент указывает причину отказа. Решение передаётся клиенту; после уплаты первого взноса договор вступает в действие.

Процесс урегулирования страхового случая приведён на рисунке 1.2. Клиент подаёт заявление о страховом случае, программное средство регистрирует его, после чего агент осматривает объект страхования и оценивает размер ущерба. Программное средство проверяет условия выплаты (действие договора на дату события, отсутствие просроченных взносов, срок подачи заявления, превышение ущербом франшизы) и рассчитывает сумму выплаты. Если агент не согласен с исходными данными расчёта, оценка ущерба уточняется; подтверждённое решение фиксируется, остаток страховой суммы уменьшается, а клиент видит решение и его обоснование в списке своих страховых случаев.

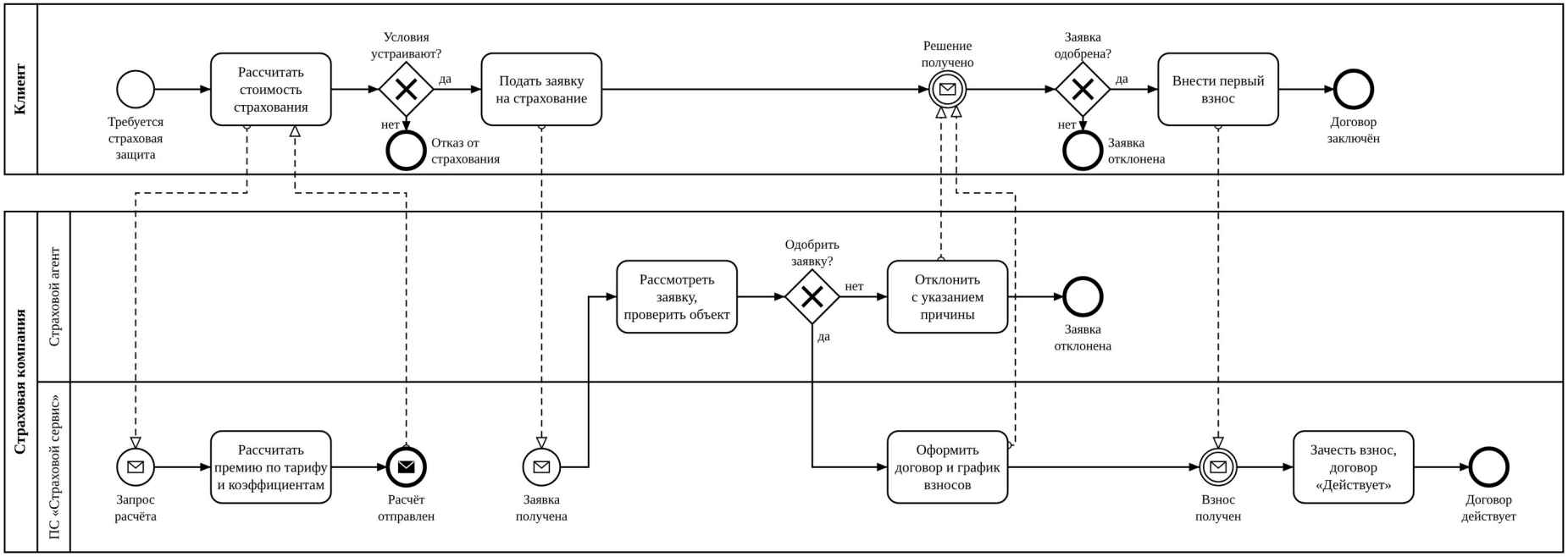


Рисунок 1.1 – Модель процесса заключения договора страхования в нотации BPMN 2.0

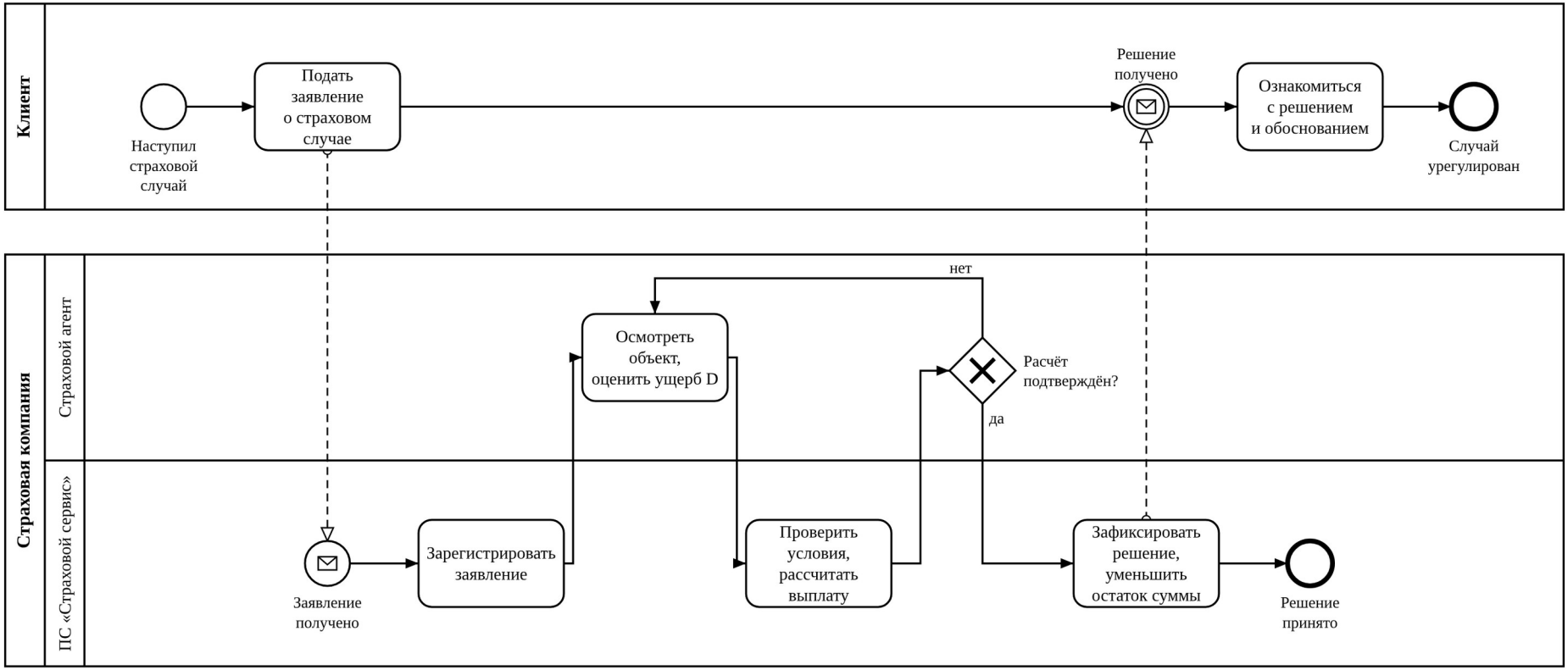


Рисунок 1.2 – Модель процесса урегулирования страхового случая в нотации BPMN 2.0

Модели показывают, что расчётные операции и проверки условий не требуют участия человека, а агент принимает решения только там, где нужна экспертная оценка: при рассмотрении заявки и оценке ущерба. Именно расчёты и проверки, а также учёт договоров, взносов и выплат подлежат автоматизации в разрабатываемом программном средстве.

1.2 Разработка функциональной модели предметной области

Функциональная модель предметной области построена в нотации IDEF0 [6]. Цель моделирования – описать процесс оказания страховых услуг физическим лицам для определения функций, подлежащих автоматизации. Точка зрения – руководитель страховой компании. Модель содержит четыре уровня: контекстную диаграмму A-0, диаграмму декомпозиции A0, диаграмму декомпозиции функции A4 и диаграмму декомпозиции функции A44.

Контекстная диаграмма приведена на рисунке 1.3. Входами процесса являются обращение клиента с данными страхователя и объекта страхования, страховые взносы и заявления о страховых случаях. Процесс управляется законодательством Республики Беларусь о страховании, а также правилами страхования и тарифами компании. Выходами являются договоры страхования (полисы), страховые выплаты или мотивированные отказы, отчётность и анализ убыточности. Механизмами выполнения процесса выступают страховые агенты, администратор и разрабатываемое программное средство.

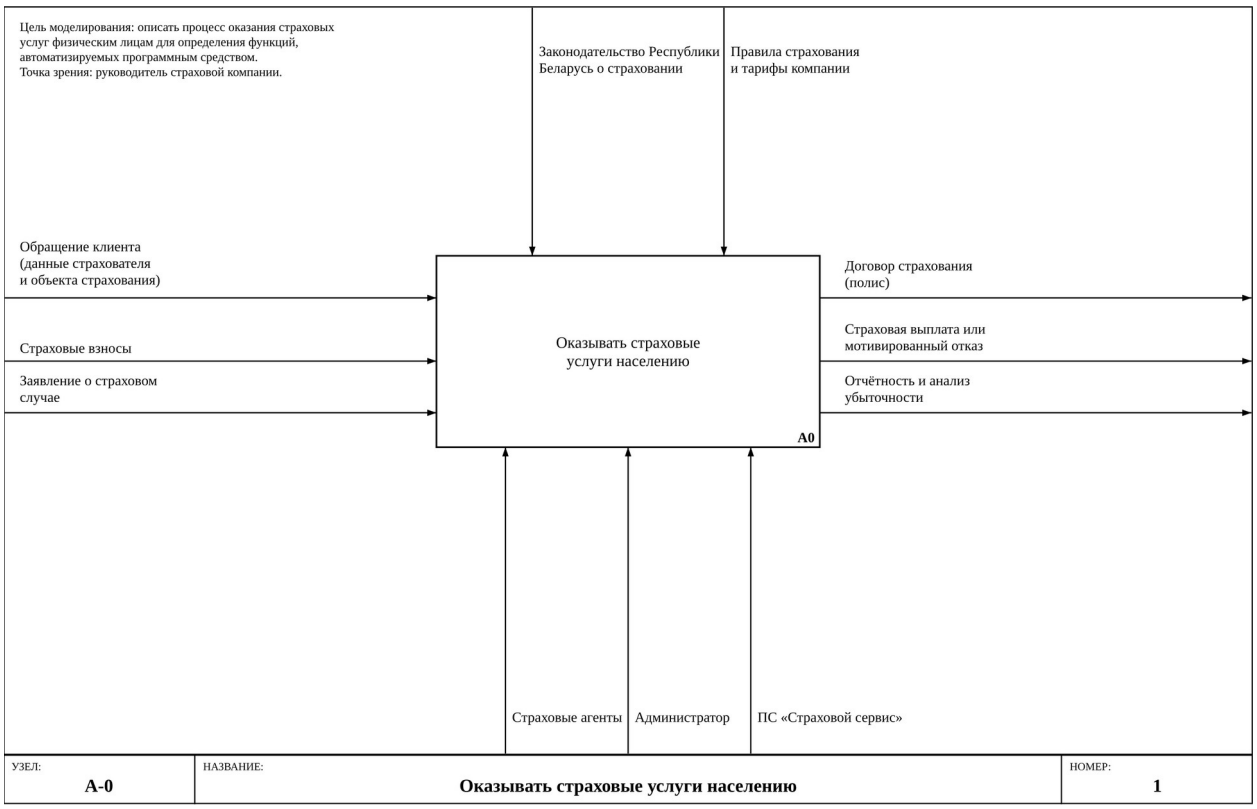


Рисунок 1.3 – Контекстная диаграмма A-0 «Оказывать страховые услуги населению»

Декомпозиция контекстной функции (рисунок 1.4) выделяет пять подпроцессов:

– A1 «Консультировать и рассчитать стоимость страхования» – по обращению клиента и тарифам компании рассчитывается стоимость страхования;

– A2 «Заключить договор страхования» – на основе расчёта оформляется договор (полис) и график взносов;

– A3 «Сопровождать договор страхования» – учитываются поступающие взносы, формируются сведения о статусе договора и об оплате;

– A4 «Урегулировать страховой случай» – по заявлению клиента с учётом статуса договора принимается решение о выплате или об отказе;

– A5 «Формировать отчётность и анализировать убыточность» – по данным о договорах, взносах и выплатах формируется отчётность для руководства.

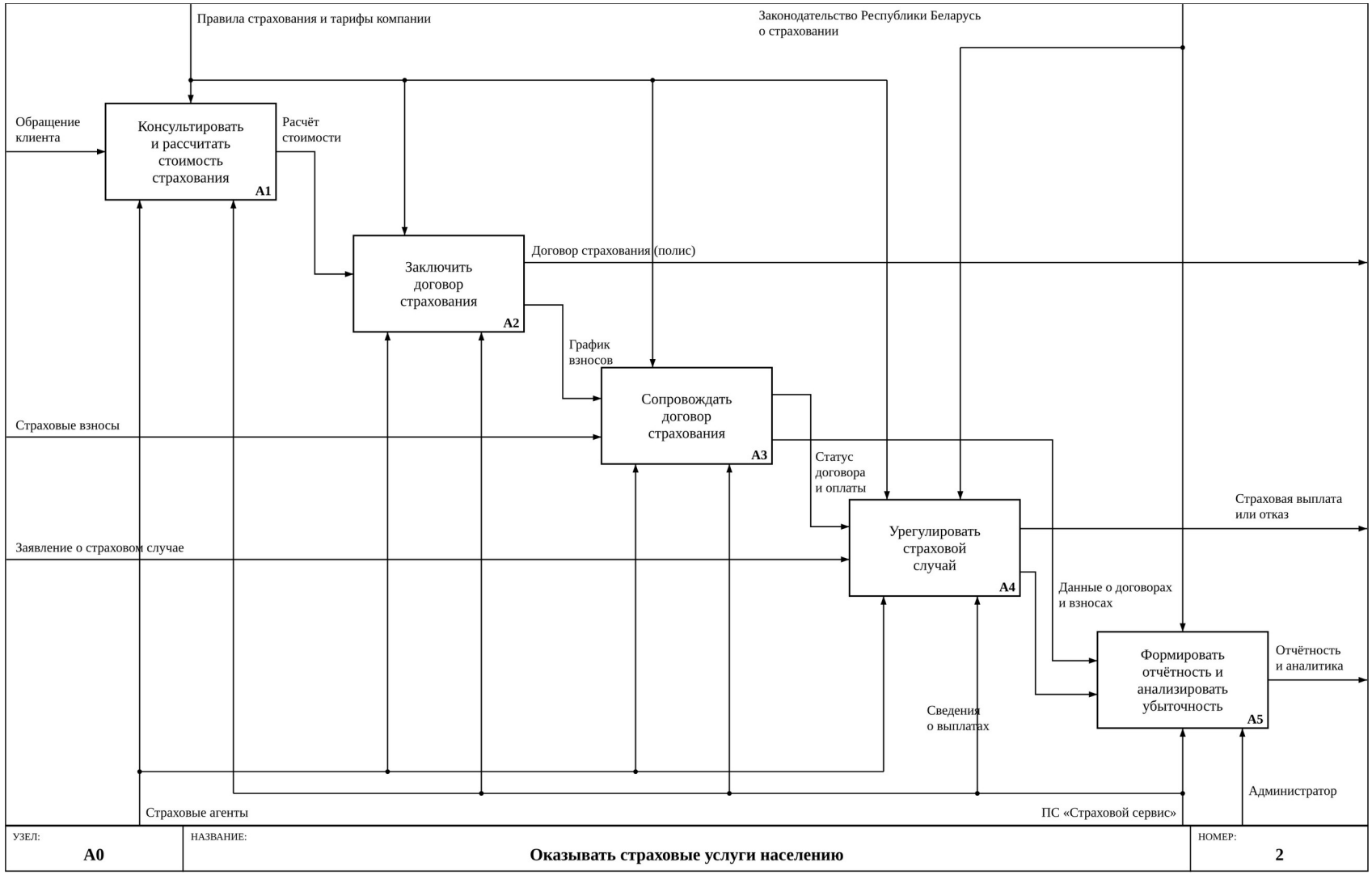


Рисунок 1.4 – Диаграмма декомпозиции A0

Наиболее ответственным подпроцессом является урегулирование страхового случая, так как ошибка в нём ведёт к прямым финансовым потерям компании или к обоснованным претензиям клиентов. Его декомпозиция приведена на рисунке 1.5. Заявление о страховом случае регистрируется (A41), затем проверяются действие договора на дату события и отсутствие просроченных взносов (A42). Если договор не действовал или взносы просрочены, формируется основание для отказа, которое передаётся сразу в функцию оформления решения (A45). В противном случае оценивается размер ущерба (A43), рассчитывается сумма выплаты (A44) и оформляется решение (A45).

Функция расчёта страховой выплаты детализирована на рисунке 1.6. Сначала определяется остаток страховой суммы с учётом ранее произведённых по договору выплат (A441). Параллельно из размера ущерба вычитается франшиза (A442). Затем выплата ограничивается остатком страховой суммы (A443), после чего выплата фиксируется, а при исчерпании страховой суммы договор переводится в статус «Исполнен» (A444). Именно эта функция реализуется алгоритмом расчёта страховой выплаты в разрабатываемом программном средстве.

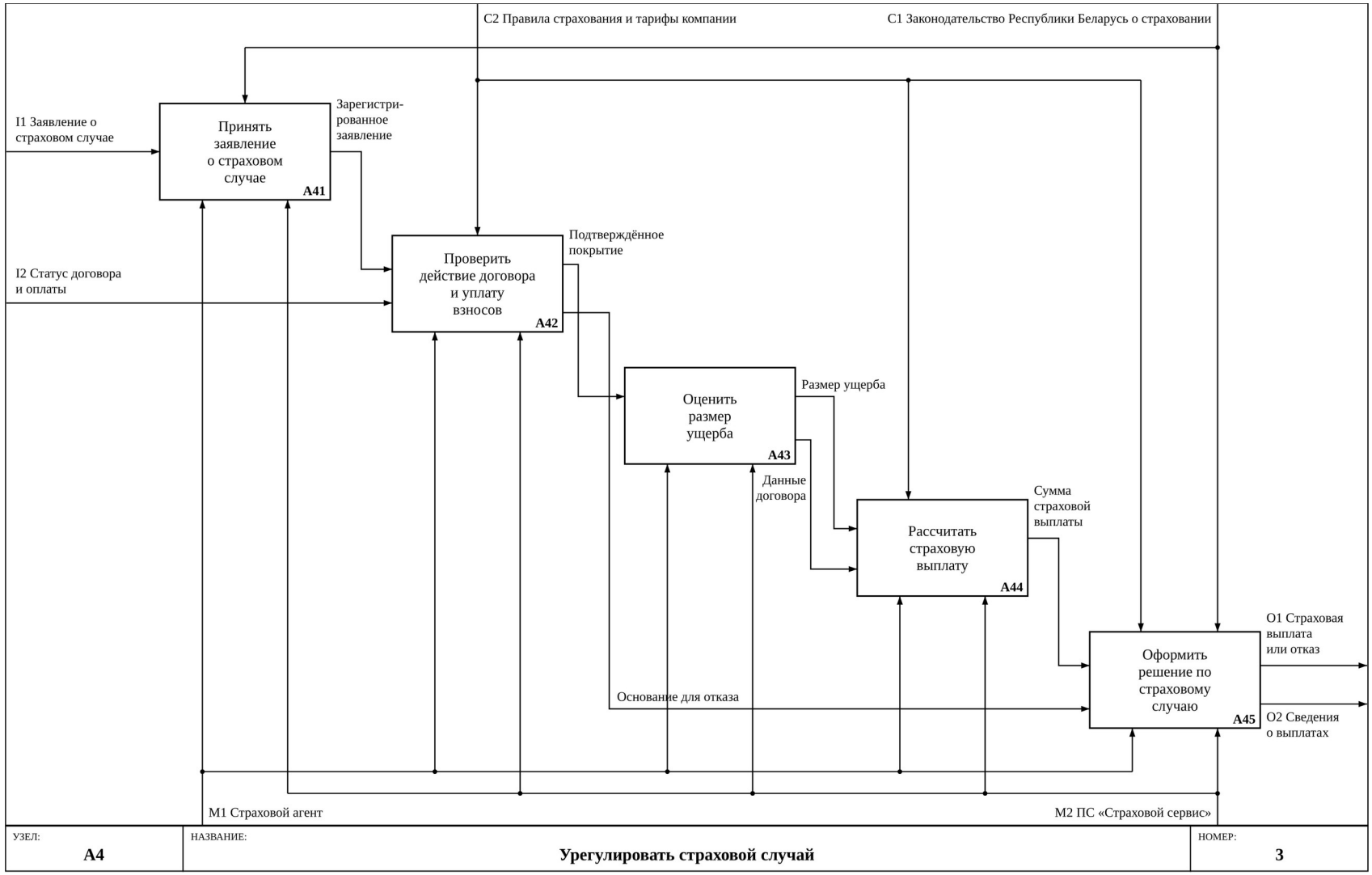


Рисунок 1.5 – Диаграмма декомпозиции А4 «Урегулировать страховой случай»

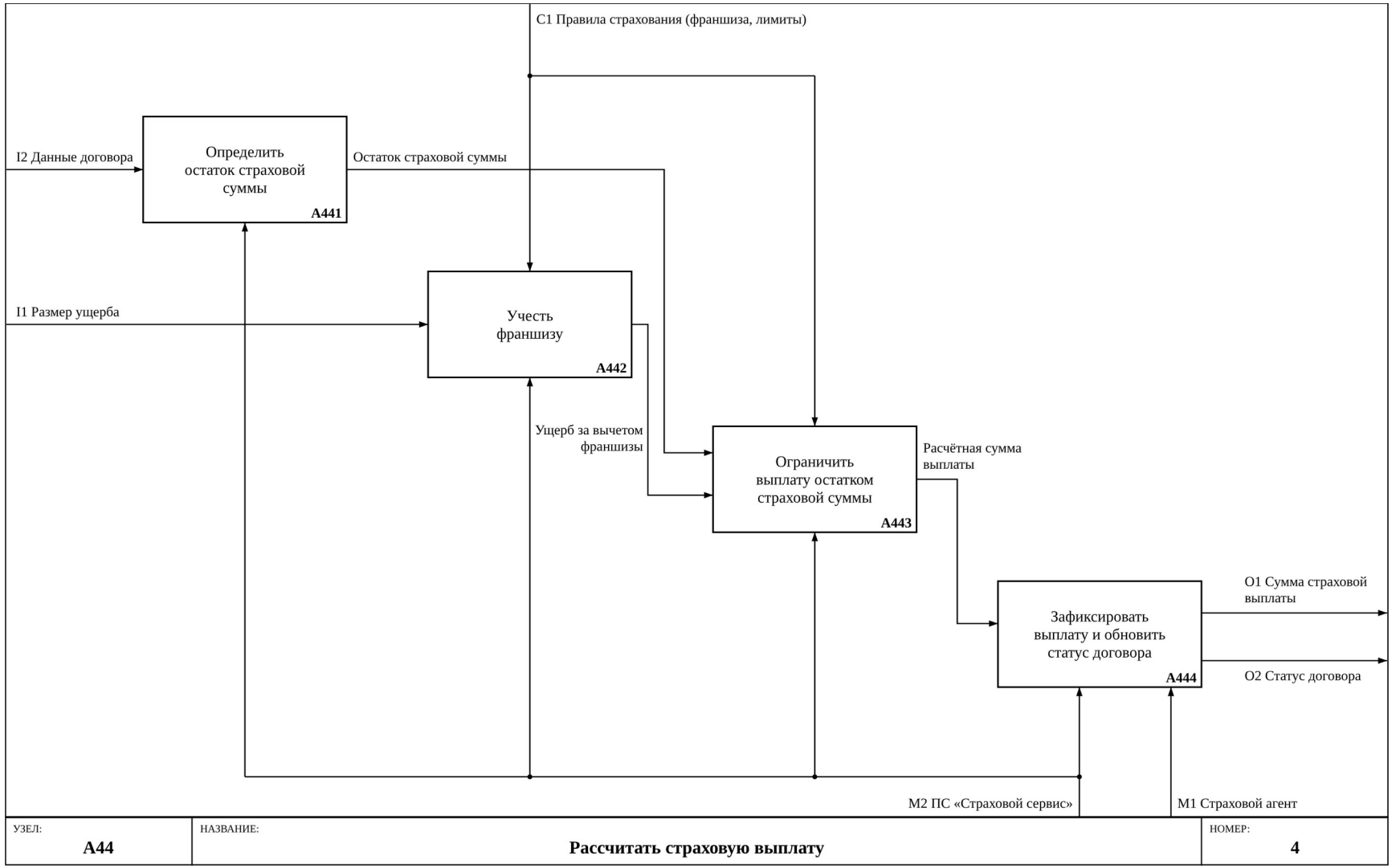


Рисунок 1.6 – Диаграмма декомпозиции A44 «Рассчитать страховую выплату»

Построенная модель показывает, что все пять подпроцессов опираются на общие данные о клиентах, договорах, взносах и выплатах и выполняются разными сотрудниками. Это обосновывает необходимость единого программного средства с централизованной базой данных и разграничением прав доступа по ролям.

1.3 Анализ требований к разрабатываемому программному средству. Спецификация функциональных требований

На основе функциональной модели выделены три роли пользователей программного средства:

– **клиент** – физическое лицо, страхователь; регистрируется в системе самостоятельно;

– **агент** – сотрудник страховой компании; учётную запись создаёт администратор;

– **администратор** – отвечает за справочники, учётные записи и аналитику.

Незарегистрированный пользователь (гость) может только зарегистрироваться в качестве клиента. Для доступа ко всем остальным функциям требуется авторизация по логину и паролю. Спецификация функциональных требований приведена в таблице 1.1.

Таблица 1.1 – Спецификация функциональных требований

Код	Требование	Роль
ФТ-01	Регистрация клиента: ввод логина, пароля, ФИО, даты рождения, паспортных данных и контактов; проверка уникальности логина и номера паспорта	Гость
ФТ-02	Авторизация по логину и паролю; хранение пароля в виде хеша SHA-256 с солью; запрет входа заблокированным пользователям; открытие окна, соответствующего роли	Все
ФТ-03	Расчёт стоимости страхования по виду, страховой сумме, сроку и франшизе с учётом возраста и истории выплат клиента	Клиент, агент
ФТ-04	Подача заявки на страхование с описанием объекта и выбранным порядком уплаты взносов	Клиент
ФТ-05	Рассмотрение заявки: одобрение с автоматическим оформлением полиса, расчётом премии и графика взносов либо отклонение с указанием причины	Агент
ФТ-06	Внесение очередного страхового взноса; перевод договора в статус «Действует» после первого взноса и снятие приостановки после погашения задолженности	Клиент
ФТ-07	Подача заявления о страховом случае с датой события, описанием и размером ущерба	Клиент

Продолжение таблицы 1.1

Код	Требование	Роль
ФТ-08	Урегулирование страхового случая: автоматическая проверка условий, расчёт выплаты с учётом франшизы и остатка страховой суммы, фиксация решения	Агент
ФТ-09	Продление договора на новый срок с пересчётом премии	Агент
ФТ-10	Досрочное расторжение договора с расчётом возврата части премии	Агент
ФТ-11	Контроль просроченных взносов и договоров, срок действия которых истекает в течение 30 дней	Агент
ФТ-12	Просмотр клиентом своих договоров, графика взносов, заявок и статусов страховых случаев	Клиент
ФТ-13	Управление видами страхования и тарифами: базовый тариф, пределы страховой суммы, максимальный срок, признак активности	Администратор
ФТ-14	Управление учётными записями агентов: создание, блокировка и разблокировка	Администратор
ФТ-15	Отчёт о продажах агентов за период (число договоров, сумма премий) с диаграммой	Администратор
ФТ-16	Анализ убыточности видов страхования за период (премии, выплаты, коэффициент убыточности) с диаграммой	Администратор
ФТ-17	Выгрузка отчётов в текстовый файл	Администратор

Требования ФТ-03, ФТ-05, ФТ-08, ФТ-10, ФТ-11, ФТ-15 и ФТ-16 реализуют высокоуровневую бизнес-логику и не сводятся к добавлению, изменению или удалению записей базы данных. Вся бизнес-логика выполняется на серверной стороне; клиентское приложение только передаёт запросы и отображает результаты.

К программному средству предъявляются следующие нефункциональные требования:

– архитектура «клиент-сервер»; сервер обрабатывает запросы нескольких клиентов параллельно, каждое подключение обслуживается в отдельном потоке;

– настройки сервера (порт, параметры подключения к базе данных, размер пула потоков) задаются в конфигурационном файле и аргументах командной строки без изменения исходного кода;

– клиентское приложение – оконное, на стандартной библиотеке Swing;

– хранение данных – в СУБД PostgreSQL, доступ через JDBC-драйвер производителя; операции, изменяющие несколько таблиц, выполняются в транзакциях;

– работа в операционной системе Windows 7 и выше при установленной среде выполнения Java;

– время отклика на типовые операции – не более 2 с при работе в локальной сети;

– все денежные расчёты выполняются в типе BigDecimal с округлением до копеек;

– ошибочный ввод пользователя не должен приводить к аварийному завершению программы: выводится понятное сообщение об ошибке.

1.4 Разработка информационной модели предметной области

Информационная модель построена в нотации IDEF1X [7] и приведена на рисунке 1.7. При её разработке выделены сущности предметной области, их атрибуты и связи, после чего отношения приведены к третьей нормальной форме: все атрибуты атомарны (1НФ), неключевые атрибуты полностью зависят от первичного ключа (2НФ) и не зависят транзитивно друг от друга (3НФ). Так, данные о человеке вынесены из учётной записи в отдельные сущности «Клиент» и «Агент», а тарифы – в сущность «Вид страхования», на которую ссылаются заявки и договоры. Модель содержит девять сущностей; в базе данных каждой из них соответствует таблица.

Сущность payments (взнос) является зависимой: её первичный ключ состоит из идентификатора договора и номера взноса, поэтому связь «договор – взносы» является идентифицирующей (сплошная линия). Остальные связи – неидентифицирующие (штриховая линия). Мощность связей: «один ко многим» по умолчанию, *P* – «один к одному или более», *Z* – «один к нулю или одному». Связи «учётная запись – клиент», «учётная запись – агент» и «заявка – договор» имеют мощность *Z*, так как одной учётной записи соответствует не более одного клиента или агента, а по одной заявке оформляется не более одного договора.

Ссылочная целостность обеспечивается внешними ключами с правилом ON DELETE RESTRICT; для взносов используется каскадное удаление вместе с договором. Допустимые значения статусов, неотрицательность денежных сумм и корректность дат контролируются ограничениями CHECK. Для частых выборок созданы индексы по статусу заявок и договоров, клиенту договора, договору страхового случая и частичный индекс по сроку неуплаченных взносов. Назначение таблиц и их полей приведено в таблицах 1.2–1.10.

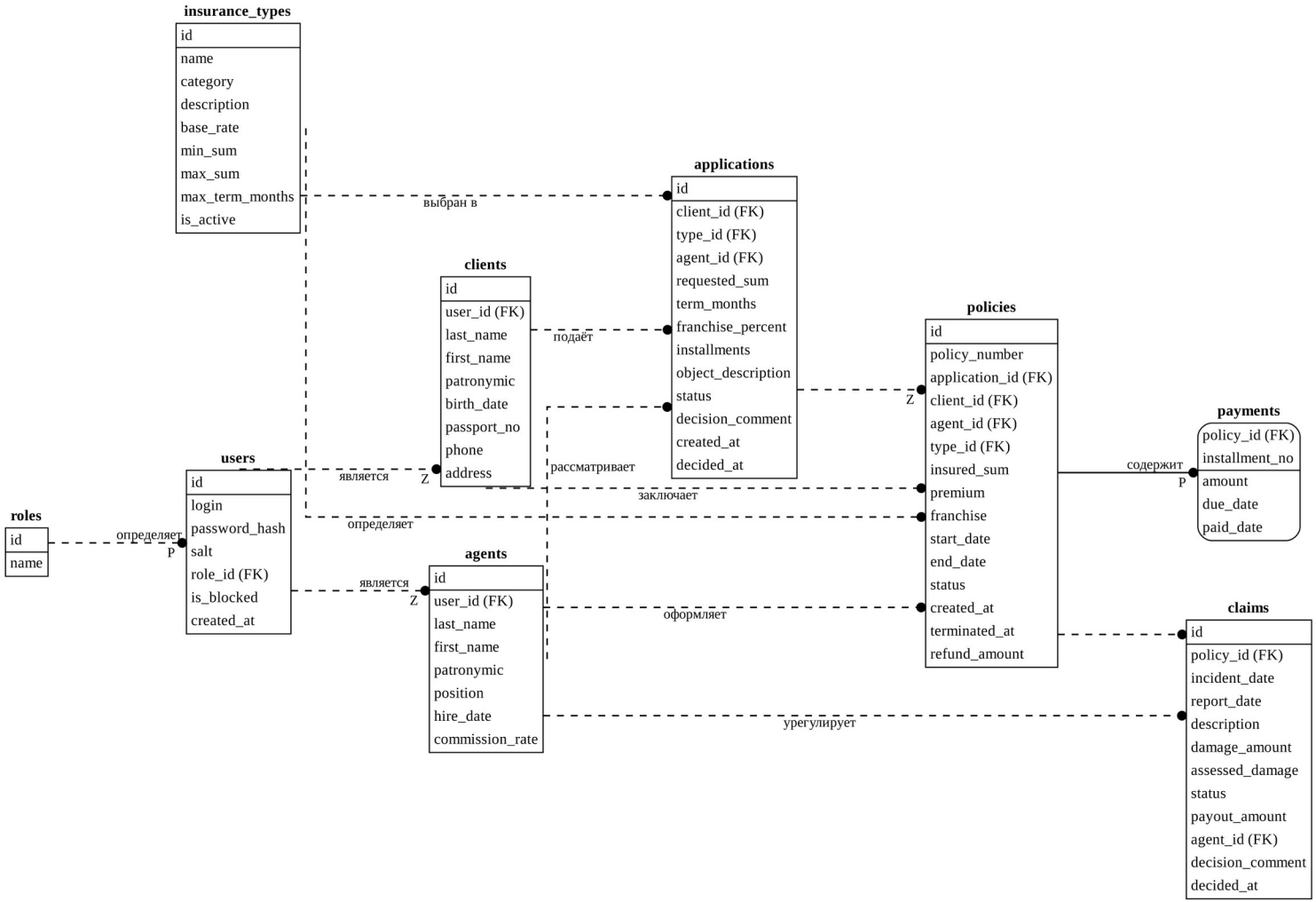


Рисунок 1.7 – Информационная модель предметной области в нотации IDEF1X

Таблица 1.2 – Структура таблицы roles (роли)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор роли
name	VARCHAR(20), UNIQUE	Код роли: CLIENT, AGENT, ADMIN

Таблица 1.3 – Структура таблицы users (учётные записи)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор учётной записи
login	VARCHAR(50), UNIQUE	Логин для входа
password_hash	CHAR(64)	Хеш SHA-256 пароля с солью
salt	CHAR(32)	Случайная соль
role_id	INTEGER, FK	Роль пользователя
is_blocked	BOOLEAN	Признак блокировки
created_at	TIMESTAMP	Дата и время регистрации

Таблица 1.4 – Структура таблицы clients (клиенты)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор клиента
user_id	INTEGER, FK, UNIQUE	Учётная запись клиента
last_name, first_name, patronymic	VARCHAR(50)	Фамилия, имя, отчество
birth_date	DATE	Дата рождения (для возрастного коэффициента)
passport_no	VARCHAR(20), UNIQUE	Номер паспорта
phone	VARCHAR(20)	Контактный телефон
address	VARCHAR(200)	Адрес проживания

Таблица 1.5 – Структура таблицы agents (страховые агенты)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор агента
user_id	INTEGER, FK, UNIQUE	Учётная запись агента
last_name, first_name, patronymic	VARCHAR(50)	Фамилия, имя, отчество
position	VARCHAR(60)	Должность
hire_date	DATE	Дата приёма на работу
commission_rate	NUMERIC(5,4)	Ставка агентского вознаграждения

Таблица 1.6 – Структура таблицы insurance_types (виды страхования)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор вида страхования
name	VARCHAR(80), UNIQUE	Наименование
category	VARCHAR(20)	Категория: PROPERTY, CASCO, ACCIDENT, TRAVEL; определяет стратегию расчёта премии
description	TEXT	Описание условий
base_rate	NUMERIC(6,4)	Базовый годовой тариф, доля страховой суммы
min_sum, max_sum	NUMERIC(12,2)	Допустимый диапазон страховой суммы, BYN
max_term_months	INTEGER	Максимальный срок страхования в месяцах
is_active	BOOLEAN	Доступен ли вид для новых заявок

Таблица 1.7 – Структура таблицы applications (заявки)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор заявки
client_id, type_id	INTEGER, FK	Клиент и вид страхования
agent_id	INTEGER, FK, NULL	Агент, рассмотревший заявку
requested_sum	NUMERIC(12,2)	Запрошенная страховая сумма
term_months	INTEGER	Срок страхования в месяцах
franchise_percent	INTEGER	Франшиза: 0, 1, 2 или 5 %
installments	INTEGER	Число взносов: 1, 2, 4 или 12
object_description	TEXT	Описание объекта страхования
status	VARCHAR(20)	NEW, APPROVED, REJECTED
decision_comment	TEXT	Комментарий агента
created_at, decided_at	TIMESTAMP	Даты подачи и рассмотрения

Таблица 1.8 – Структура таблицы policies (договоры страхования)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор договора
policy_number	VARCHAR(20), UNIQUE	Номер полиса
application_id	INTEGER, FK, UNIQUE, NULL	Заявка-основание
client_id, agent_id, type_id	INTEGER, FK	Страхователь, агент, вид страхования
insured_sum	NUMERIC(12,2)	Страховая сумма
premium	NUMERIC(12,2)	Страховая премия
franchise	NUMERIC(12,2)	Франшиза в денежном выражении
start_date, end_date	DATE	Период действия

Продолжение таблицы 1.8

Поле	Тип	Назначение
status	VARCHAR(20)	WAITING_PAYMENT, ACTIVE, SUSPENDED, EXPIRED, TERMINATED, FULFILLED
created_at, terminated_at	TIMESTAMP	Даты оформления и расторжения
refund_amount	NUMERIC(12,2)	Сумма возврата при расторжении

Таблица 1.9 – Структура таблицы payments (страховые взносы)

Поле	Тип	Назначение
policy_id	INTEGER, PK, FK	Договор
installment_no	INTEGER, PK	Номер взноса в графике
amount	NUMERIC(12,2)	Сумма взноса
due_date	DATE	Срок уплаты
paid_date	DATE, NULL	Фактическая дата уплаты

Таблица 1.10 – Структура таблицы claims (страховые случаи)

Поле	Тип	Назначение
id	SERIAL, PK	Идентификатор страхового случая
policy_id	INTEGER, FK	Договор
incident_date, report_date	DATE	Даты события и заявления
description	TEXT	Описание события
damage_amount	NUMERIC(12,2)	Заявленный клиентом размер ущерба
assessed_damage	NUMERIC(12,2), NULL	Размер ущерба, оценённый агентом (акт осмотра)
status	VARCHAR(20)	REPORTED, APPROVED, REJECTED
payout_amount	NUMERIC(12,2)	Сумма страховой выплаты (0 при отказе)
agent_id	INTEGER, FK, NULL	Агент, принявший решение
decision_comment	TEXT	Обоснование решения
decided_at	TIMESTAMP	Дата решения

1.5 UML-модели представления программного средства и их описание

Для описания программного средства с разных точек зрения построены UML-диаграммы [8]: вариантов использования, состояний, последовательности, классов и развёртывания.

Диаграмма вариантов использования (рисунок 1.8) показывает функции, доступные каждой роли. Абстрактный актёр «Пользователь» обобщает

клиента, агента и администратора и связан с вариантом «Авторизация». Варианты «Рассмотрение заявки и оформление полиса» и «Урегулирование страхового случая» включают (include) расчёт стоимости страхования, формирование графика взносов и расчёт страховой выплаты, так как эти операции выполняются автоматически. Выгрузка отчёта в файл расширяет (extend) варианты построения отчётов. Всего программное средство поддерживает семнадцать вариантов использования, из них двенадцать реализуют нетривиальную бизнес-логику.

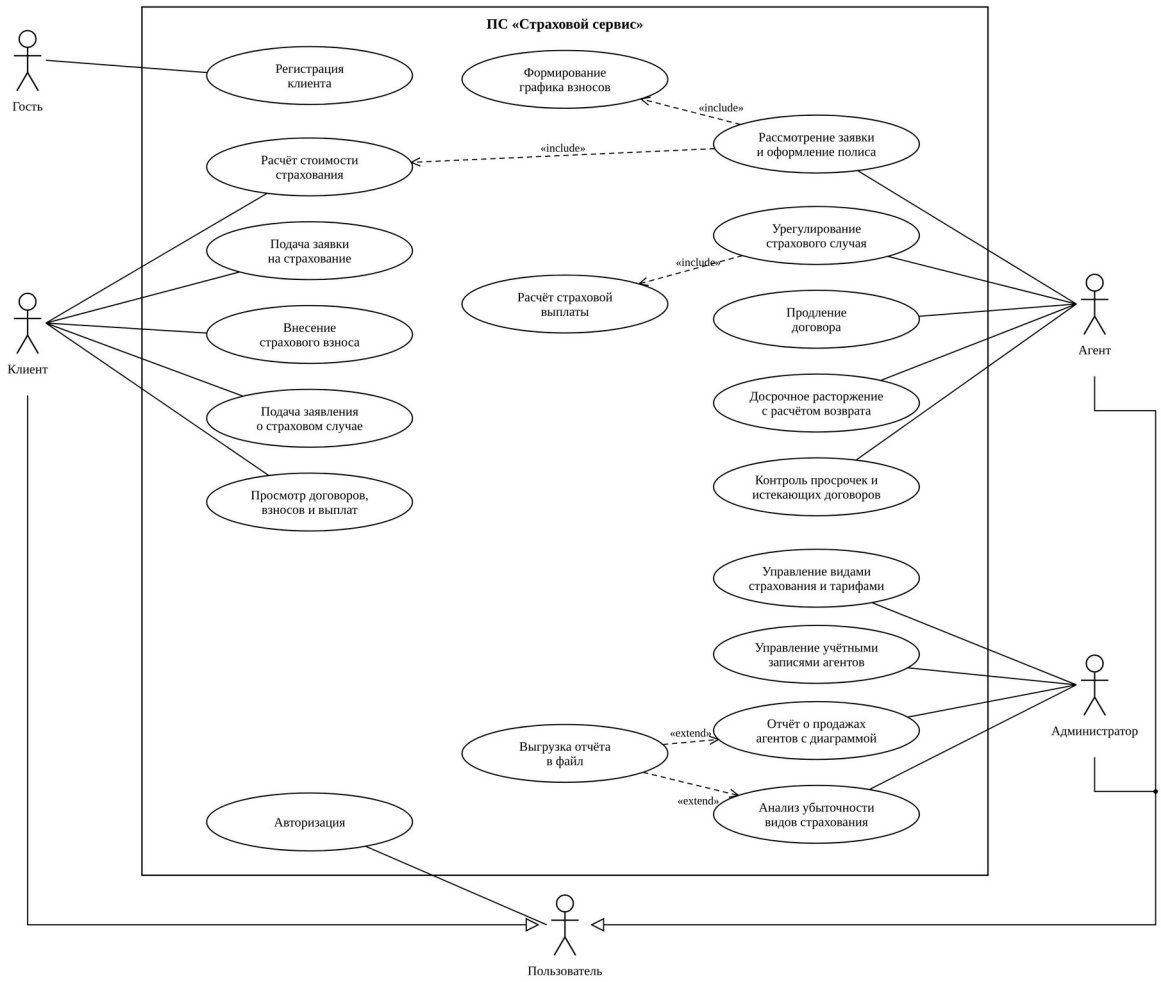


Рисунок 1.8 – Диаграмма вариантов использования

Договор страхования является центральным объектом системы, поэтому для него построена диаграмма состояний (рисунок 1.9). Оформленный договор ожидает оплаты первого взноса и после его внесения начинает действовать. При просрочке очередного взноса договор приостанавливается, а при просрочке более 30 дней расторгается. Действующий договор может быть досрочно расторгнут с возвратом части премии, исполнен при исчерпании страховой суммы выплатами или истечь по окончании срока. Продление не

меняет состояние истёкшего договора: оформляется новый договор, который начинается со дня, следующего за окончанием прежнего. Переходы, зависящие от времени, выполняет сервер: при запуске и каждые шесть часов он пересчитывает состояния всех договоров.

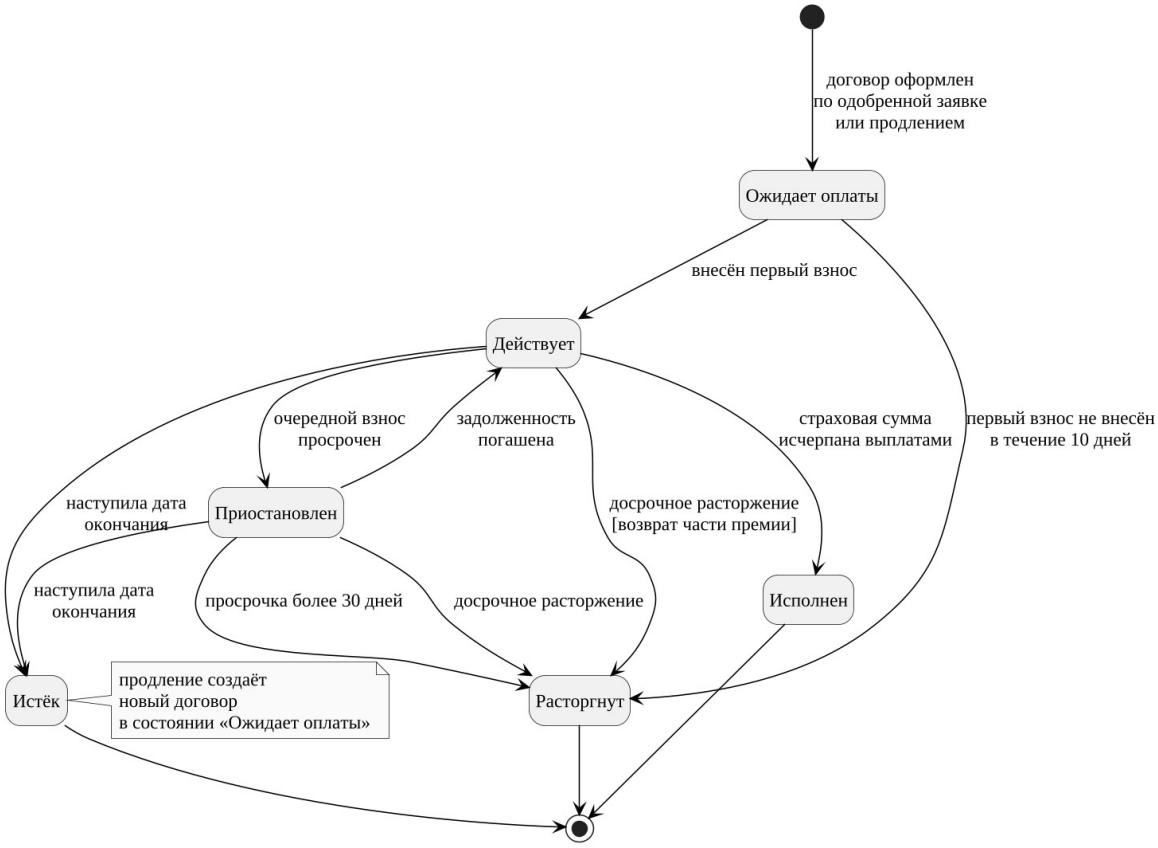


Рисунок 1.9 – Диаграмма состояний договора страхования

Диаграмма последовательности (рисунок 1.10) описывает взаимодействие объектов при оформлении полиса агентом. Панель заявок клиентского приложения отправляет серверу сериализованный запрос APPROVE_APPLICATION, который обрабатывается в потоке пула ClientHandler. Диспетчер команд CommandDispatcher проверяет, что пользователь авторизован и имеет роль агента, и вызывает сервис ApplicationService. Сервис в одной транзакции блокирует строку заявки (SELECT … FOR UPDATE), проверяет её статус, с помощью PremiumCalculator рассчитывает премию и сохраняет договор, график взносов и новое состояние заявки. Если заявку уже рассмотрел другой агент, клиенту возвращается сообщение об ошибке, а транзакция откатывается.

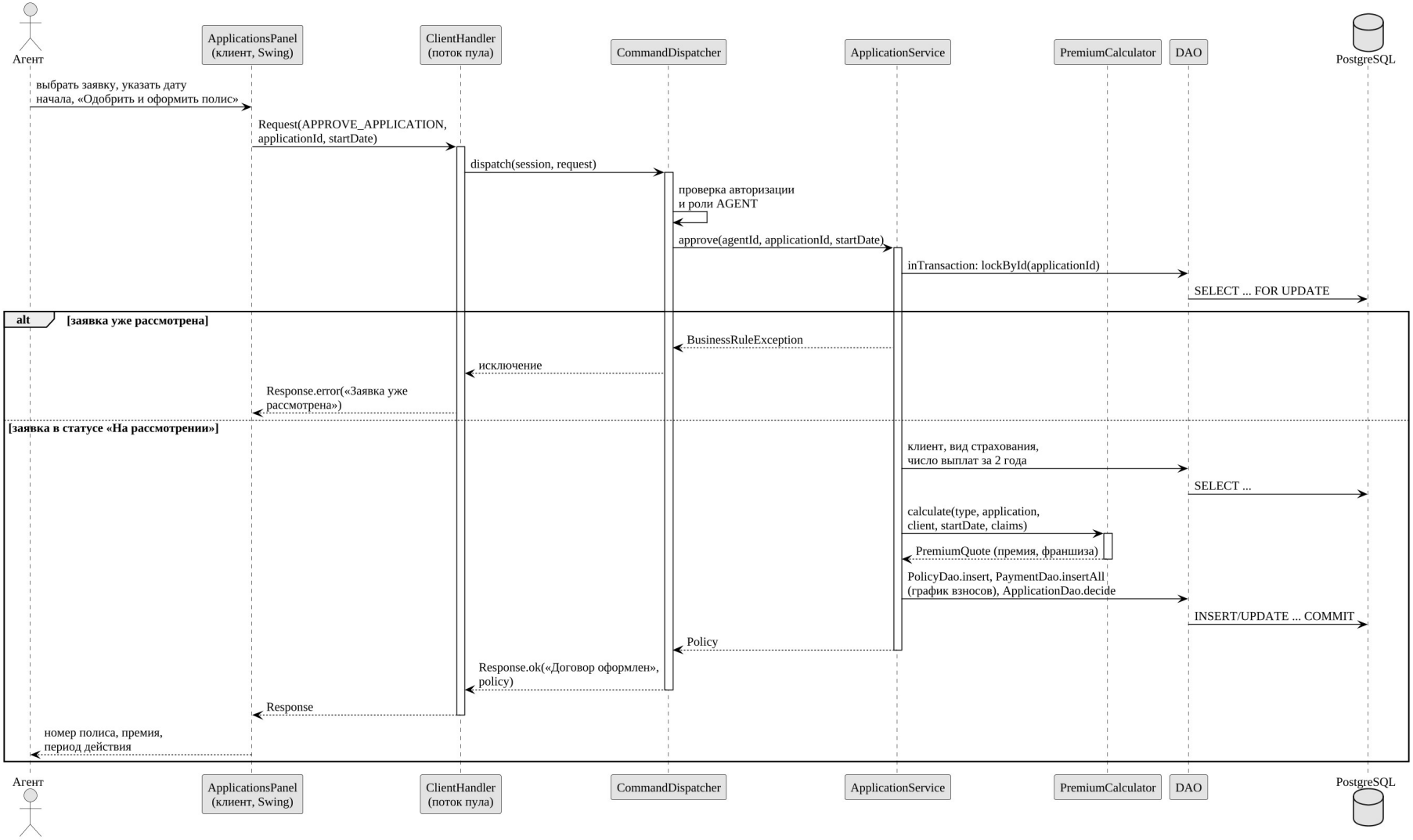


Рисунок 1.10 – Диаграмма последовательности оформления полиса

Диаграмма классов (рисунок 1.11) отражает модель данных из пакета by.bsuir.insurance.common.model, общую для клиента и сервера. Все сущности наследуются от абстрактного сериализуемого класса Entity, который хранит идентификатор и переопределяет методы equals и hashCode; клиент и агент имеют общий абстрактный предок Person с полями фамилии, имени и отчества. Состояния заявки, договора и страхового случая, роли и категории видов страхования представлены перечислениями с русскими названиями для интерфейса. Договор связан композицией с графиком взносов и агрегацией со страховыми случаями. Помимо полей таблиц, классы содержат вычисляемые поля для отображения (ФИО клиента, наименование вида страхования, уплаченная сумма), которые сервер заполняет при выборке. Классы серверной части описаны в подразделе 2.2.

Диаграмма развёртывания (рисунок 1.12) показывает физическую конфигурацию системы. Клиентское приложение запускается на компьютерах пользователей и взаимодействует с сервером приложений по протоколу TCP/IP. Сервер приложений обращается к СУБД PostgreSQL через JDBC-драйвер, который включён в исполняемый архив сервера. Настройки сервера и клиента хранятся в файлах server.properties и client.properties. Сервер приложений и сервер базы данных могут располагаться как на разных компьютерах, так и на одном.

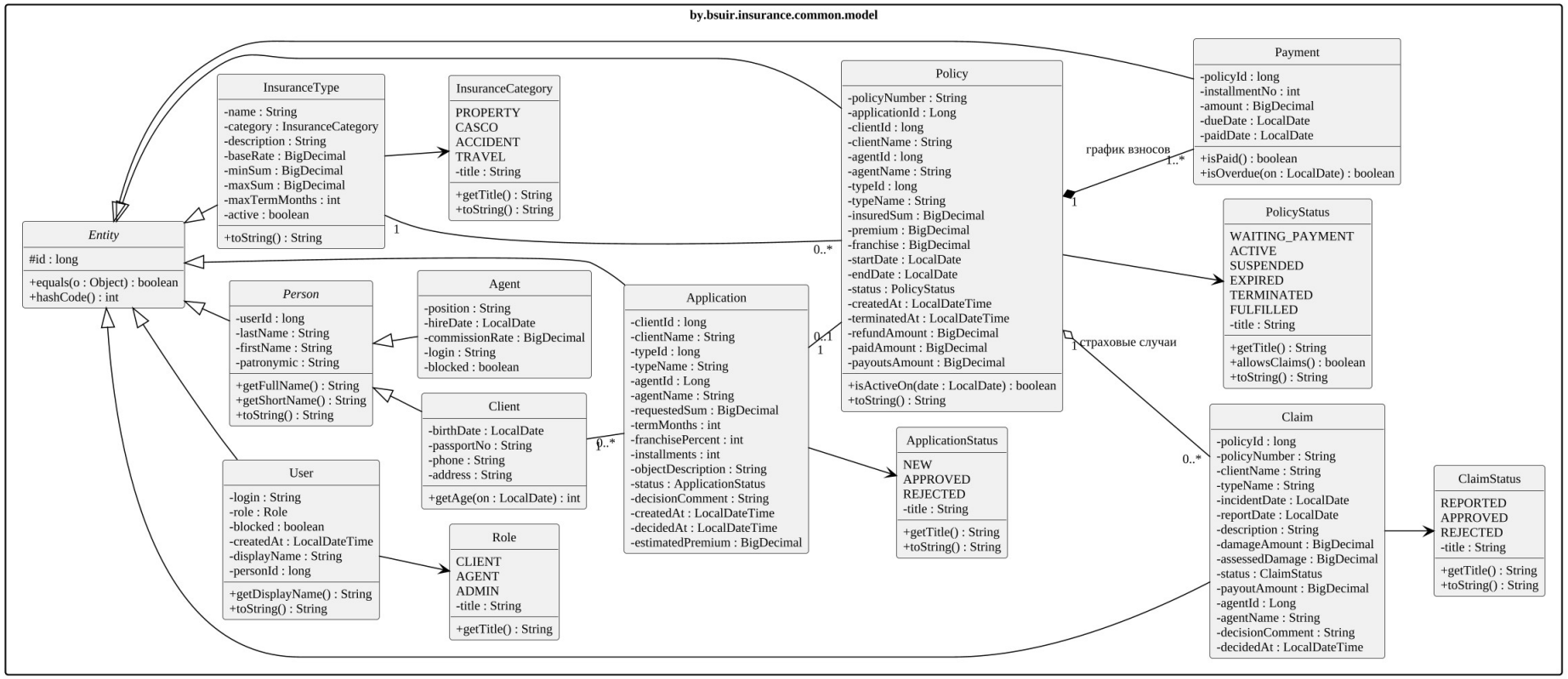


Рисунок 1.11 – Диаграмма классов модели данных

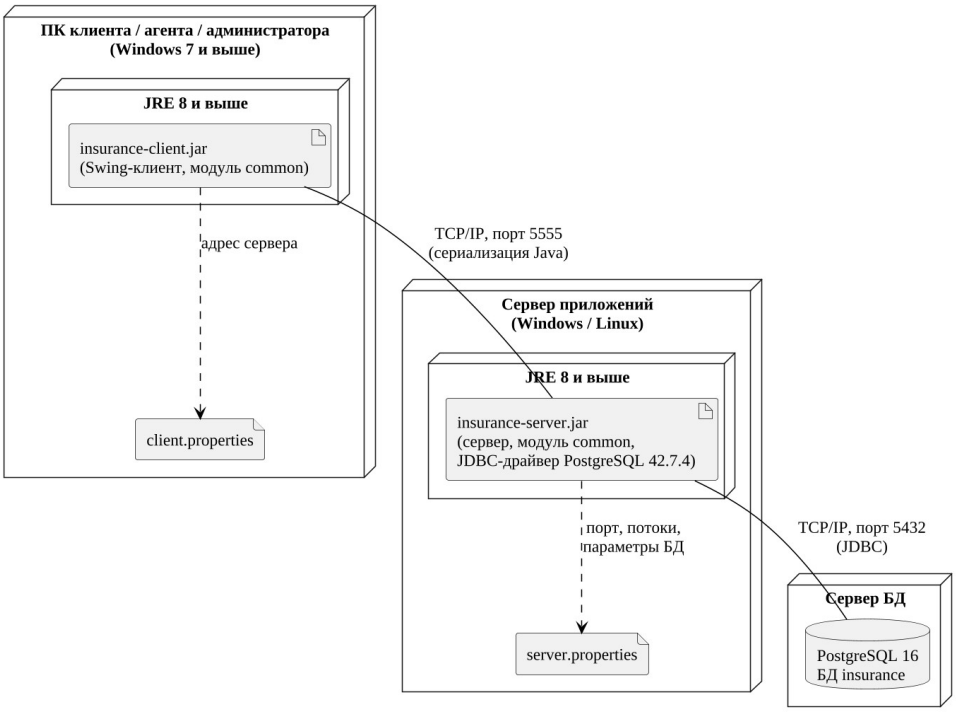


Рисунок 1.12 – Диаграмма развёртывания

2 ПРОЕКТИРОВАНИЕ И КОНСТРУИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Постановка задачи

Необходимо разработать программное средство «Страховой сервис», автоматизирующее процесс оказания страховых услуг физическим лицам: расчёт стоимости страхования, приём и рассмотрение заявок, оформление договоров с графиком страховых взносов, учёт оплаты, урегулирование страховых случаев, продление и расторжение договоров, а также подготовку отчётности для руководства.

Для реализации поставленной цели курсового проекта необходимо выполнить следующие задачи:

– изучить предметную область, а именно порядок заключения и сопровождения договоров добровольного страхования и урегулирования страховых случаев;

– реализовать клиент-серверное приложение с многопоточным сервером, обрабатывающим запросы нескольких клиентов параллельно;

– разработать базу данных, приведённую к третьей нормальной форме, и связать с ней сервер через JDBC-драйвер PostgreSQL;

– реализовать на сервере алгоритмы расчёта страховой премии, графика взносов, страховой выплаты и возврата части премии;

– создать оконные формы для трёх ролей пользователей и обеспечить авторизацию и разграничение доступа;

– предоставить администратору аналитическую информацию в виде таблиц и диаграмм с возможностью выгрузки в файл;

– сделать приложение простым и понятным в использовании, обеспечить понятные сообщения об ошибках;

– протестировать полученное приложение.

Разработка системы предусмотрена для трёх ролей: клиента, страхового агента и администратора. Незарегистрированный пользователь может только зарегистрироваться в качестве клиента.

Со стороны клиента программа должна обеспечивать следующие возможности:

– регистрация и вход в систему (UC1, UC2);

– расчёт стоимости страхования с отображением всех поправочных коэффициентов (UC3);

– подача заявки на страхование (UC4) и просмотр состояния заявок;

– просмотр своих договоров и графика взносов (UC12), внесение очередного взноса (UC6);

– подача заявления о страховом случае и просмотр решения по нему (UC7).

Со стороны страхового агента программа должна обеспечивать следующие возможности:

– рассмотрение заявок с предварительным расчётом премии, оформление договора или отказ с указанием причины (UC5);

– урегулирование страховых случаев с автоматической проверкой условий и расчётом выплаты (UC8);

– продление договоров (UC9) и досрочное расторжение с расчётом возврата части премии (UC10);

– контроль просроченных взносов и договоров, срок действия которых истекает (UC11).

Со стороны администратора программа должна обеспечивать следующие возможности:

– управление видами страхования и тарифами (UC13);

– создание учётных записей агентов, блокировка и разблокировка пользователей (UC14);

– отчёт о продажах агентов за период (UC15) и анализ убыточности видов страхования (UC16) с диаграммами;

– выгрузка отчётов в текстовый файл (UC17).

Входные и выходные данные основных функций программного средства приведены в таблице 2.1.

Таблица 2.1 – Входные и выходные данные основных функций

Функция	Входные данные	Выходные данные
Расчёт стоимости (UC3)	Вид страхования, страховая сумма, срок, франшиза, число взносов, описание объекта; дата рождения клиента и история выплат из БД	Страховая премия, франшиза в BYN, размер взноса, значения коэффициентов
Оформление договора (UC5)	Заявка, дата начала действия	Договор с номером полиса, график взносов
Внесение взноса (UC6)	Договор	Отметка об оплате, новое состояние договора
Урегулирование случая (UC8)	Заявление, оценённый агентом ущерб, договор, график взносов, предыдущие выплаты	Решение (выплата или отказ), сумма выплаты, обоснование по шагам
Расторжение (UC10)	Договор, уплаченные взносы, произведённые выплаты, дата расторжения	Сумма возврата и её расчёт, состояние «Расторгнут»
Контроль (UC11)	Текущая дата, графики взносов, даты окончания договоров	Список просроченных взносов и истекающих договоров с телефонами клиентов

Продолжение таблицы 2.1

Функция	Входные данные	Выходные данные
Отчёты (UC15, UC16)	Период отчёта	Таблица и диаграмма; текстовый файл при выгрузке

Ограничения: программное средство должно работать в операционной системе Windows 7 и выше при установленной среде выполнения Java версии 8 или новее; сервер и СУБД могут располагаться на отдельном компьютере локальной сети. Все денежные величины выражаются в белорусских рублях с точностью до копеек.

2.2 Архитектурные решения

Программное средство построено по трёхуровневой архитектуре «клиент – сервер приложений – сервер базы данных» (рисунок 2.1). Уровень представления реализован оконным клиентским приложением на Swing, уровень бизнес-логики – многопоточным сервером, уровень данных – СУБД PostgreSQL. Клиент не обращается к базе данных напрямую: вся бизнес-логика и все проверки выполняются на сервере, а клиент только формирует запросы и отображает результаты. Это исключает расхождение правил расчёта на разных рабочих местах и позволяет изменять тарифы и алгоритмы без обновления клиентских приложений.

Исходный код организован как многомодульный проект Maven из трёх модулей: common (общий код клиента и сервера), server и client. Назначение пакетов приведено в таблице 2.2.

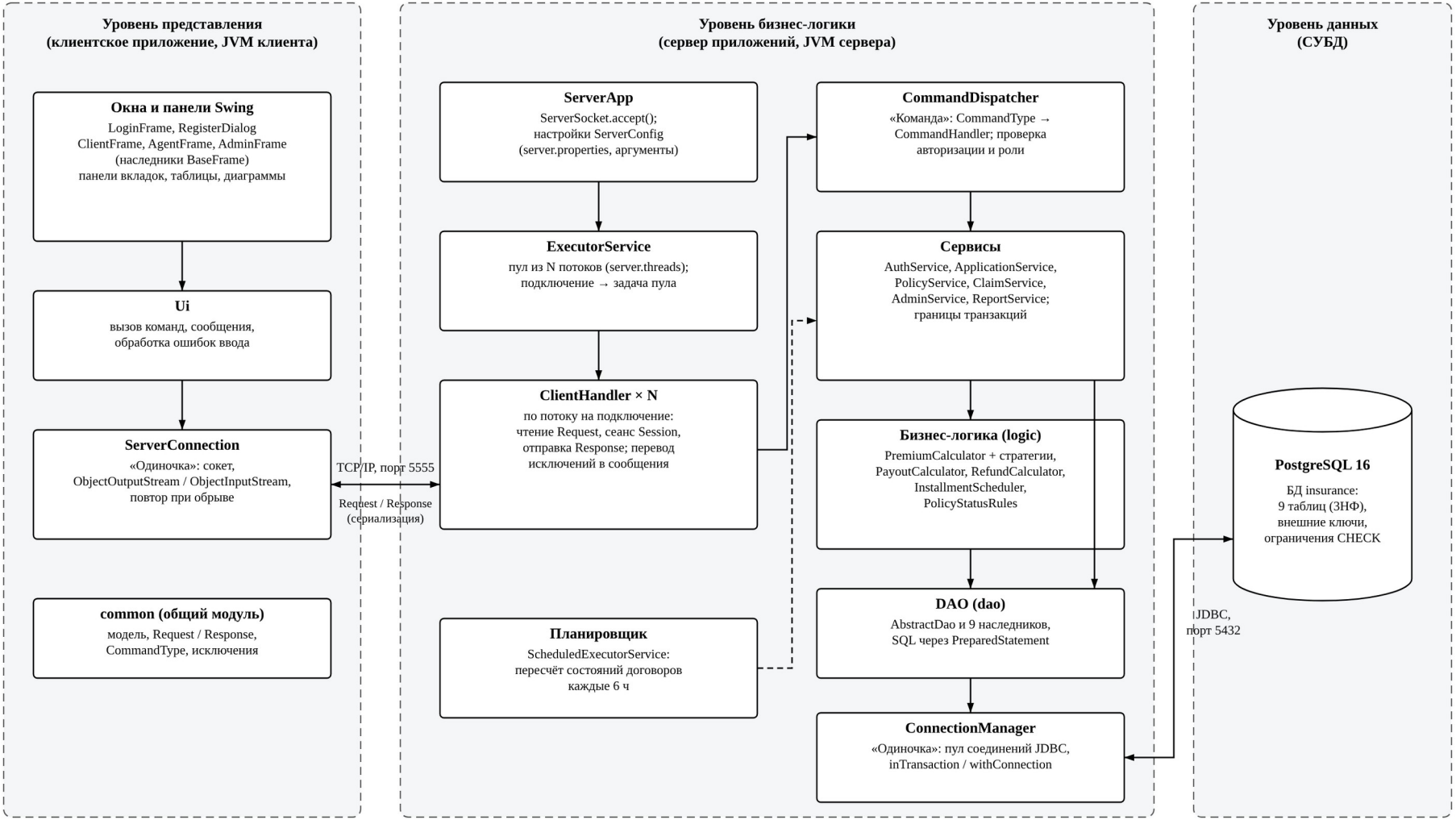


Рисунок 2.1 – Архитектура программного средства

Таблица 2.2 – Модули и пакеты программного средства

Пакет	Назначение и основные классы
common.model	Сущности предметной области (Entity, Person, Client, Agent, User, InsuranceType, Application, Policy, Payment, Claim), перечисления состояний и объекты результатов расчётов (PremiumQuote, PayoutDecision, Refund Calculation, SalesReportRow, LossRatioRow, ControlItem)
common.protocol	Протокол обмена: Request, Response, перечисление команд CommandType с ролями доступа
common.exception	Иерархия исключений InsuranceException: Validation Exception, AuthException, BusinessRuleException
common.util	Money – округление и форматирование денежных сумм
server	ServerApp – точка входа, ServerConfig – настройки
server.net	ClientHandler – обслуживание подключения, Session – сеанс, CommandDispatcher и CommandHandler – диспетчеризация команд
server.service	Сервисы вариантов использования: AuthService, ApplicationService, PolicyService, ClaimService, AdminService, ReportService, Validator
server.logic	Алгоритмы бизнес-логики: PremiumCalculator и стратегии, InstallmentScheduler, PayoutCalculator, RefundCalculator, PolicyStatusRules
server.dao, server.db	Доступ к данным: AbstractDao и девять наследников, пул соединений ConnectionManager
server.security	PasswordUtil – хеширование паролей
client, client.net	ClientApp – точка входа, ServerConnection – соединение с сервером
client.ui и подпакеты	LoginFrame, RegisterDialog, BaseFrame и окна ролей, панели вкладок, компоненты EntityTable, Form, BarChartPanel, PieChartPanel

2.2.1 Протокол обмена. Клиент и сервер обмениваются объектами Request и Response по протоколу TCP/IP; объекты передаются средствами стандартной сериализации Java через потоки ObjectOutputStream и ObjectInputStream [9]. Запрос содержит тип команды и именованные параметры (идентификаторы, даты, суммы, объекты модели), ответ – признак успеха, текстовое сообщение и данные результата. Для каждой команды в перечислении CommandType указан набор ролей, которым она доступна; пустой набор означает команду, доступную без авторизации. Перечень команд приведён в таблице 2.3.

Таблица 2.3 – Команды протокола обмена

Команда	Роль	Назначение
LOGIN, REGISTER_CLIENT	Все, без авторизации	Вход в систему, регистрация клиента
LOGOUT	Клиент, агент, администратор	Завершение сеанса
GET_ACTIVE_TYPES	Клиент, агент, администратор	Действующие виды страхования
CALCULATE_PREMIUM	Клиент, агент	Расчёт страховой премии (UC3)
GET_PAYMENTS	Клиент, агент, администратор	График взносов договора (UC12)
SUBMIT_APPLICATION	Клиент	Подача заявки (UC4)
GET_MY_APPLICATIONS, GET_MY_POLICIES, GET_MY_CLAIMS, GET_MY_PROFILE	Клиент	Заявки, договоры, страховые случаи и профиль клиента
PAY_INSTALLMENT	Клиент	Внесение очередного взноса (UC6)
REPORT_CLAIM	Клиент	Заявление о страховом случае (UC7)
GET_NEW_APPLICATIONS	Агент	Заявки на рассмотрении с расчётной премией
APPROVE_APPLICATION, REJECT_APPLICATION	Агент	Оформление договора или отказ (UC5)
GET_POLICIES	Агент, администратор	Договоры с фильтром по состоянию
RENEW_POLICY	Агент	Продление договора (UC9)
PREVIEW_REFUND, TERMINATE_POLICY	Агент	Расчёт возврата и расторжение (UC10)
GET_PENDING_CLAIMS, GET_ALL_CLAIMS	Агент, администратор	Страховые случаи
PREVIEW_PAYOUT, SETTLE_CLAIM	Агент	Расчёт выплаты и решение по случаю (UC8)
GET_CONTROL_LIST, REFRESH_STATUSES	Агент, администратор	Контроль просрочек, пересчёт состояний (UC11)
GET_ALL_TYPES, SAVE_TYPE	Администратор	Виды страхования и тарифы (UC13)
GET_AGENTS, CREATE_AGENT, GET_USERS, SET_USER_BLOCKED	Администратор	Учётные записи (UC14)
SALES_REPORT, LOSS_RATIO_REPORT	Администратор	Отчёты за период (UC15, UC16)

2.2.2 Многопоточность. Главный поток сервера ServerApp принимает подключения методом ServerSocket.accept() и передаёт каждое из них в пул потоков фиксированного размера (Executors.newFixedThreadPool), размер

которого задаётся параметром server.threads. Подключение обслуживается задачей ClientHandler, которая в цикле читает запросы, выполняет их и отправляет ответы, пока клиент не отключится. У каждого подключения свой объект Session с данными авторизованного пользователя, поэтому сеансы разных клиентов изолированы друг от друга [10].

Диспетчер команд, сервисы и классы бизнес-логики не хранят изменяемого состояния и используются всеми потоками совместно. Соединения с базой данных выдаёт пул ConnectionManager на основе блокирующей очереди ArrayBlockingQueue: поток берёт соединение, выполняет запросы и возвращает его; при отсутствии свободных соединений поток ожидает до 10 с. Операции, изменяющие несколько таблиц, выполняются в транзакциях (метод inTransaction): при успехе – COMMIT, при исключении – ROLLBACK. Чтобы два агента не могли одновременно оформить одну заявку или принять два решения по одному страховому случаю, сервисы блокируют обрабатываемую строку запросом SELECT … FOR UPDATE [11]: второй поток дожидается завершения первой транзакции и получает сообщение о том, что заявка уже рассмотрена.

Отдельный поток ScheduledExecutorService при запуске сервера и затем каждые шесть часов пересчитывает состояния договоров по правилам диаграммы состояний (рисунок 1.9). Агент может выполнить пересчёт вручную на вкладке «Контроль».

2.2.3 Настройка. Параметры сервера задаются без изменения исходного кода: значения по умолчанию хранятся в ресурсе server.properties внутри архива, их переопределяют файл server.properties в рабочем каталоге (или файл, указанный аргументом --config=путь) и аргументы командной строки вида --ключ=значение. Параметры приведены в таблице 2.4. Клиент читает адрес сервера из файла client.properties (параметры server.host и server.port) или из аргументов запуска.

Таблица 2.4 – Параметры сервера

Параметр	По умолчанию	Назначение
server.port	5555	Порт, на котором сервер принимает подключения
server.threads	20	Число потоков пула, равное числу одновременно обслуживаемых клиентов
db.url	jdbc:postgresql://localhost:5432/insurance	Адрес базы данных
db.user	postgres	Пользователь СУБД
db.password	Не задан	Пароль пользователя СУБД
db.pool.size	10	Число соединений в пуле

2.2.4 Безопасность. Все пользователи проходят авторизацию; пароли хранятся в виде хеша SHA-256 от строки, составленной из случайной соли длиной 16 байт и пароля, а сравнение хешей выполняется за постоянное время. Права проверяются на сервере для каждой команды по роли из сеанса, поэтому изменение клиентского приложения не даёт доступа к чужим функциям. Клиент может получить только свои договоры и страховые случаи; заблокированный пользователь не может войти в систему. Все SQL-запросы выполняются через PreparedStatement с параметрами, что исключает SQL-инъекции.

2.2.5 Обработка ошибок. Нарушения правил предметной области сервисы сообщают исключениями иерархии InsuranceException: ValidationException – некорректные входные данные, AuthException – ошибка авторизации или прав, BusinessRuleException – нарушение бизнес-правила. ClientHandler преобразует такие исключения в ответ с текстом ошибки; ошибки базы данных записываются в журнал сервера, а клиенту возвращается обобщённое сообщение. Клиентское приложение показывает сообщения в диалоговых окнах; ошибки формата ввода (дата, число) выявляются ещё до отправки запроса.

2.2.6 Объектно-ориентированные техники. Требования задания к использованию средств объектно-ориентированного программирования выполнены, как показано в таблице 2.5.

Таблица 2.5 – Реализация требований к объектно-ориентированным техникам

Требование	Реализация
Собственная иерархия классов (не менее 5)	Entity → Person → Client, Agent; Entity → User, InsuranceType, Application, Policy, Payment, Claim (10 классов); InsuranceException → ValidationException, AuthException, BusinessRuleException; AbstractDao → 9 DAO; PremiumStrategy → AbstractPremiumStrategy → 4 стратегии
Расширение классов JDK	BaseFrame extends JFrame, RegisterDialog extends JDialog, панели extends JPanel, EntityTable extends JTable, EntityTable Model extends AbstractTableModel, InsuranceException extends Exception
Инкапсуляция	Закрытые поля сущностей с методами доступа; закрытые конструкторы одиночек
Перегрузка методов	PremiumCalculator.calculate (полный и упрощённый расчёт), Ui.format(LocalDate) и Ui.format(LocalDateTime), Response.ok(data) и Response.ok(message, data)
Переопределение методов	toString, equals, hashCode в сущностях; extraFactor, riskKeywords, riskSurcharge в стратегиях; refresh в панелях вкладок

Продолжение таблицы 2.5

Требование	Реализация
Сериализация	Сущности, Request и Response реализуют Serializable и передаются по сети
Абстрактные типы данных	Интерфейсы PremiumStrategy, CommandHandler, SqlFunction; абстрактные классы Entity, Person, AbstractPremiumStrategy, AbstractDao, BaseFrame, RefreshablePanel
Статические методы	PasswordUtil.hash, Money.round и Money.format, Installment Scheduler.build, PolicyStatusRules.evaluate, PremiumStrategy Factory.forType, фабрики Response.ok и Response.error
Обработка исключений	Иерархия InsuranceException; перехват в ClientHandler; откат транзакций; повторное подключение клиента при обрыве связи

2.2.7 Паттерны проектирования [12]. В программном средстве применены следующие паттерны:

– «Одиночка» – ServerConfig, ConnectionManager и ServerConnection существуют в единственном экземпляре и доступны через статический метод getInstance();

– «Стратегия» – расчёт коэффициента вида страхования вынесен в интерфейс PremiumStrategy с реализациями для каждой категории страхования; добавление нового вида не требует изменения калькулятора;

– «Фабрика» – PremiumStrategyFactory выбирает стратегию по категории вида страхования;

– «Шаблонный метод» – AbstractPremiumStrategy.typeFactor задаёт общий порядок расчёта, а наследники переопределяют шаги extraFactor, riskKeywords и riskSurcharge;

– «Команда» – каждая команда протокола обрабатывается объектом CommandHandler; CommandDispatcher хранит соответствие «тип команды – обработчик» и проверяет права;

– «Объект доступа к данным» (DAO) – SQL-запросы инкапсулированы в классах-наследниках AbstractDao.

Классы серверной части и их связи показаны на рисунке 2.2 в нотации UML [13], классы бизнес-логики – на рисунке 2.3. Для краткости на диаграммах приведены только основные методы.

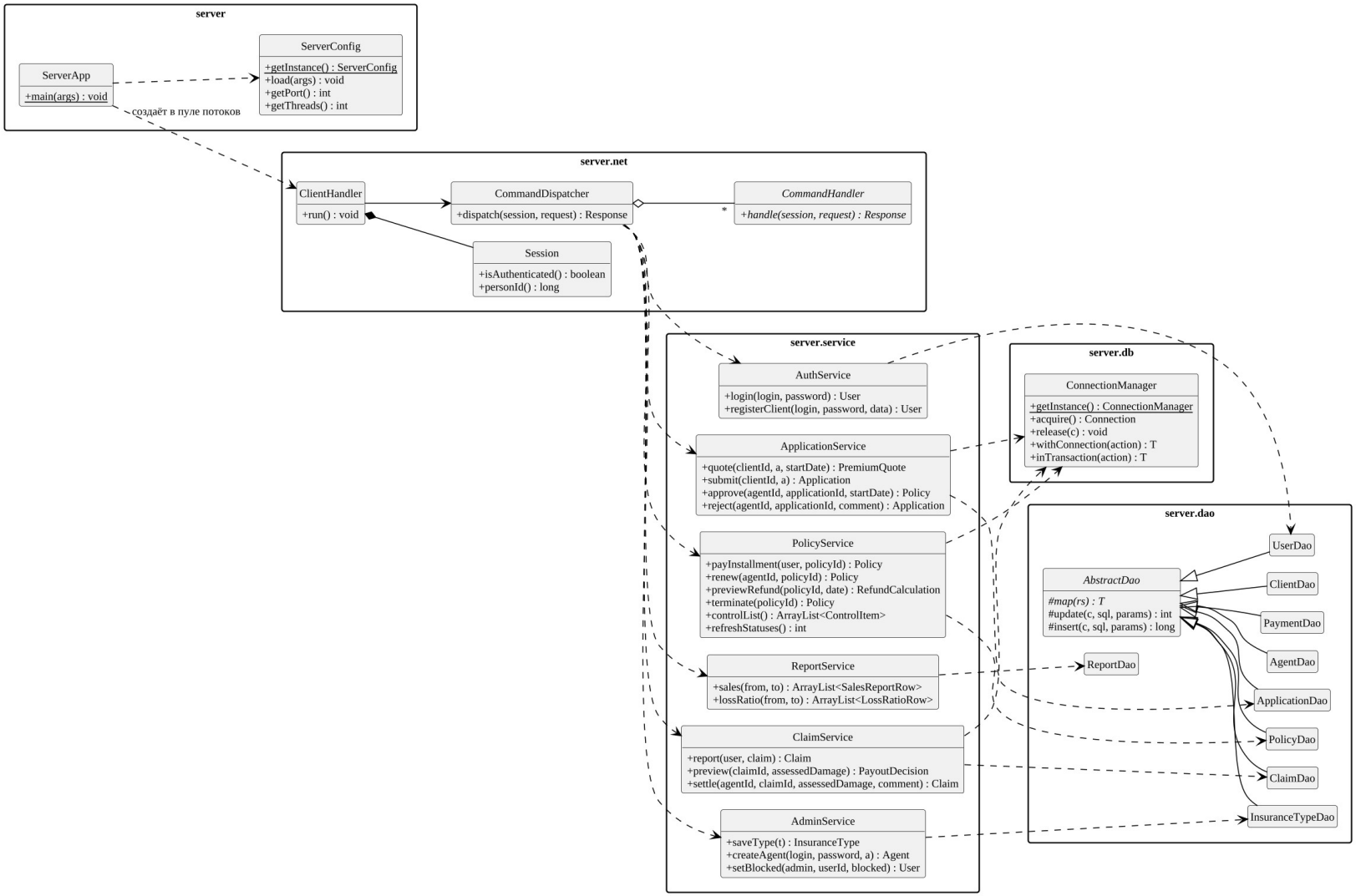


Рисунок 2.2 – Диаграмма классов серверной части

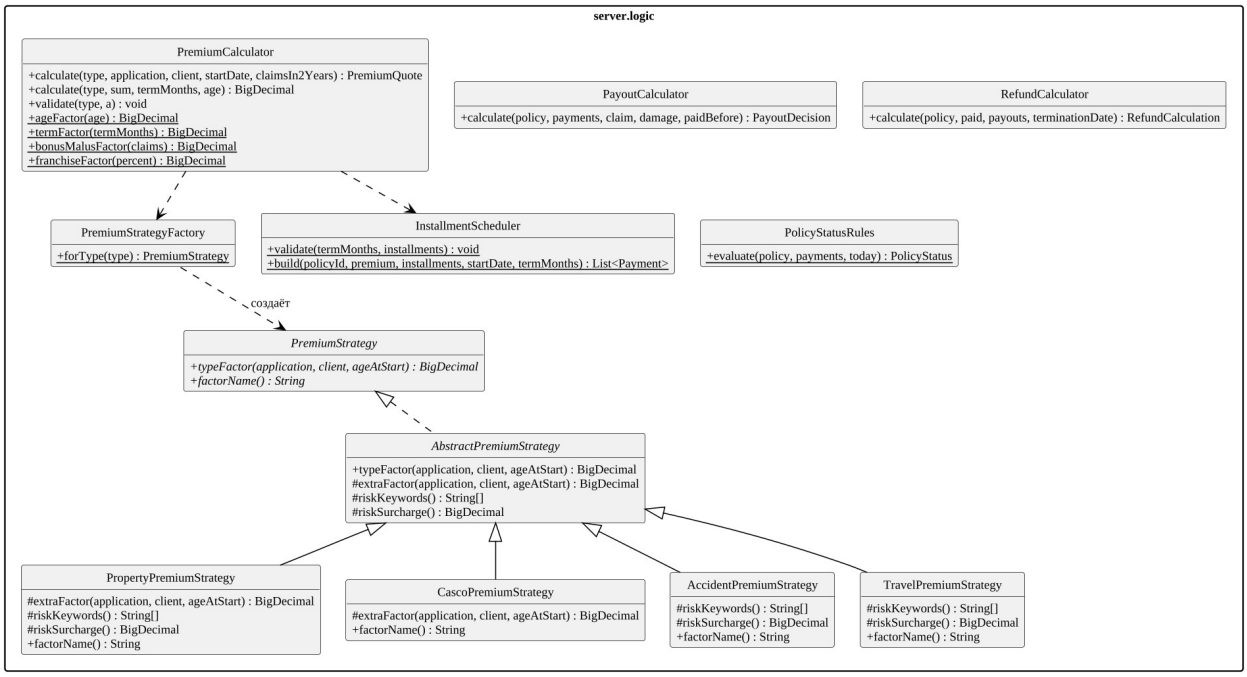


Рисунок 2.3 – Диаграмма классов бизнес-логики (пакет server.logic)

Преимущества и недостатки принятых архитектурных решений приведены в таблице 2.6.

Таблица 2.6 – Преимущества и недостатки архитектурных решений

Решение	Преимущества	Недостатки
Бизнес-логика только на сервере	Единые правила расчёта для всех рабочих мест; клиент нельзя использовать для обхода проверок; изменение тарифов без обновления клиентов	Каждое действие требует обращения к серверу; без сервера клиент не работает
Поток пула на подключение	Простая модель: сеанс и потоки ввода-вывода принадлежат одному потоку; число потоков ограничено настройкой	Число одновременно обслуживаемых клиентов равно размеру пула, остальные ожидают освобождения потока
Сериализация Java по TCP	Не требуются сторонние библиотеки и описание схемы сообщений; объекты модели передаются без преобразования	Клиент и сервер должны иметь совместимые версии классов; протокол непригоден для клиентов на других языках
JDBC без ORM	Полный контроль над SQL, блокировками и транзакциями; нет зависимостей и накладных расходов фреймворка	Ручное сопоставление строк результата с объектами (метод map в каждом DAO)
Пул соединений	Нет затрат на установку соединения при каждом запросе; ограничение нагрузки на СУБД	Соединения должны проверяться перед выдачей (метод isValid)

2.3 Описание алгоритмов, реализующих ключевую бизнес-логику разрабатываемого программного средства

Ключевая бизнес-логика программного средства сосредоточена в пакете server.logic. Схемы алгоритмов выполнены в соответствии с ГОСТ 19.701–90 [14]. Программный код классов, реализующих описанные алгоритмы, приведён в приложении Б.

2.3.1 Алгоритм расчёта страховой премии реализован классом PremiumCalculator. Страховая премия рассчитывается по формуле

$$П = С \times Т \times К_в \times К_с \times К_{бм} \times К_ф \times К_т, \quad (2.1)$$

где П – страховая премия, BYN;
С – страховая сумма, BYN;
Т – базовый годовой тариф вида страхования, доля страховой суммы;
$К_в$ – коэффициент возраста страхователя на дату начала договора;
$К_с$ – коэффициент срока страхования;
$К_{бм}$ – коэффициент бонус-малус по числу страховых выплат клиенту за два года;
$К_ф$ – коэффициент франшизы;
$К_т$ – коэффициент вида страхования, рассчитываемый стратегией.

Значения коэффициентов $К_в$, $К_с$, $К_{бм}$ и $К_ф$ приведены в таблице 2.7. Для договоров сроком меньше года к коэффициенту срока применяется надбавка 10 %, поскольку краткосрочное страхование дороже в расчёте на месяц.

Таблица 2.7 – Поправочные коэффициенты страховой премии

Коэффициент	Условие	Значение
$К_в$ (возраст страхователя)	До 25 лет	1,20
	От 25 до 60 лет включительно	1,00
	Старше 60 лет	1,35
$К_с$ (срок m в месяцах)	$m \geq 12$	$m / 12$
	$m < 12$	$m / 12 \times 1{,}1$
$К_{бм}$ (число выплат k)	$k = 0$	0,90
	$k = 1$	1,00
	$k \geq 2$	1,25
$К_ф$ (франшиза f)	$f = 0$ %	1,00
	$f = 1$ %	0,95
	$f = 2$ %	0,90
	$f = 5$ %	0,80

Коэффициент вида страхования $К_т$ рассчитывается стратегией, которую фабрика PremiumStrategyFactory выбирает по категории вида. Базовый класс AbstractPremiumStrategy умножает дополнительный множитель стратегии на надбавку за повышенный риск, если описание объекта страхования содержит ключевые слова этого вида (таблица 2.8).

Таблица 2.8 – Коэффициент вида страхования $К_т$

Категория (стратегия)	Дополнительный множитель	Надбавка за повышенный риск
Имущество (PropertyPremium Strategy)	0,95 при страховой сумме больше 100 000 BYN	1,20, если в описании есть «деревян…»
КАСКО (Casco PremiumStrategy)	1,15 для водителя младше 25 лет	нет
Несчастные случаи (AccidentPremium Strategy)	1,00	1,30, если в описании есть «спорт»
Выезжающие за рубеж (Travel PremiumStrategy)	1,00	1,50, если в описании есть «горнолыж…» или «спорт»

Перед расчётом параметры проверяются: вид страхования должен быть доступен для новых договоров, страховая сумма – находиться в пределах, установленных для вида, срок – не превышать максимального, франшиза – принимать значения 0, 1, 2 или 5 %, число взносов – 1, 2, 4 или 12, причём срок должен делиться на число взносов без остатка. Число выплат клиенту за два года до начала договора определяется запросом к базе данных. Результат округляется до копеек. Схема алгоритма приведена на рисунке 2.4.

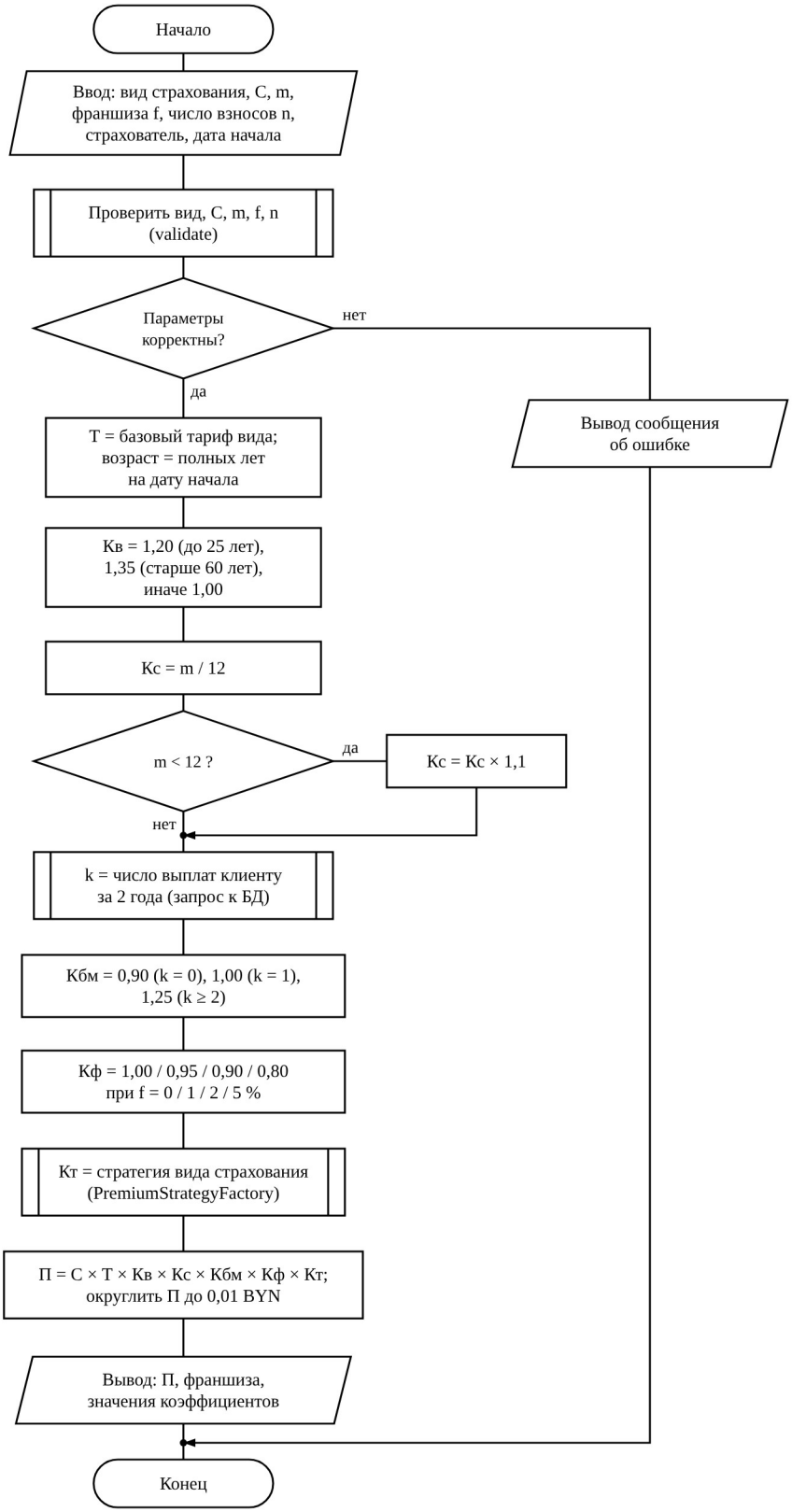


Рисунок 2.4 – Схема алгоритма расчёта страховой премии

Пример расчёта. Клиентка 27 лет без выплат за последние два года страхует квартиру на 80 000 BYN на 12 месяцев с франшизой 1 % и оплатой четырьмя взносами; базовый тариф – 0,005:

$$П = 80\,000 \times 0{,}005 \times 1{,}00 \times 1{,}00 \times 0{,}90 \times 0{,}95 \times 1{,}00 = 342{,}00 \text{ BYN}. \quad (2.2)$$

Франшиза составляет 800,00 BYN, каждый из четырёх взносов – 85,50 BYN. Этот же результат показывает клиентское приложение (рисунок 4.4).

2.3.2 Формирование графика взносов выполняет класс InstallmentScheduler. Премия П делится на n равных взносов с округлением вниз до копейки; копейки, оставшиеся от деления, добавляются к последнему взносу, поэтому сумма графика всегда равна премии:

$$П_i = \lfloor П / n \rfloor,\ i = 1, \ldots, n-1; \quad П_n = П - (n-1) \times \lfloor П / n \rfloor, \quad (2.3)$$

где $\lfloor\ \rfloor$ – округление вниз до 0,01 BYN.

Первый взнос уплачивается в дату начала договора, следующие – через m / n месяцев. Договор вступает в силу после уплаты первого взноса.

2.3.3 Алгоритм расчёта страховой выплаты реализован классом PayoutCalculator и соответствует функции A44 функциональной модели (рисунок 1.6). Исходными данными являются договор, график взносов, заявление о страховом случае, оценённый агентом ущерб D и сумма предыдущих выплат по договору W. Алгоритм последовательно проверяет условия выплаты; при нарушении любого из них формируется отказ с указанием причины:

– дата события должна входить в период действия договора;

– договор не должен быть расторгнут до даты события;

– на дату события не должно быть просроченных взносов;

– заявление должно быть подано не позднее 30 дней с даты события;

– ущерб должен превышать франшизу Ф;

– остаток страховой суммы $R = С - W$ должен быть положительным.

Если все условия выполнены, сумма выплаты рассчитывается по формуле

$$В = \min(D - Ф;\ С - W). \qquad (2.4)$$

Если выплата исчерпывает страховую сумму, договор переводится в состояние «Исполнен». Помимо решения, алгоритм возвращает обоснование по шагам, которое агент видит перед утверждением решения. Схема алгоритма приведена на рисунке 2.5; она же вынесена на чертёж графической части.

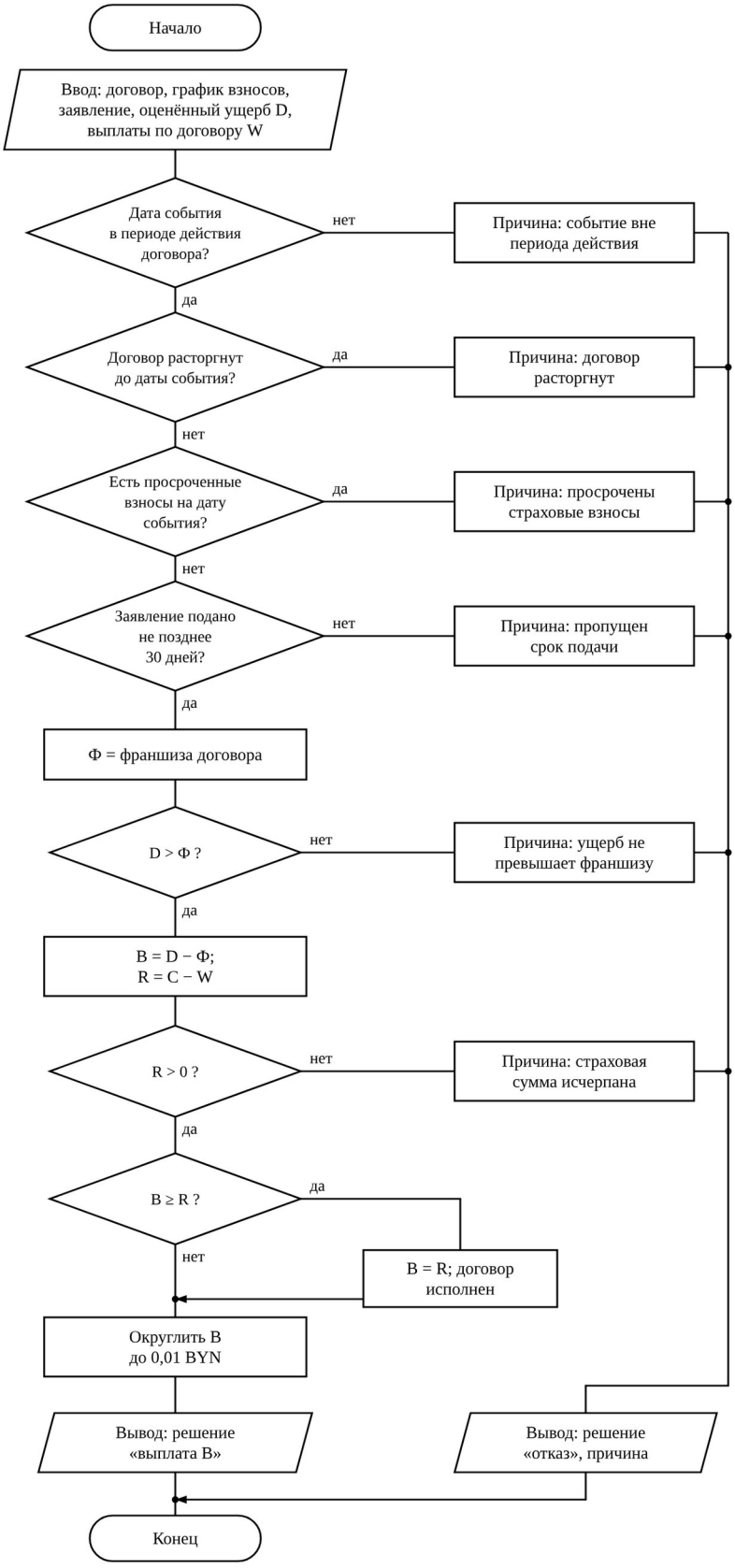


Рисунок 2.5 – Схема алгоритма расчёта страховой выплаты

Пример. По договору медицинского страхования выезжающих за рубеж со страховой суммой 30 000 BYN и нулевой франшизой клиент заявил ущерб

640 BYN через два дня после события; агент исключил из счёта расходы на такси и оценил ущерб в 600 BYN. Все условия выполнены, предыдущих выплат нет, поэтому В = min(600 − 0; 30 000 − 0) = 600,00 BYN (рисунок 4.14).

2.3.4 Расчёт возврата части премии при досрочном расторжении выполняет класс RefundCalculator. Страховщик возвращает часть уплаченных взносов, пропорциональную неистёкшему сроку договора, за вычетом расходов на ведение дела (25 %) и произведённых выплат:

$$В_в = \max(0;\ У \times D_н / D_в \times (1 - 0{,}25) - W), \qquad (2.5)$$

где $В_в$ – сумма возврата, BYN;

У – сумма уплаченных взносов, BYN;

$D_н$ – число неистёкших дней действия договора;

$D_в$ – общее число дней действия договора;

W – сумма произведённых по договору выплат, BYN.

Например, по договору КАСКО с уплаченной премией 1 475,50 BYN, расторгаемому за 234 дня до окончания годового срока, возврат составляет 1 475,50 × 234 / 365 × 0,75 = 709,45 BYN (рисунок 4.18).

2.3.5 Правила смены состояний договора реализованы классом PolicyStatusRules и соответствуют диаграмме состояний (рисунок 1.9): договор в состоянии «Ожидает оплаты» становится действующим после уплаты первого взноса и расторгается, если взнос не уплачен в течение 10 дней после оформления; действующий договор приостанавливается при просрочке очередного взноса и расторгается при просрочке более 30 дней; после даты окончания договор переходит в состояние «Истёк». Правила применяются при каждой оплате и периодически ко всем договорам.

2.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс клиентского приложения построен на библиотеке Swing [15, 16] и организован по ролям: после входа открывается главное окно, соответствующее роли пользователя, а функции роли размещены на вкладках этого окна. Схема переходов между окнами приведена на рисунке 2.6.

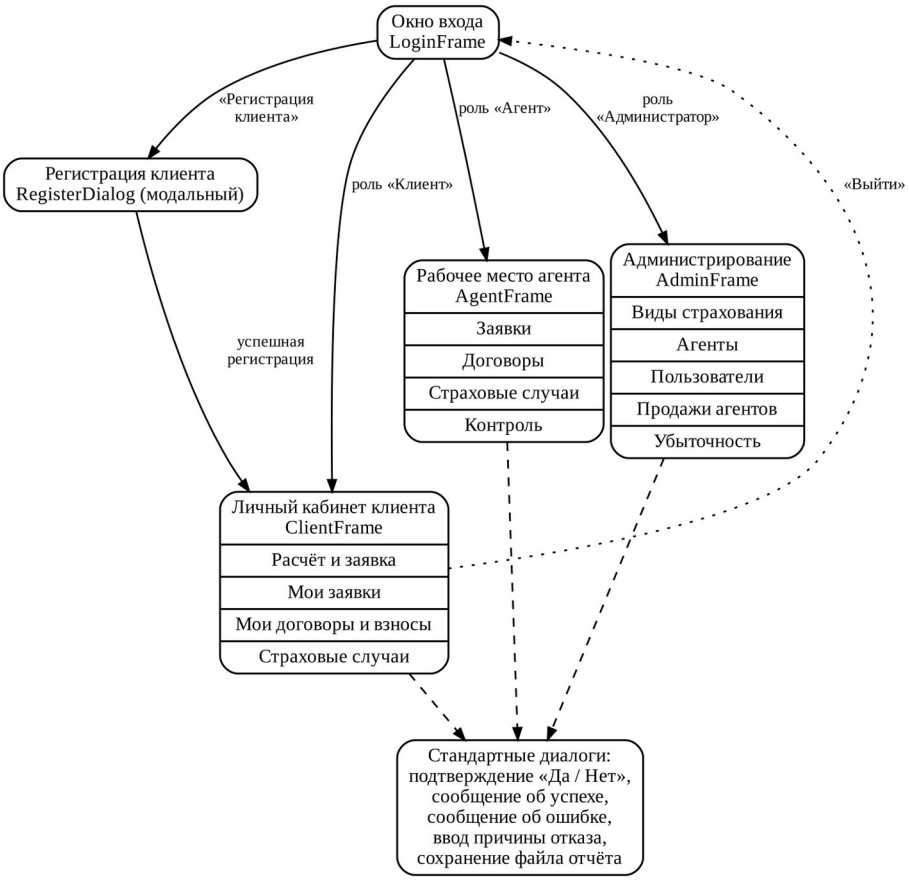


Рисунок 2.6 – Схема навигации пользовательского интерфейса

При проектировании интерфейса соблюдались следующие принципы:

– одна вкладка соответствует одному или двум связанным вариантам использования, поэтому для выполнения любой операции достаточно одного-двух переходов;

– на каждой вкладке слева или сверху расположена таблица объектов, снизу или справа – форма действия над выбранным объектом; кнопки названы по выполняемому действию;

– в подписях полей указаны единицы измерения и формат ввода, суммы отображаются с разделителем разрядов и двумя знаками после запятой, даты – в формате дд.мм.гггг;

– необратимые действия (оплата взноса, решение по страховому случаю, расторжение, продление) выполняются только после подтверждения; в окне подтверждения показываются расчётные суммы;

– результаты расчётов сопровождаются пояснением: значения коэффициентов премии, шаги проверки страхового случая, составляющие суммы возврата;

– все сообщения и надписи стандартных диалогов выводятся на русском языке.

Для единообразия общие элементы вынесены в базовые классы: BaseFrame формирует шапку окна с именем пользователя, ролью и кнопкой «Выйти»; RefreshablePanel обновляет данные вкладки при её открытии; EntityTable отображает списки объектов с сортировкой по любому столбцу и выравниванием денежных сумм; Form размещает подписи и поля ввода в две колонки. Используется тема оформления Nimbus с фирменным тёмно-синим цветом шапки, поэтому приложение выглядит одинаково в разных версиях Windows.

Окно входа (рисунок 2.7) содержит поля логина и пароля и кнопку регистрации нового клиента; в нижней части указан адрес сервера.

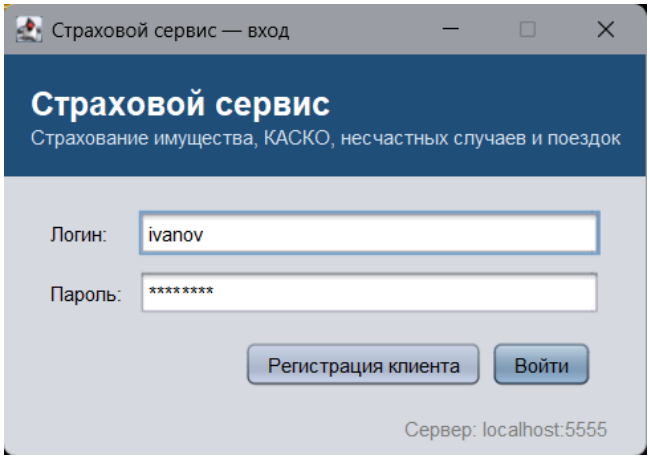


Рисунок 2.7 – Окно входа в систему

Главное окно клиента состоит из вкладок «Расчёт и заявка», «Мои заявки», «Мои договоры и взносы» и «Страховые случаи». Рабочее место агента включает вкладки «Заявки», «Договоры», «Страховые случаи» и «Контроль» (рисунок 2.8): в верхней части – таблица заявок, ожидающих рассмотрения, в нижней – сведения об объекте страхования, дата начала договора и кнопки решения.

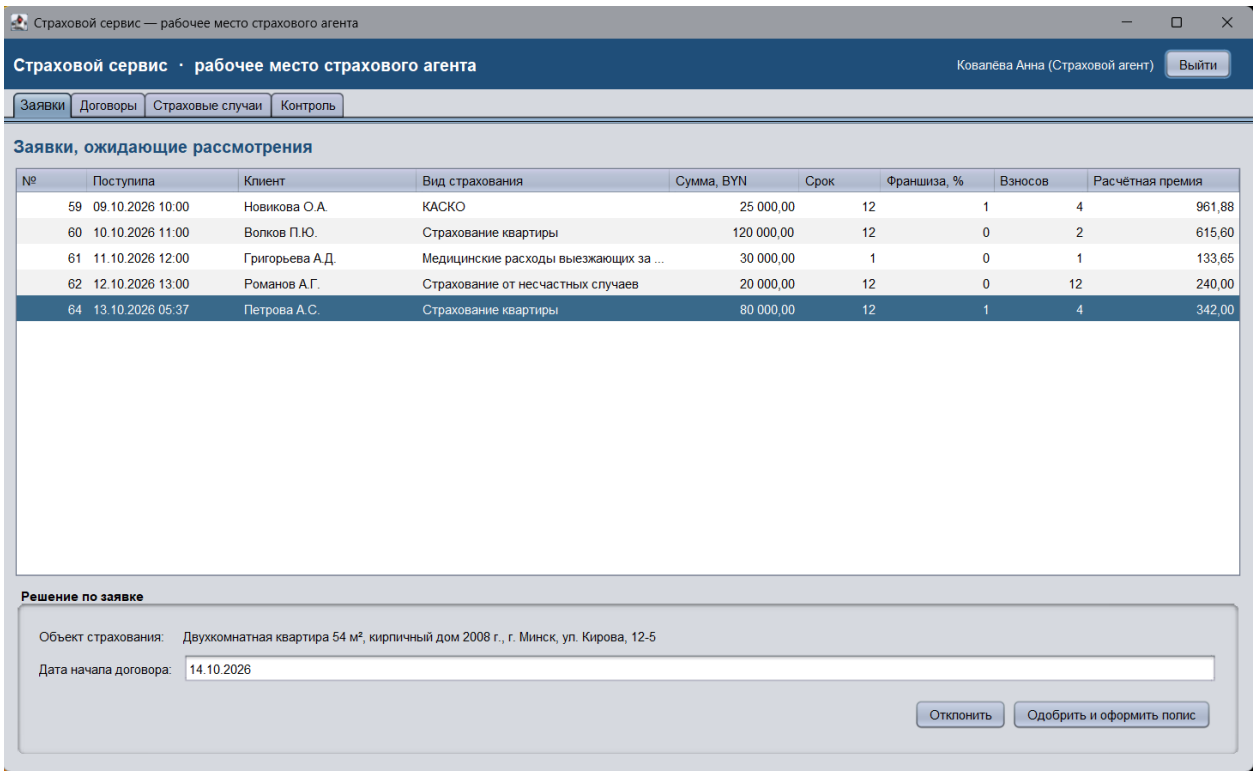


Рисунок 2.8 – Рабочее место страхового агента, вкладка «Заявки»

Окно администратора содержит вкладки управления видами страхования, агентами и пользователями, а также вкладки отчётов. Отчёты строятся за указанный период и выводятся таблицей и диаграммой (рисунок 2.9); столбчатые и круговые диаграммы рисуются собственными компонентами BarChartPanel и PieChartPanel, наследниками JPanel, без сторонних библиотек.

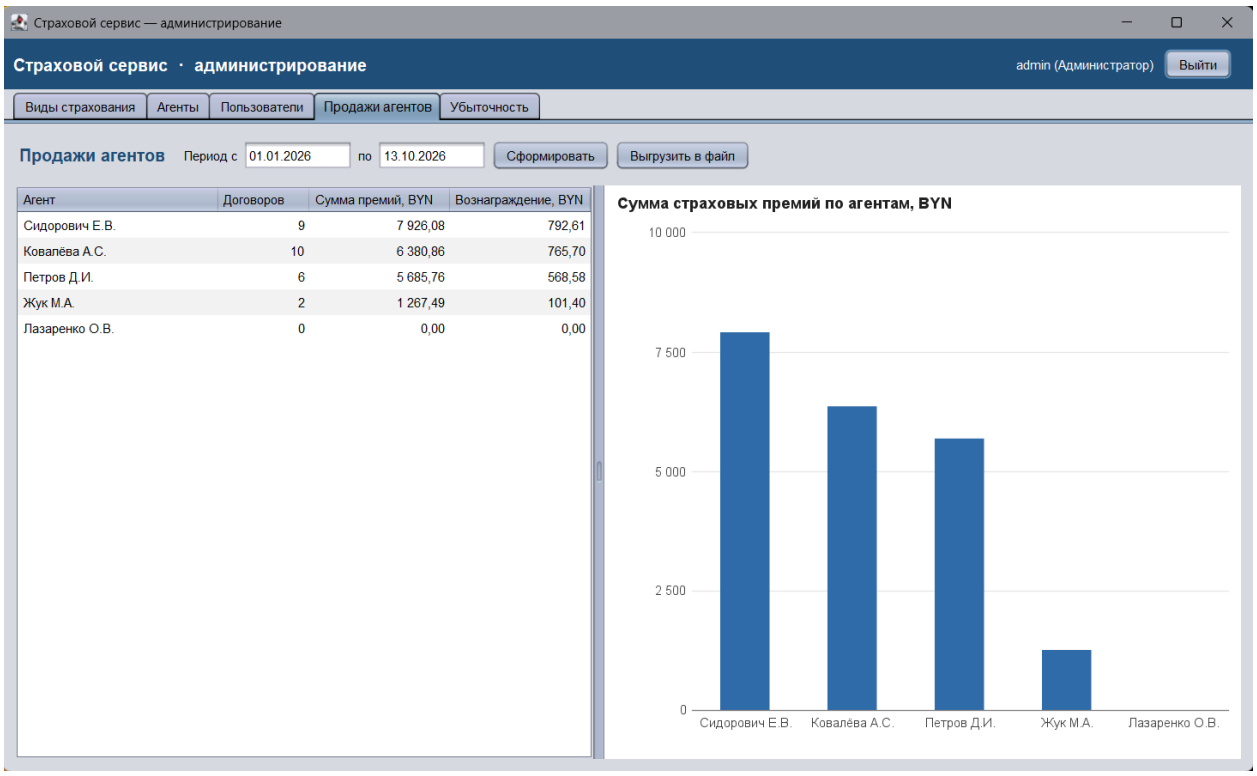


Рисунок 2.9 – Окно администратора, отчёт о продажах агентов

Ошибки ввода обнаруживаются при выполнении действия; сообщение указывает поле и допустимые значения (рисунок 2.10). Проверки, связанные с правилами предметной области, выполняет сервер, а клиент выводит его сообщение.

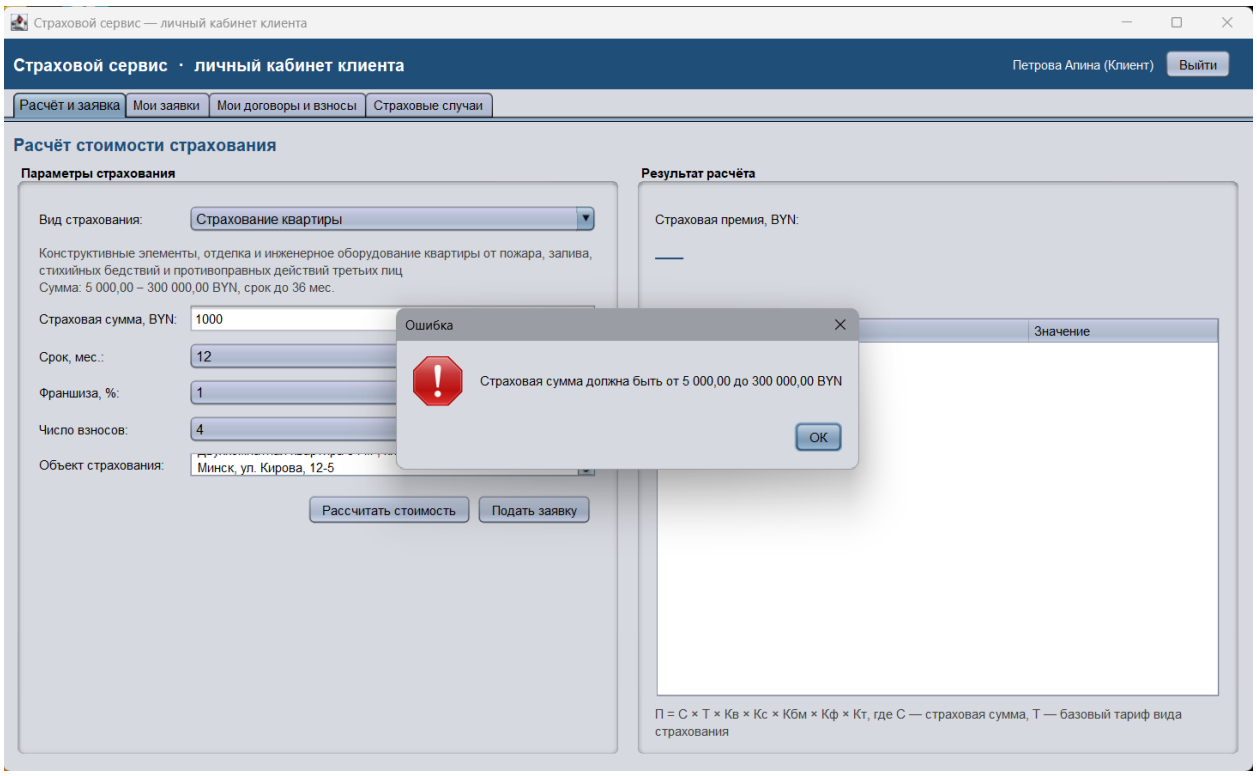


Рисунок 2.10 – Сообщение об ошибке ввода страховой суммы

2.5 Обоснование выбора компонентов и технологий для реализации программного средства

Выбор технологий определялся требованиями задания: язык Java, архитектура «клиент-сервер» с многопоточным сервером, оконный интерфейс на стандартной библиотеке, доступ к СУБД через драйвер производителя, работа в Windows 7 и выше, актуальность версий на начало 2025 года.

2.5.1 Язык и платформа. Программа написана на Java [17, 18] и собирается компилятором JDK 21 (выпуск с долгосрочной поддержкой) с параметром --release 8. Байт-код версии Java 8 выполняется как в среде Java 8, которую можно установить в Windows 7, так и в современных версиях Java; при этом компилятор проверяет, что используются только API Java 8. Использованы возможности Java 8: лямбда-выражения и ссылки на методы, пакет java.time для дат, потоки данных [19].

2.5.2 Библиотека пользовательского интерфейса. Сравнение допустимых по заданию библиотек приведено в таблице 2.9. Выбрана библиотека Swing: она входит в состав любой среды выполнения Java 8–21 и не требует установки дополнительных модулей, что важно для работы в Windows 7.

Таблица 2.9 – Сравнение библиотек пользовательского интерфейса

Критерий	Swing	JavaFX	AWT
Наличие в JRE	Входит во все версии	Отдельный набор модулей начиная с Java 11	Входит во все версии
Набор компонентов	Таблицы, вкладки, деревья, диалоги	Богатый, с CSS и анимацией	Минимальный, без таблиц
Работа в Windows 7	Да	Требует отдельной установки OpenJFX, поддержка не гарантируется	Да
Пользовательские компоненты	Наследование JPanel, Java2D	Canvas, Shape	Canvas

2.5.3 СУБД. Сравнение СУБД приведено в таблице 2.10. Выбрана PostgreSQL 16 [11, 20]: она поддерживает ограничения CHECK, блокировки строк SELECT … FOR UPDATE, частичные индексы и свободно распространяется; встраиваемая SQLite не подходит для многопользовательского сервера.

Таблица 2.10 – Сравнение СУБД

Критерий	PostgreSQL 16	MySQL 8	SQLite 3
Модель работы	Сервер	Сервер	Встраиваемая библиотека
Ограничения CHECK	Да	Да (с версии 8.0.16)	Да
Блокировка строк FOR UPDATE	Да	Да	Нет (блокировка всей БД)
Частичные индексы	Да	Нет	Да
Лицензия	PostgreSQL (свободная)	GPL / коммерческая	Общественное достояние

2.5.4 Доступ к данным. Используется JDBC с драйвером PostgreSQL JDBC 42.7.4 [21] без ORM-фреймворков. Объём модели (9 таблиц) невелик, а алгоритмам требуется явное управление транзакциями и блокировками; фреймворк уровня Hibernate увеличил бы размер дистрибутива и потребовал бы изучения его правил загрузки связанных объектов. Повторяющийся код обращения к JDBC вынесен в абстрактный класс AbstractDao.

2.5.5 Сетевое взаимодействие. Обмен построен на сокетах TCP и сериализации Java. Альтернативы – веб-сервис REST с форматом JSON или RMI – потребовали бы сторонних библиотек (сервер HTTP, преобразование в

JSON) либо сложной настройки реестра RMI, не давая преимуществ для клиента, написанного на том же языке.

2.5.6 Сборка и тестирование. Проект собирается Apache Maven [22]: модуль common подключается к серверу и клиенту как зависимость, плагин maven-shade-plugin собирает исполняемые архивы со всеми зависимостями, модульные тесты на JUnit 4 [23] выполняются при каждой сборке. Версии использованных компонентов, актуальные на начало 2025 года, приведены в таблице 2.11.

Таблица 2.11 – Версии использованных компонентов

Компонент	Версия	Назначение
JDK (Eclipse Temurin)	21 LTS	Компиляция (байт-код Java 8) и выполнение
PostgreSQL	16.4	СУБД
PostgreSQL JDBC Driver	42.7.4	Драйвер доступа к БД
Apache Maven	3.9.9	Сборка проекта
maven-compiler-plugin	3.13.0	Компиляция с параметром release 8
maven-shade-plugin	3.6.0	Сборка исполняемых архивов
maven-surefire-plugin	3.2.5	Запуск модульных тестов
JUnit	4.13.2	Модульное тестирование

3 ТЕСТИРОВАНИЕ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОГРАММНОГО СРЕДСТВА

Работоспособность программного средства проверялась на четырёх уровнях:

– модульное тестирование алгоритмов бизнес-логики (JUnit 4, выполняется при каждой сборке);

– сквозное тестирование серверной части через сетевой протокол: автоматический сценарий проходит все варианты использования, проверяя как успешные операции, так и отказы;

– нагрузочное тестирование многопоточного сервера с одновременной работой до 200 клиентов;

– ручное функциональное тестирование через пользовательский интерфейс по сквозному примеру раздела 4.

Тестирование выполнялось на компьютере, характеристики которого приведены в таблице 3.1. Сервер, клиент и СУБД работали на одном компьютере; база данных перед каждым прогоном создавалась заново скриптами приложения В и заполнялась демонстрационными данными.

Таблица 3.1 – Конфигурация тестового стенда

Компонент	Характеристика
Операционная система	Microsoft Windows 11 Pro
Процессор	Intel Core i5-11400F, 2,6 ГГц, 12 логических процессоров
Оперативная память	32 ГБ
Среда выполнения Java	Eclipse Temurin 21.0.12 (байт-код Java 8)
СУБД	PostgreSQL 16.4
Сборка и тесты	Apache Maven 3.9.9, JUnit 4.13.2

3.1 Модульное тестирование. Модульные тесты проверяют классы пакета server.logic и класс хеширования паролей без обращения к базе данных и сети. Для алгоритмов расчёта премии и выплаты тесты построены так, чтобы пройти все ветви схем алгоритмов (рисунки 2.4 и 2.5): каждой ветви отказа соответствует отдельный тест. Ожидаемые значения рассчитаны вручную по формулам подраздела 2.3; перечень тестов приведён в таблице 3.2.

Таблица 3.2 – Модульные тесты

Класс	Тест	Проверяемая ситуация	Ожидаемый результат
Premium CalculatorTest	propertyStandardClient WithoutClaims	Квартира 50 000 BYN, 12 месяцев, клиент 41 год, выплат нет	Премия 225,00 BYN, франшиза 0
	cascoYoungDriverWith Franchise	КАСКО 30 000 BYN, водитель 23 года, 1 выплата, франшиза 2 %	Премия 1 676,70 BYN, франшиза 600,00
	travelShortTermSenior ActiveRest	Поездка на 1 месяц, клиент 71 год, 2 выплаты, горнолыжный отдых	Премия 139,22 BYN
	largeSumProperty Discount	Деревянный дом 200 000 BYN, 24 месяца, франшиза 5 %	Премия 1 641,60 BYN
	sumOutOfRange Rejected	Страховая сумма ниже минимальной	Validation Exception
	termOverMaximum Rejected	Срок больше максимального для вида	Validation Exception
	inactiveTypeRejected	Вид страхования недоступен	Validation Exception
	factorsFollowTariff Tables	Граничные значения возраста, $К_{бм}$, $К_{ф}$, $К_{с}$	Значения по таблице 2.7
Installment SchedulerTest	monthlyScheduleKeeps TotalAndDates	Премия 100,00 на 12 взносов с 31.01.2026	11 × 8,33 и 8,37; сумма 100,00; сроки 28.02 … 31.12
	quarterlySchedule	Четыре взноса за год	Сроки через 3 месяца
	termNotDivisible Rejected	Срок не делится на число взносов	Validation Exception
	wrongInstallments Rejected	Число взносов 3	Validation Exception
PayoutCalculator Test	payoutIsDamageMinus Franchise	Все условия выполнены	Выплата $D - Ф$
	rejectedWhenOutside Period	Событие вне периода действия	Отказ
	rejectedWhenPremium Overdue	Просроченный взнос на дату события	Отказ
	rejectedWhenReported TooLate	Заявление позже 30 дней	Отказ
	rejectedWhenDamage WithinFranchise	Ущерб не больше франшизы	Отказ
	payoutLimitedBy RemainingSum	Ущерб 2 000 при остатке суммы 500	Выплата 500,00, сумма исчерпана
	rejectedWhenSum Exhausted	Остаток страховой суммы равен 0	Отказ

Продолжение таблицы 3.2

Класс	Тест	Проверяемая ситуация	Ожидаемый результат
RefundCalculatorTest	refundProportionalToRemainingDays	Уплачено 1 200, осталось 182 из 365 дней	Возврат 448,77 BYN
	payoutsReduceRefundToZero	Выплаты больше расчётного возврата	Возврат 0
PolicyStatusRulesTest	activeWithoutDebts	Действующий договор без долгов	«Действует»
	suspendedWhenInstallmentOverdue	Просрочен очередной взнос	«Приостановлен»
	terminatedAfter30DaysOverdue	Просрочка более 30 дней	«Расторгнут»
	expiredAfterEndDate	Наступила дата окончания	«Истёк»
	waitingBecomesActiveAfterFirstPayment	Уплачен первый взнос	«Действует»
	waitingTerminatedWhenFirstPaymentMissed	Первый взнос не уплачен 10 дней	«Расторгнут»
PasswordUtilTest	hashIsStableAndSalted	Один пароль с разными солями	Хеши различны, длина 64
	matchesChecksPassword	Проверка верного и неверного пароля	true / false

Все 29 модульных тестов выполнены успешно; при сборке командой mvn package модуль surefire выводит итог «Tests run: 29, Failures: 0, Errors: 0, Skipped: 0». В ходе разработки тесты выявили, например, ошибку округления последнего взноса графика, которая была исправлена до интеграции.

3.2 Сквозное тестирование серверной части. Класс EndToEndScenario подключается к работающему серверу по сетевому протоколу так же, как клиентское приложение, и последовательно выполняет все варианты использования от имени клиента, агента и администратора. Для каждого действия задан ожидаемый результат; проверяются как успешные операции, так и реакция на ошибочные действия: неверный пароль, обращение к функции другой роли, повторное рассмотрение заявки, доступ к чужому договору, дата события в будущем. Результаты прогона приведены в таблице 3.3: все 37 проверок пройдены.

Таблица 3.3 – Результаты сквозного тестирования серверной части

Проверка	Вариант использования	Ожидаемый результат	Фактический результат	Итог
1 Вход с неверным паролем	UC2	Отказ: «…Неверный логин или пароль…»	Отказ: «Неверный логин или пароль»	Пройден
2 Запрос без авторизации	UC2	Отказ: «…Требуется авторизация…»	Отказ: «Требуется авторизация»	Пройден
3 Регистрация клиента test34869	UC1	Учётная запись создана, вход выполнен	Создана, роль Клиент	Пройден
4 Повторная регистрация с тем же логином	UC1	Отказ: «…уже занят…»	Отказ: «Логин «test34869» уже занят»	Пройден
5 Регистрация лица младше 18 лет	UC1	Отказ: «…18 лет…»	Отказ: «Заключать договоры могут лица, достигшие 18 лет»	Пройден
6 Получение активных видов страхования	UC3	5 видов	5 видов	Пройден
7 Расчёт премии: квартира, 50 000 BYN, 12 месяцев	UC3	225,00 BYN (50 000 × 0,005 × 0,90)	225,00 BYN	Пройден
8 Расчёт с суммой выше максимальной	UC3	Отказ: «…Страховая сумма должна быть…»	Отказ: «Страховая сумма должна быть от 5 000,00 до 300 000,00 BYN»	Пройден
9 Подача заявки	UC4	Заявка в статусе «На рассмотрении»	На рассмотрении	Пройден
10 Клиент вызывает функцию агента	UC2	Отказ: «… Недостаточно прав…»	Отказ: «Недостаточно прав для выполнения операции»	Пройден
11 Агент видит новую заявку	UC5	Заявка в списке с расчётной премией	Есть в списке	Пройден
12 Оформление полиса по заявке	UC5	Полис, премия 225,00, «Ожидает оплаты»	INS-2026-000059, 225,00, Ожидает оплаты	Пройден
13 Повторное оформление той же заявки	UC5	Отказ: «…уже рассмотрена…»	Отказ: «Заявка уже рассмотрена: Одобрена»	Пройден
14 Просмотр графика взносов	UC12	2 взноса по 112,50	2 взноса: 112,50 и 112,50	Пройден

Продолжение таблицы 3.3

Проверка	Вариант использования	Ожидаемый результат	Фактический результат	Итог
15 Внесение первого взноса	UC6	Договор переходит в «Действует»	Действует	Пройден
16 Другой клиент запрашивает чужой договор	UC12	Отказ: «…Нет доступа…»	Отказ: «Нет доступа к чужому договору»	Пройден
17 Заявление о страховом случае	UC7	Зарегистрировано, статус «Заявлен»	Заявлен	Пройден
18 Заявление с датой события в будущем	UC7	Отказ: «…не может быть в будущем…»	Отказ: «Дата «Дата события» не может быть в будущем»	Пройден
19 Расчёт выплаты (ущерб оценён в 3 000)	UC8	Выплата 3 000,00 (франшиза 0)	Выплата 3 000,00	Пройден
20 Утверждение решения по страховому случаю	UC8	Статус «Выплата произведена»	Выплата произведена, 3 000,00	Пройден
21 Повторное решение по тому же случаю	UC8	Отказ: «…уже принято…»	Отказ: «Решение по страховому случаю уже принято»	Пройден
22 Ущерб больше остатка страховой суммы	UC8	Выплата 47 000,00, договор «Исполнен»	47 000,00, договор: Исполнен	Пройден
23 Продление договора, до окончания которого больше 30 дней	UC9	Отказ: «…Продлить можно…»	Отказ: «Продлить можно истёкший договор или действующий договор, срок которого заканчивается в течение 30 дней»	Пройден
24 Продление истёкшего договора	UC9	Новый договор «Ожидает оплаты»	Новый договор INS-2026-000060 (Ожидает оплаты)	Пройден
25 Досрочное расторжение с расчётом возврата	UC10	«Расторгнут», возврат = расчёту	Расторгнут, возврат 1 979,52 (расчёт 1 979,52)	Пройден
26 Список просроченных взносов и истекающих договоров	UC11	Непустой список	4 записи	Пройден

Продолжение таблицы 3.3

Проверка	Вариант использования	Ожидаемый результат	Фактический результат	Итог
27 Пересчёт состояний договоров	UC11	Выполнен	Состояния договоров пересчитаны, изменено: 0	Пройден
28 Добавление вида страхования	UC13	Сохранён с новым идентификатором	id = 7	Пройден
29 Тариф больше 1	UC13	Отказ: «…долей страховой суммы…»	Отказ: «Базовый тариф задаётся долей страховой суммы, например 0,0050»	Пройден
30 Создание учётной записи агента	UC14	Агент создан	Новикова Дарья, логин agent34869	Пройден
31 Вход заблокированного пользователя	UC14	Отказ: «… заблокирована…»	Отказ: «Учётная запись заблокирована. Обратитесь к администратору»	Пройден
32 Вход после разблокировки	UC14	Вход выполнен	Вход: Тестов Тест	Пройден
33 Отчёт о продажах агентов за год	UC15	Строка по каждому агенту	5 агентов, 27 договоров	Пройден
34 Анализ убыточности за 2025–2026 годы	UC16	Строка по каждому виду с коэффициентом	5 видов, например, страхование квартиры: 1 973,0 %	Пройден
35 Отчёт с перепутанными датами периода	UC15	Отказ: «…Начало периода…»	Отказ: «Начало периода должно быть не позже его окончания»	Пройден
36 20 клиентов одновременно: вход и запросы	–	Все запросы выполнены	20 из 20 за 18 мс	Пройден
37 Два агента одновременно оформляют одну заявку	UC5	Оформлен ровно один договор	Успешных оформлений: 1	Пройден

Две последние проверки относятся к многопоточной работе. В первой 20 клиентов одновременно входят в систему и выполняют запросы; все запросы обработаны. Во второй два агента в параллельных потоках одновременно оформляют одну и ту же заявку: благодаря блокировке строки

заявки в транзакции договор оформляется ровно один раз, а второй агент получает сообщение «Заявка уже рассмотрена».

3.3 Нагрузочное тестирование. Класс LoadTest запускает заданное число клиентов, которые одновременно подключаются к серверу, входят в систему и выполняют по 10 запросов: получение списка видов страхования и расчёт премии с обращением к базе данных. Сервер работал с параметрами по умолчанию: 20 потоков, 10 соединений с БД. Результаты после разогревочного прогона приведены в таблице 3.4.

Таблица 3.4 – Результаты нагрузочного тестирования

Число клиентов	Число запросов	Число ошибок	Общее время, мс	Среднее время запроса, мс	Максимальное время запроса, мс
10	110	0	53	3,3	19,7
20	220	0	83	2,5	10,6
50	550	0	165	1,6	7,1
100	1100	0	220	1,7	10,2
200	2200	0	328	1,3	12,5

При 200 одновременных клиентах, что в 10 раз больше числа потоков пула, все 2 200 запросов выполнены без ошибок за 0,33 с. Клиенты, для которых не нашлось свободного потока, ожидают в очереди пула, поэтому общее время растёт медленнее числа клиентов, а среднее время запроса остаётся в пределах единиц миллисекунд, что значительно меньше нефункционального требования (2 с).

3.4 Функциональное тестирование через интерфейс. Тестирование выполнено по сквозному примеру подраздела 4.5: все семнадцать вариантов использования выполнены в клиентском приложении, а результаты операций совпали с ожидаемыми. Проверена также реакция интерфейса на ошибки: неверный пароль (рисунок 4.2), вход заблокированного пользователя (рисунок 4.25), страховая сумма вне допустимого диапазона (рисунок 2.10). Во всех случаях выводится понятное сообщение, а приложение продолжает работу.

4 ИНСТРУКЦИЯ ПО РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ И СКВОЗНОЙ ТЕСТОВЫЙ ПРИМЕР

4.1 Требования к программному и аппаратному обеспечению

Требования к компьютерам, на которых разворачиваются компоненты программного средства, приведены в таблице 4.1. Сервер приложений, СУБД и клиент могут работать на одном компьютере.

Таблица 4.1 – Требования к программному и аппаратному обеспечению

Компонент	Программное обеспечение	Аппаратное обеспечение
Клиентское приложение	Windows 7 и выше; Java SE 8 и выше (JRE)	Процессор 1 ГГц, оперативная память 2 ГБ, экран не менее 1280 × 768, сеть
Сервер приложений	Windows 7 и выше или Linux; Java SE 8 и выше	Процессор 2 ГГц, оперативная память 4 ГБ, сеть
Сервер базы данных	PostgreSQL 16 (подходят версии 12 и выше)	Оперативная память 1 ГБ, 1 ГБ на диске
Сборка из исходного кода	JDK 11 и выше (рекомендуется 21), Apache Maven 3.9	Доступ в интернет для загрузки зависимостей

Исходный код, скрипты базы данных и пояснительная записка размещены в публичном репозитории https://github.com/Danilakov277/insurance-service-kp. Готовые исполняемые архивы получаются в каталогах server/target и client/target после сборки.

4.2 Развертывание базы данных

Порядок развёртывания базы данных:

1 Установить PostgreSQL 16 с параметрами по умолчанию (порт 5432, пользователь postgres) и запомнить пароль пользователя postgres.

2 Открыть командную строку в каталоге проекта. Если у пользователя postgres задан пароль, указать его в переменной окружения: set PGPASSWORD=пароль.

3 Выполнить файл create-database.bat или команды psql -U postgres -h localhost -f database\01_schema.sql и psql -U postgres -h localhost -f database\02_demo_data.sql. Первый скрипт (приложение В) создаёт базу данных insurance, таблицы, ограничения и индексы, второй загружает демонстрационные данные.

4 Проверить результат: команда psql -U postgres -d insurance -c "select count(*) from policies" должна вывести число договоров (58 в демонстрационных данных).

Демонстрационные данные содержат виды страхования, четыре агента, 15 клиентов, около 60 договоров за 2025–2026 годы с графиками взносов и страховыми случаями. Учётные записи для входа приведены в таблице 4.2.

Таблица 4.2 – Учётные записи демонстрационных данных

Роль	Логин	Пароль	Пользователь
Администратор	admin	admin123	–
Страховой агент	kovaleva, petrov, sidorovich, zhuk	agent123	Ковалёва А. С. и другие
Клиент	ivanov, novikova, kozlov и другие (всего 15)	client123	Иванов С. А. и другие

4.3 Сборка, настройка и запуск сервера

Для сборки из исходного кода в каталоге проекта выполняется команда mvn clean package. Maven загружает зависимости, компилирует модули common, server и client, выполняет модульные тесты и создаёт архивы server\target\insurance-server.jar и client\target\insurance-client.jar, в которые включены все зависимости, в том числе JDBC-драйвер.

Параметры сервера задаются в файле server.properties в каталоге запуска (таблица 2.4). Если сервер БД находится на другом компьютере или у пользователя postgres есть пароль, изменяются параметры db.url и db.password. Параметры можно также передать при запуске, например run-server.bat --server.port=6000 --db.password=secret.

Сервер запускается файлом run-server.bat (команда java -jar server\target\insurance-server.jar). При успешном запуске в консоль выводятся сообщения о подключении к базе данных, номере порта и числе потоков, а также результат первого пересчёта состояний договоров (рисунок 4.1). Сервер работает, пока открыто окно консоли; остановка – комбинацией клавиш Ctrl+C.

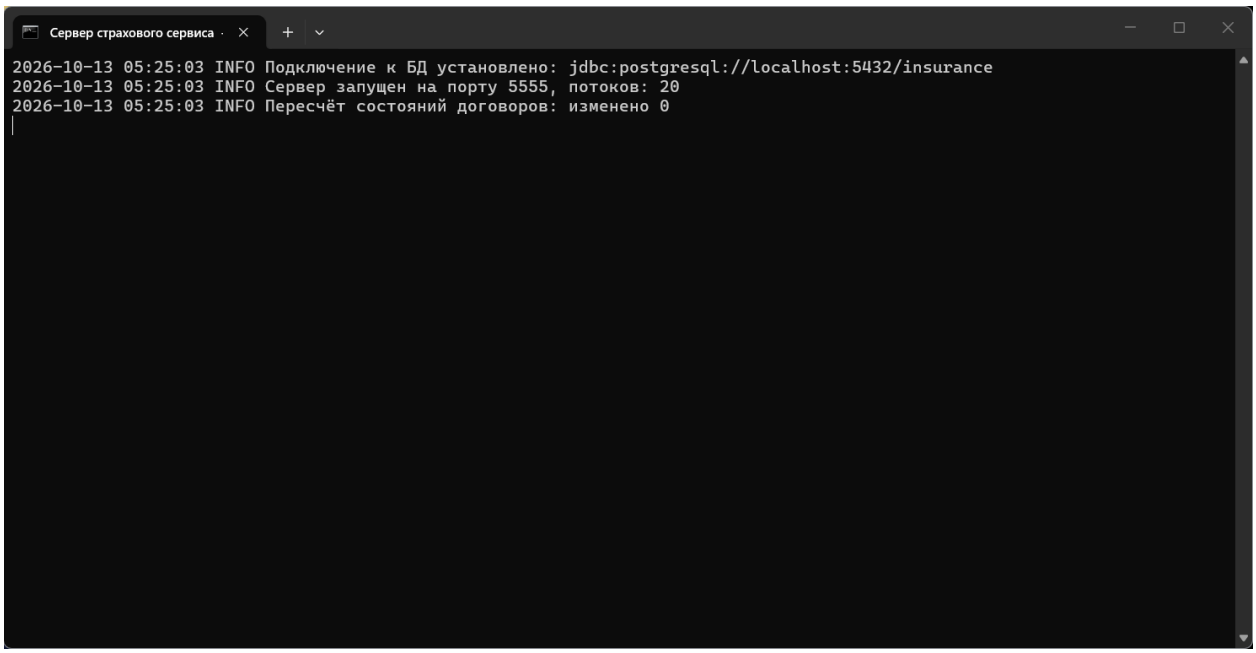


Рисунок 4.1 – Окно консоли сервера после запуска

4.4 Запуск клиентского приложения

Клиентское приложение запускается файлом run-client.bat или двойным щелчком по файлу insurance-client.jar. Адрес сервера задаётся в файле client.properties рядом с архивом (server.host и server.port) или аргументами запуска, например run-client.bat --server.host=192.168.1.10. После запуска открывается окно входа (рисунок 2.7), в нижней части которого указан адрес сервера. Если сервер недоступен, при попытке входа выводится сообщение «Нет связи с сервером».

4.5 Сквозной тестовый пример

Сквозной пример демонстрирует все варианты использования программного средства на базе с демонстрационными данными: новый клиент регистрируется, рассчитывает стоимость страхования и подаёт заявку, агент оформляет договор, клиент вносит взнос и заявляет о страховом случае, агент урегулирует случай, продлевает и расторгает договоры, администратор управляет справочниками и учётными записями и строит отчёты. Пример выполнялся 13.10.2026; все приведённые снимки получены в ходе одного прогона.

4.5.1 Авторизация (UC2). Пользователь вводит логин и пароль в окне входа. При неверном пароле сервер отказывает во входе, и выводится сообщение об ошибке (рисунок 4.2).

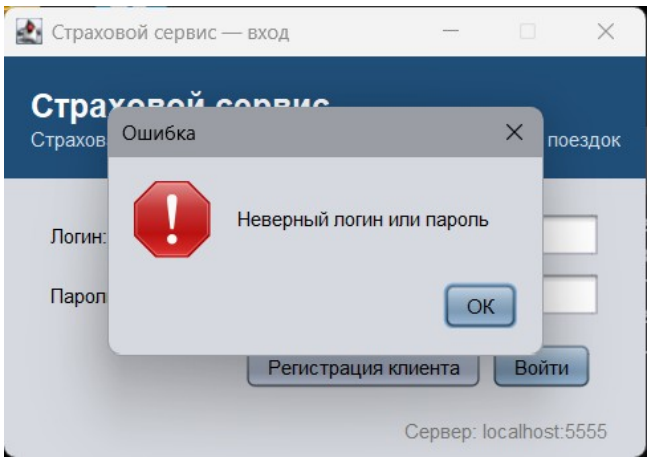


Рисунок 4.2 – Сообщение о неверном логине или пароле

4.5.2 Регистрация клиента (UC1). Новый клиент нажимает «Регистрация клиента» и заполняет форму (рисунок 4.3). Сервер проверяет уникальность логина и номера паспорта, формат паспорта и телефона, совершеннолетие клиента, сохраняет учётную запись с хешем пароля и выполняет вход; открывается личный кабинет клиента.

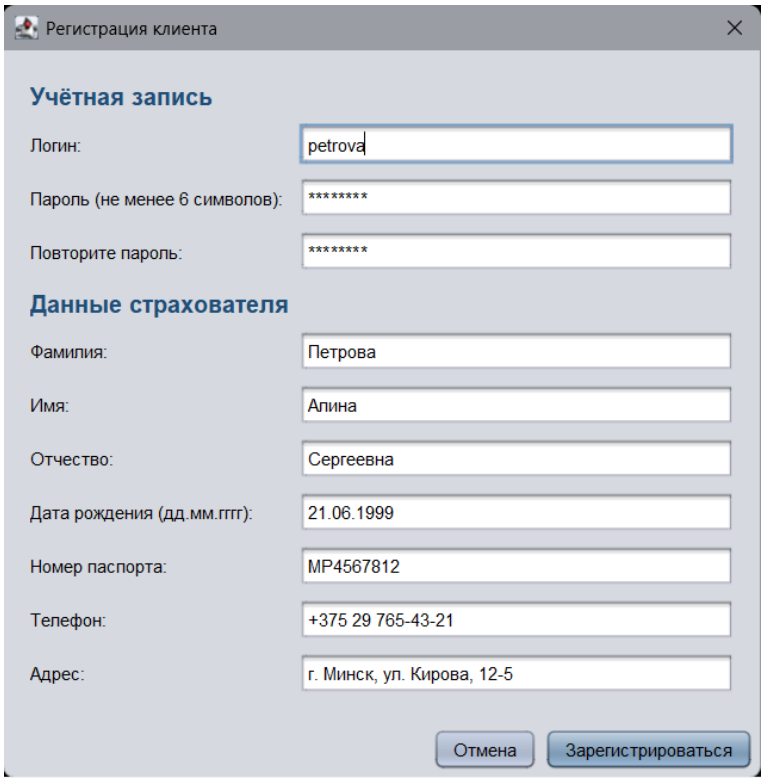


Рисунок 4.3 – Регистрация нового клиента

4.5.3 Расчёт стоимости страхования (UC3). На вкладке «Расчёт и заявка» клиентка выбирает вид «Страхование квартиры», вводит страховую сумму 80 000 BYN, срок 12 месяцев, франшизу 1 %, четыре взноса и описание

квартиры и нажимает «Рассчитать стоимость». Если сумма выходит за пределы, установленные для вида страхования, выводится сообщение (рисунок 2.10). Результат расчёта (рисунок 4.4) содержит премию 342,00 BYN, франшизу, размер взноса и значения всех коэффициентов; расчёт совпадает с примером подраздела 2.3.

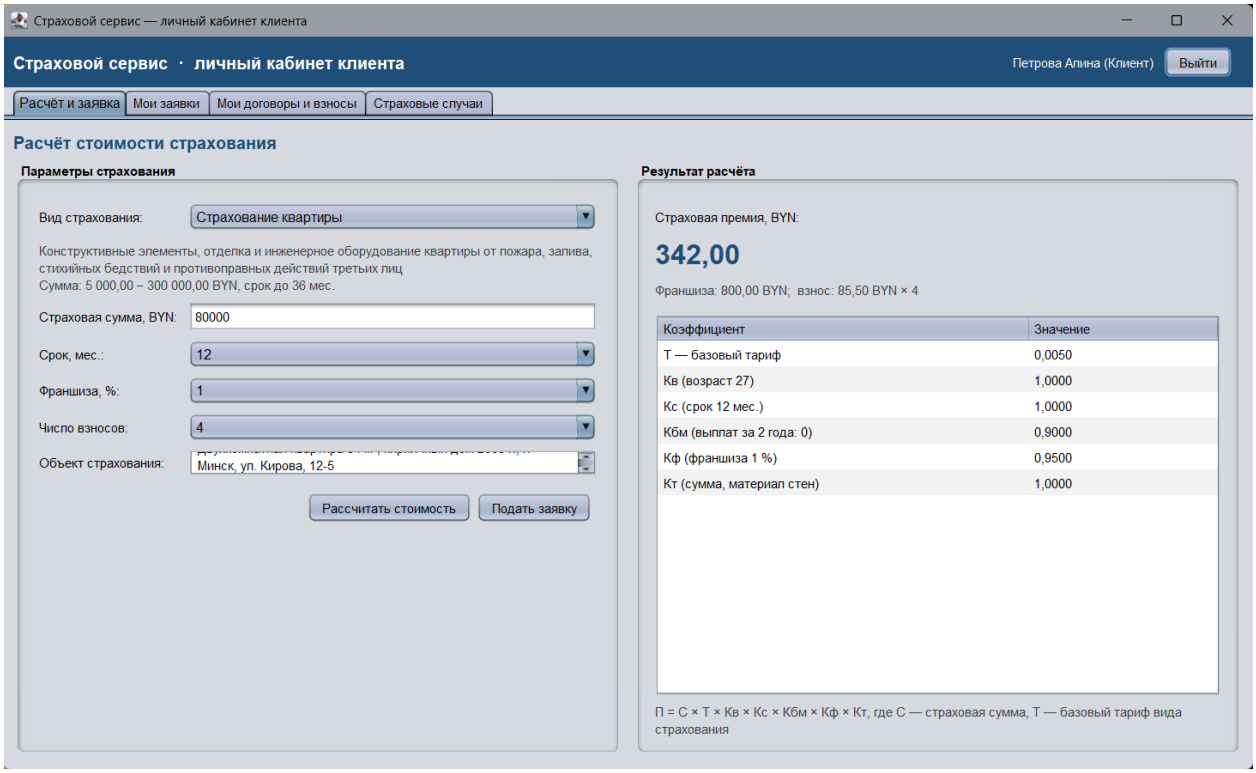


Рисунок 4.4 – Результат расчёта стоимости страхования

4.5.4 Подача заявки (UC4). Кнопка «Подать заявку» отправляет заявку на рассмотрение (рисунок 4.5). На вкладке «Мои заявки» заявка отображается в состоянии «На рассмотрении» (рисунок 4.6).

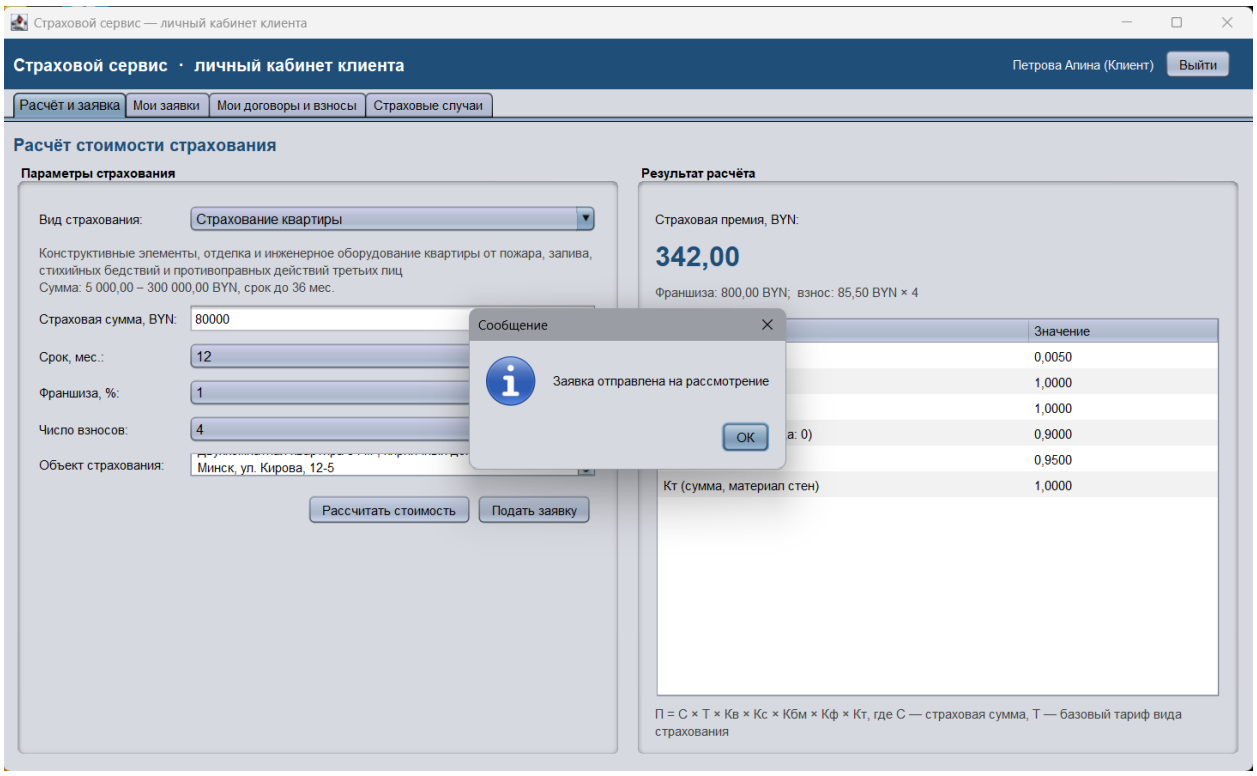


Рисунок 4.5 – Подтверждение отправки заявки

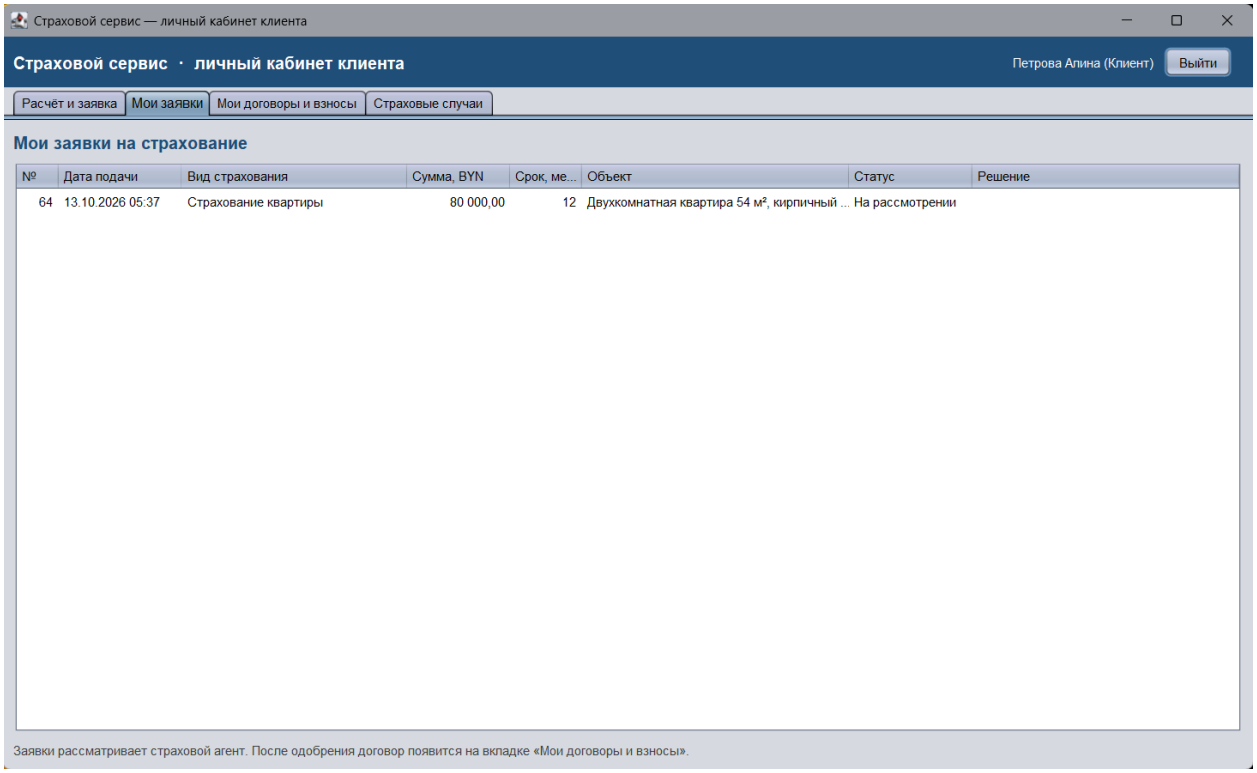


Рисунок 4.6 – Заявки клиента

4.5.5 Рассмотрение заявки и оформление договора (UC5). Агент входит в систему; на вкладке «Заявки» отображаются заявки, ожидающие

рассмотрения, с предварительно рассчитанной премией (рисунок 2.8). Агент выбирает заявку, проверяет описание объекта, указывает дату начала договора и нажимает «Одобрить и оформить полис». Сервер в одной транзакции рассчитывает премию, создаёт договор с номером полиса и график взносов (рисунок 4.7). Другую заявку агент отклоняет, указав причину отказа (рисунок 4.8).

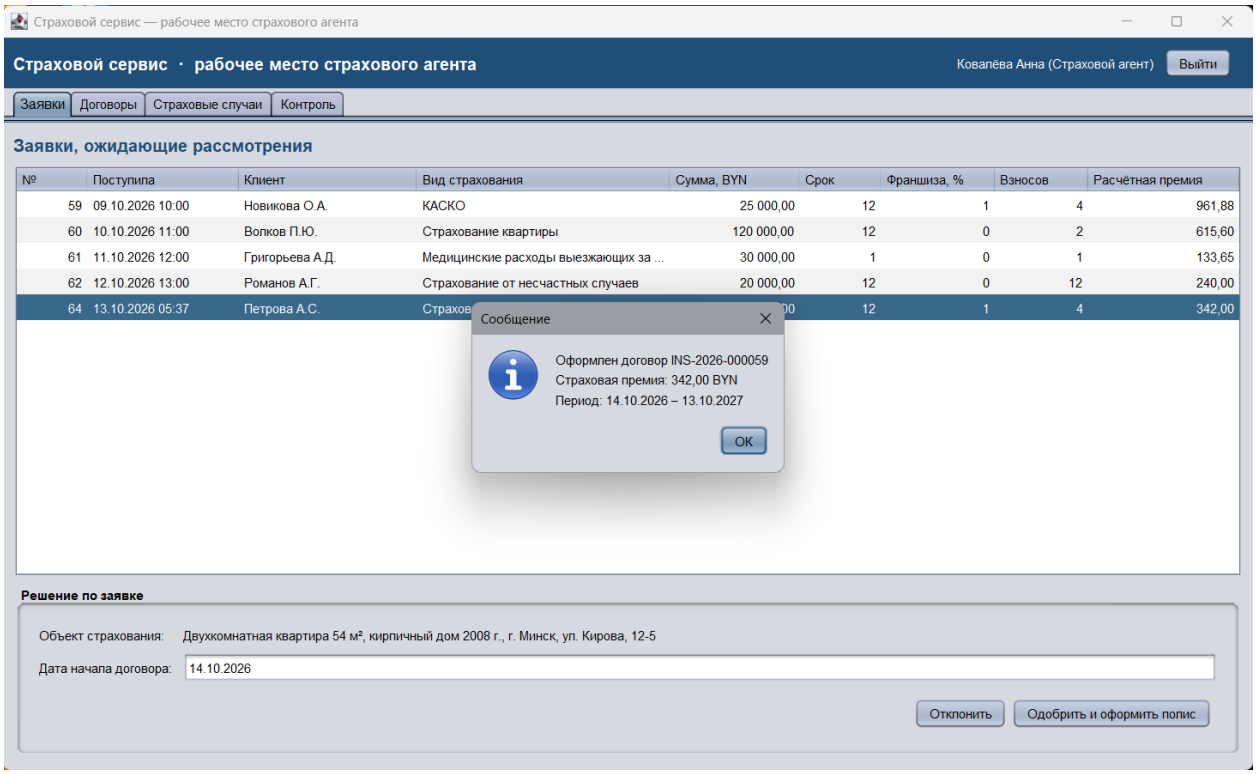


Рисунок 4.7 – Сообщение об оформлении договора

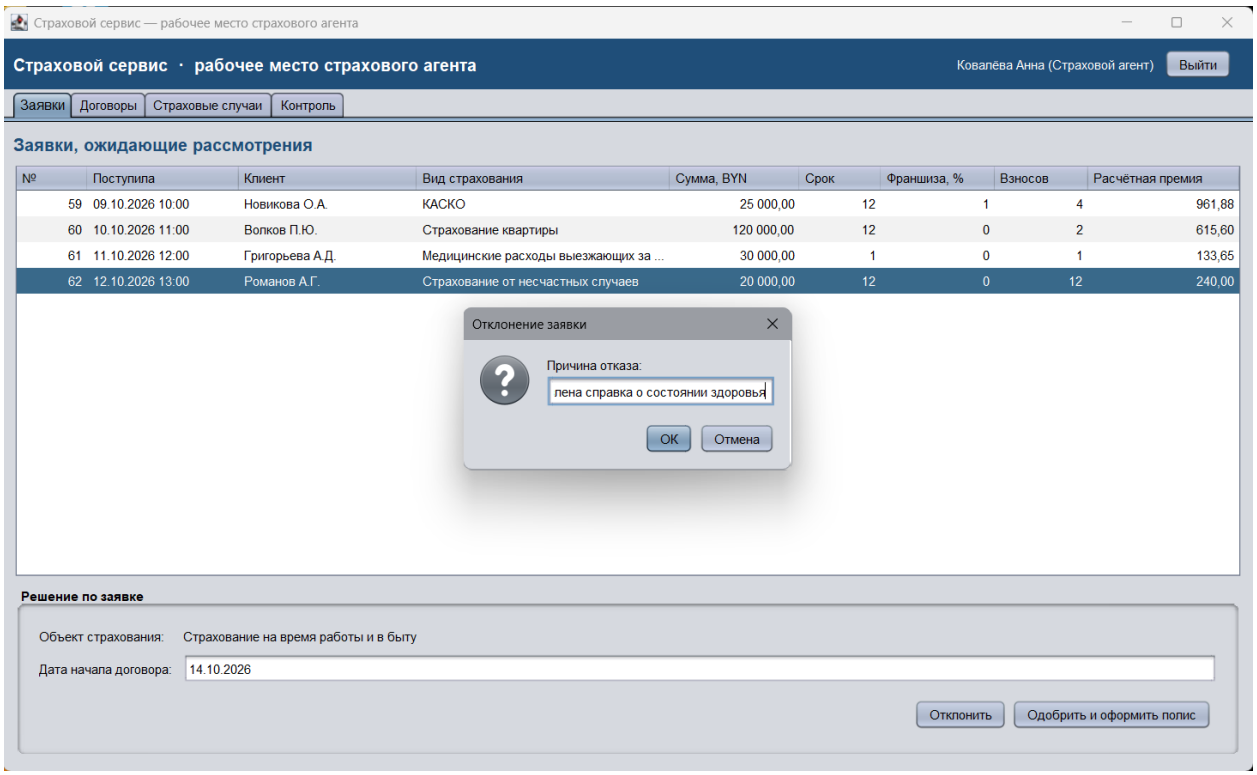


Рисунок 4.8 – Отклонение заявки с указанием причины

4.5.6 Просмотр договоров и графика взносов (UC12), внесение взноса (UC6). Клиентка открывает вкладку «Мои договоры и взносы» и выбирает новый договор в состоянии «Ожидает оплаты» (рисунок 4.9): в нижней части показан график из четырёх взносов по 85,50 BYN. После нажатия «Внести очередной взнос» и подтверждения (рисунок 4.10) первый взнос отмечается уплаченным, а договор переходит в состояние «Действует» (рисунок 4.11).

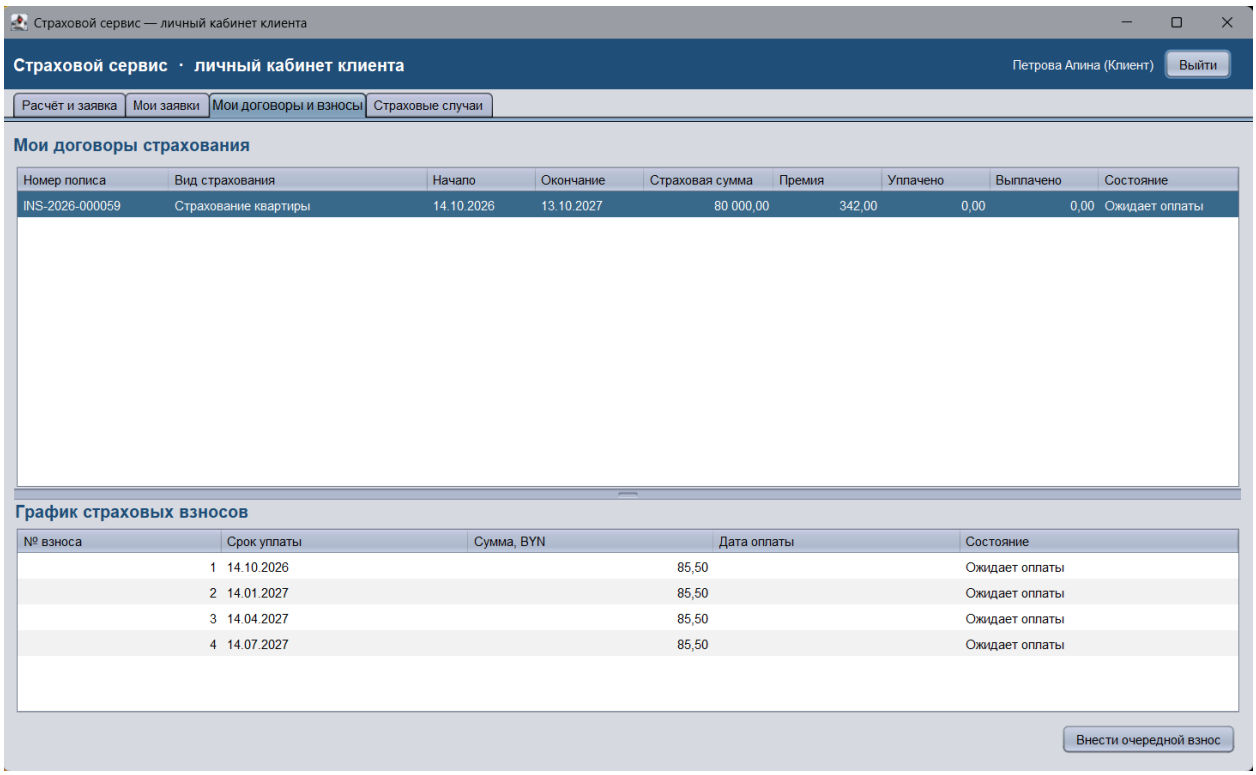


Рисунок 4.9 – Договор клиента и график взносов

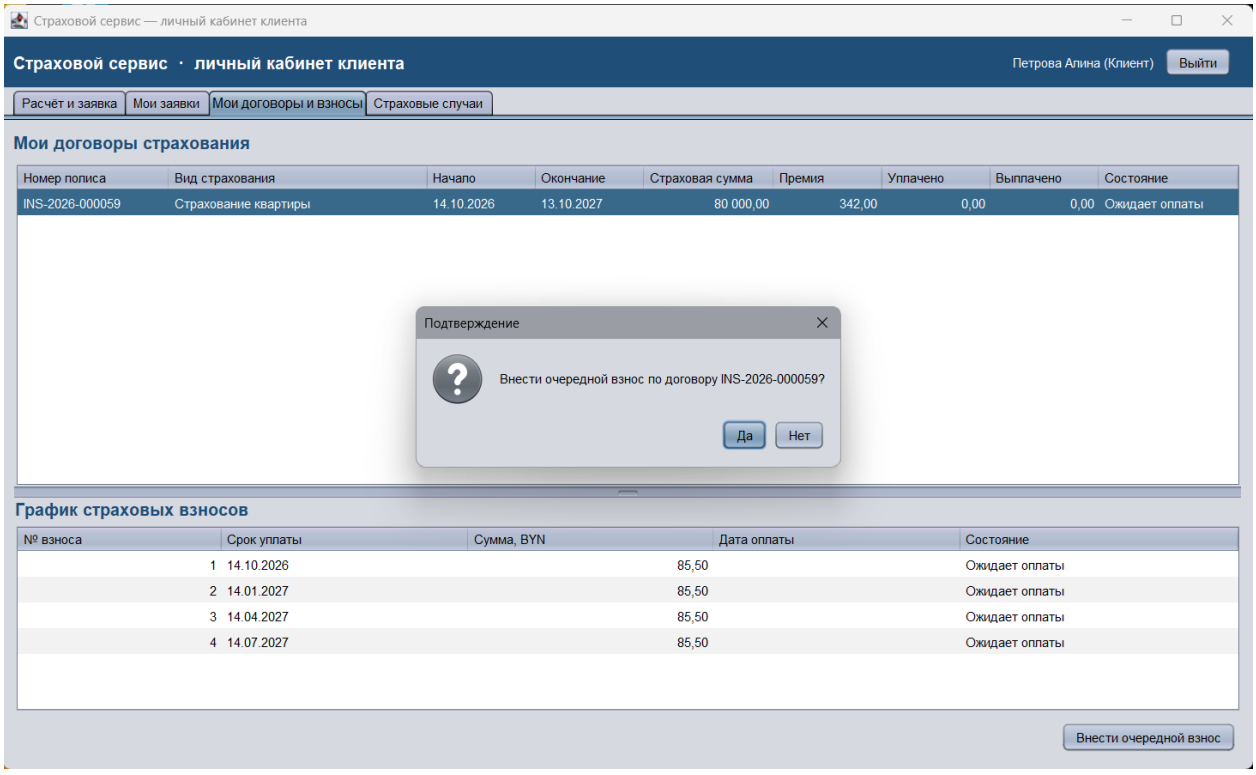


Рисунок 4.10 – Подтверждение внесения взноса

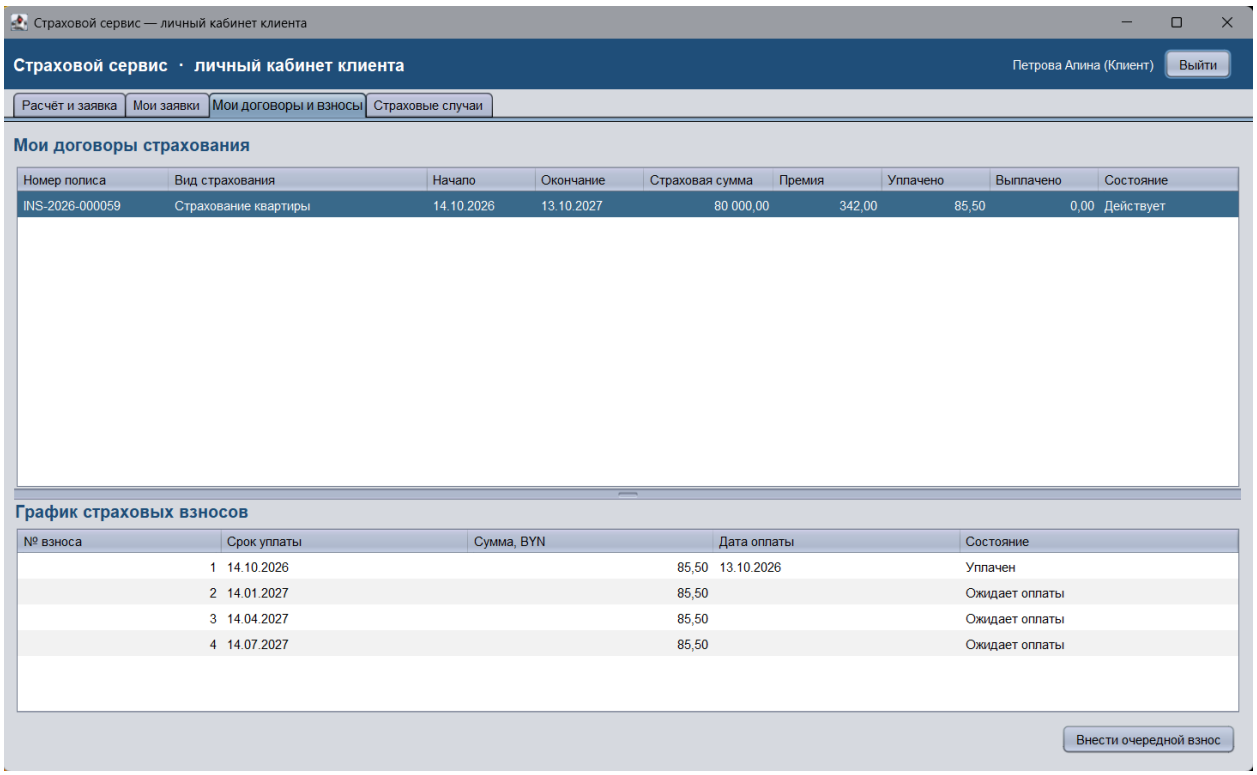


Рисунок 4.11 – Договор после уплаты первого взноса

4.5.7 Заявление о страховом случае (UC7). Клиент Иванов С. А., имеющий действующий договор медицинского страхования выезжающих за рубеж, открывает вкладку «Страховые случаи», выбирает договор, указывает дату события, размер ущерба 640 BYN и описание (рисунок 4.12). После отправки заявление регистрируется (рисунок 4.13) и появляется в списке в состоянии «Заявлен».

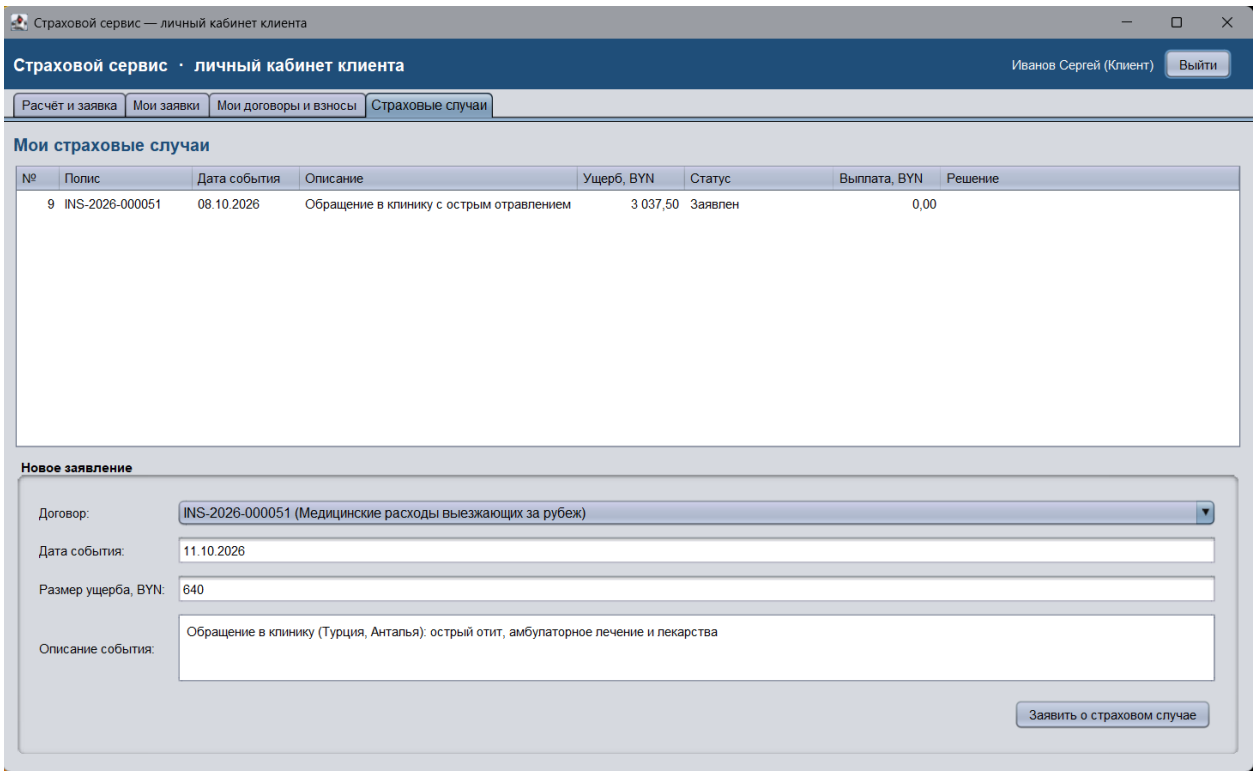


Рисунок 4.12 – Заполнение заявления о страховом случае

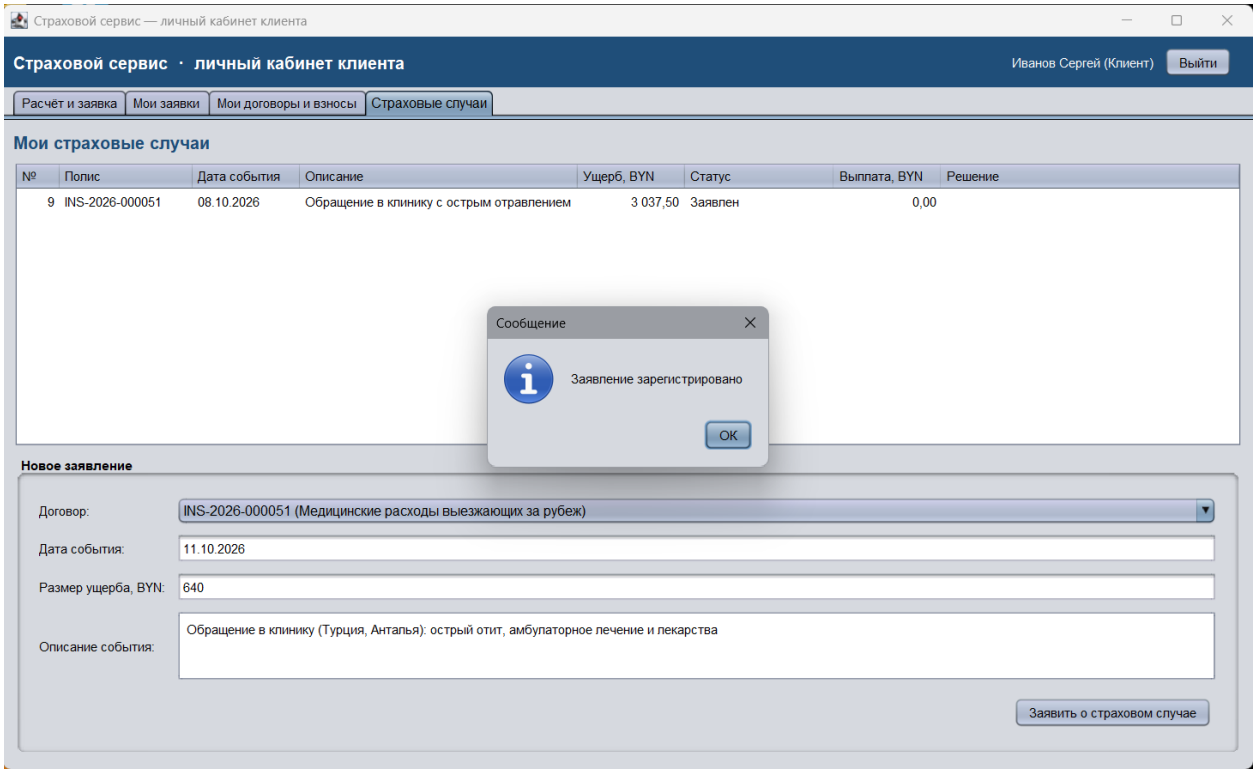


Рисунок 4.13 – Регистрация заявления о страховом случае

4.5.8 Урегулирование страхового случая (UC8). Агент открывает вкладку «Страховые случаи», выбирает заявление, вводит оценённый ущерб

600 BYN и комментарий из акта осмотра и нажимает «Рассчитать выплату». Сервер выполняет алгоритм расчёта выплаты и возвращает решение с обоснованием по шагам (рисунок 4.14). Решение фиксируется после подтверждения агентом (рисунок 4.15); клиент видит сумму выплаты и обоснование в своём списке страховых случаев.

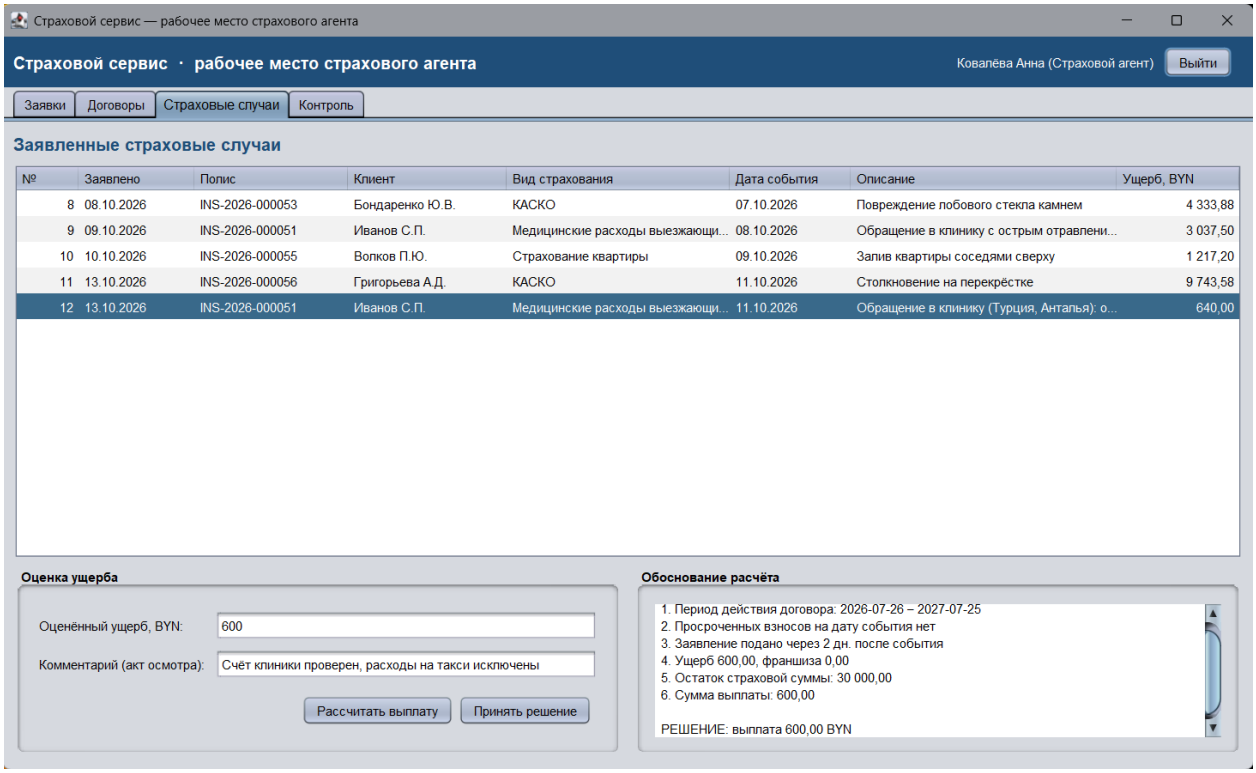


Рисунок 4.14 – Расчёт страховой выплаты с обоснованием

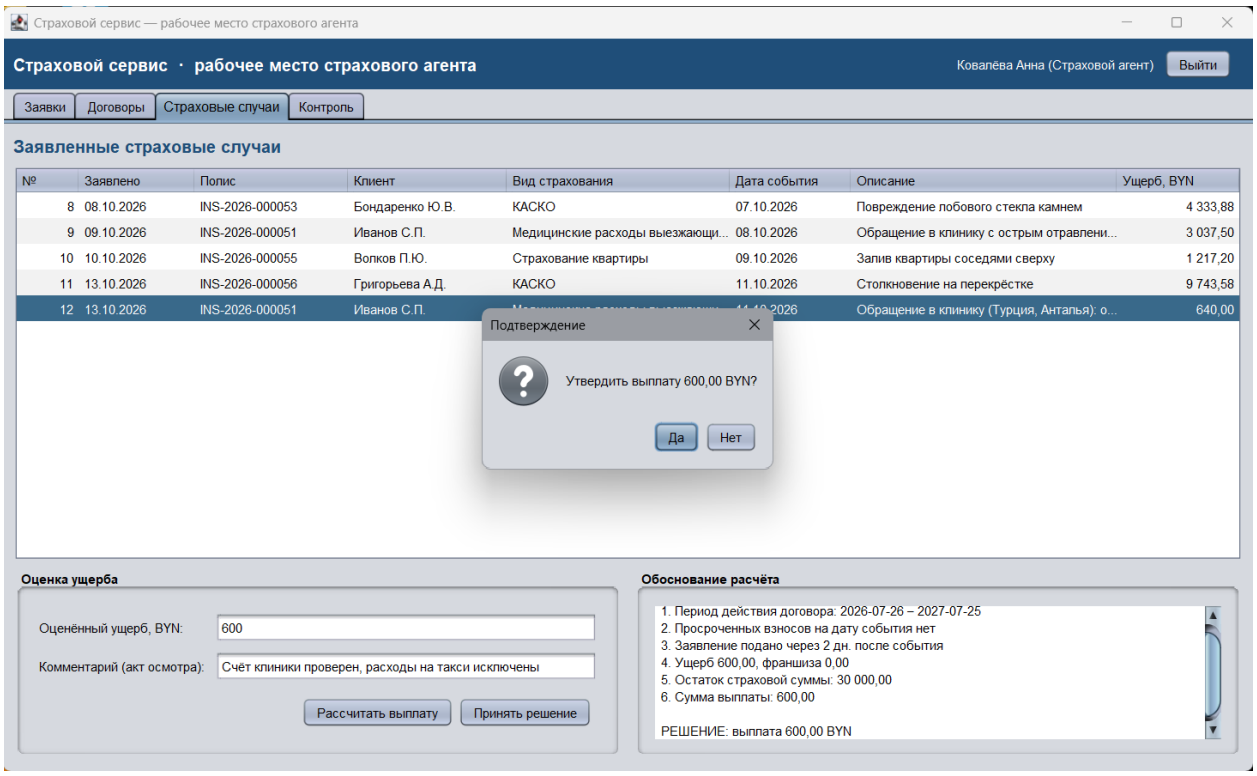


Рисунок 4.15 – Подтверждение решения по страховому случаю

4.5.9 Продление договора (UC9). На вкладке «Договоры» агент выбирает фильтр «Истёк» и договор клиента (рисунок 4.16), нажимает «Продлить» и подтверждает действие. Сервер оформляет новый договор на тот же срок с пересчётом премии по действующим тарифам и истории выплат; новый договор начинается со следующего дня и ожидает оплаты (рисунок 4.17).

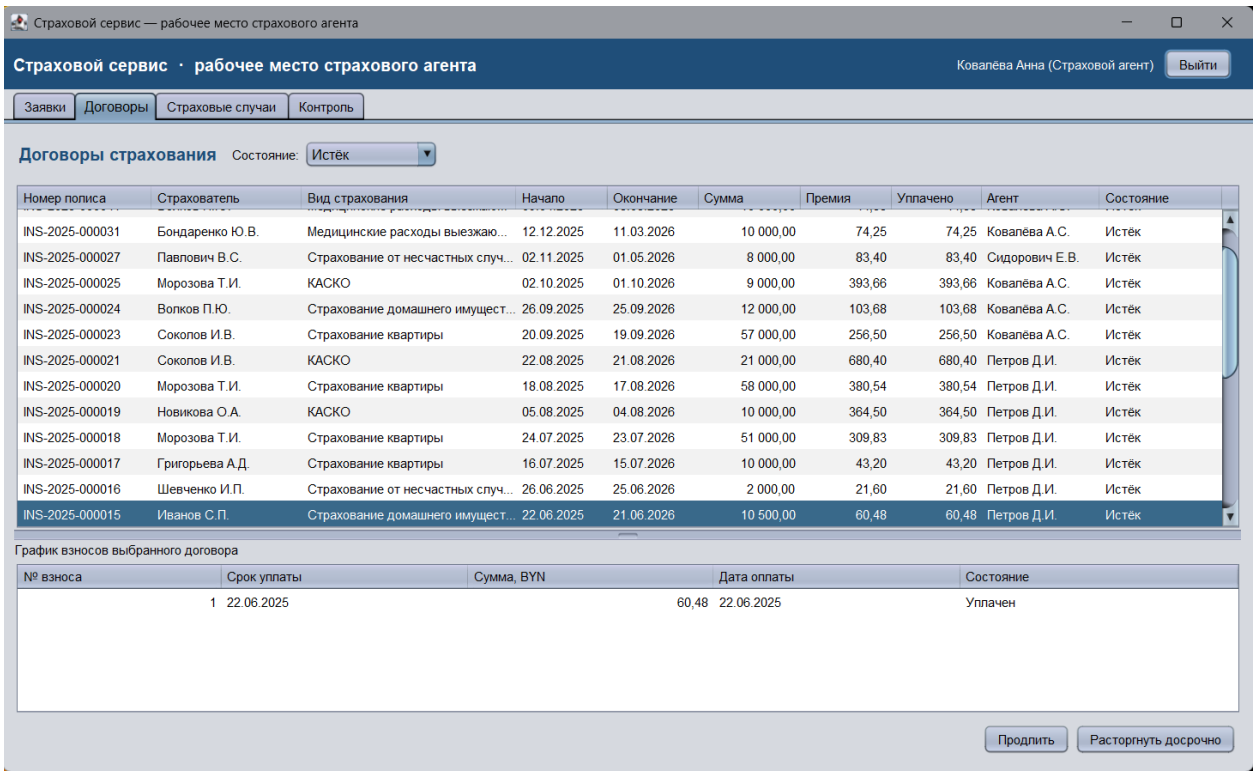


Рисунок 4.16 – Список истёкших договоров

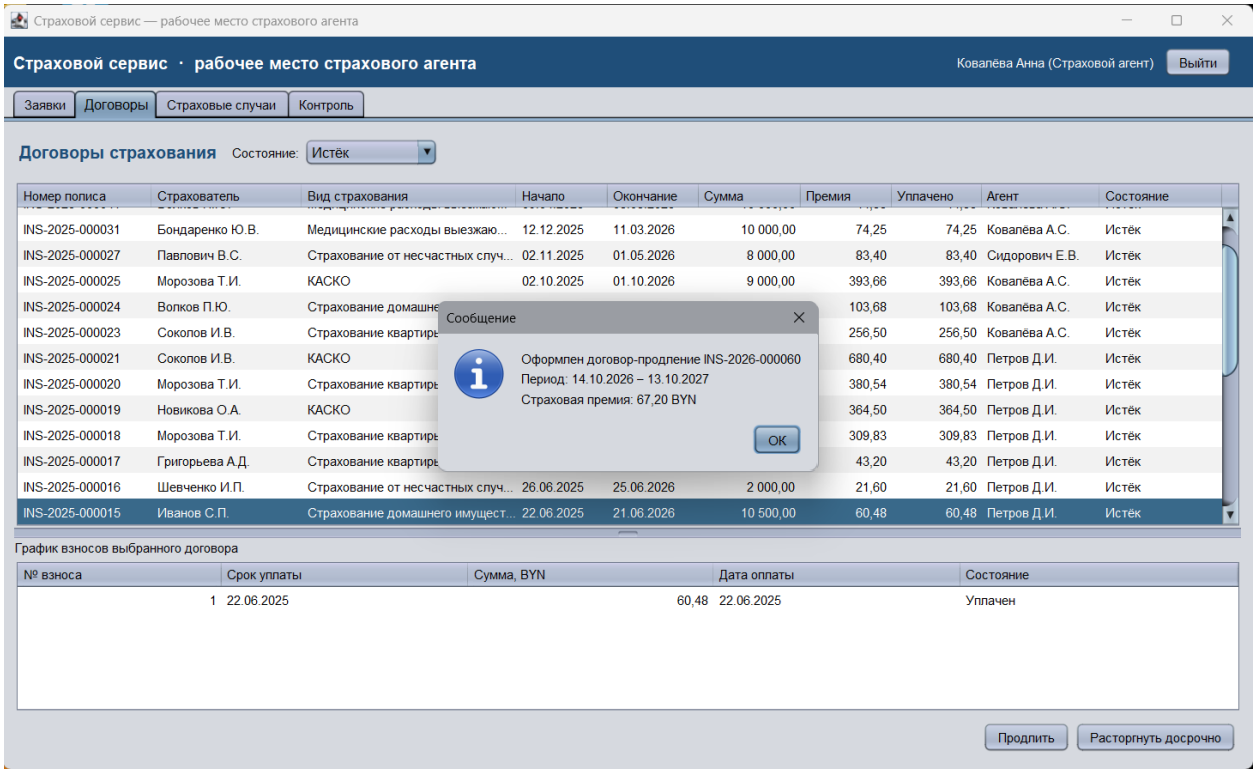


Рисунок 4.17 – Сообщение о продлении договора

4.5.10 Досрочное расторжение договора (UC10). Агент выбирает действующий договор КАСКО и нажимает «Расторгнуть досрочно». Перед

расторжением сервер рассчитывает сумму возврата, а окно подтверждения показывает её составляющие (рисунок 4.18): уплачено 1 475,50 BYN, неистёкший срок 234 из 365 дней, удержание 25 %, к возврату 709,45 BYN. После подтверждения договор переходит в состояние «Расторгнут».

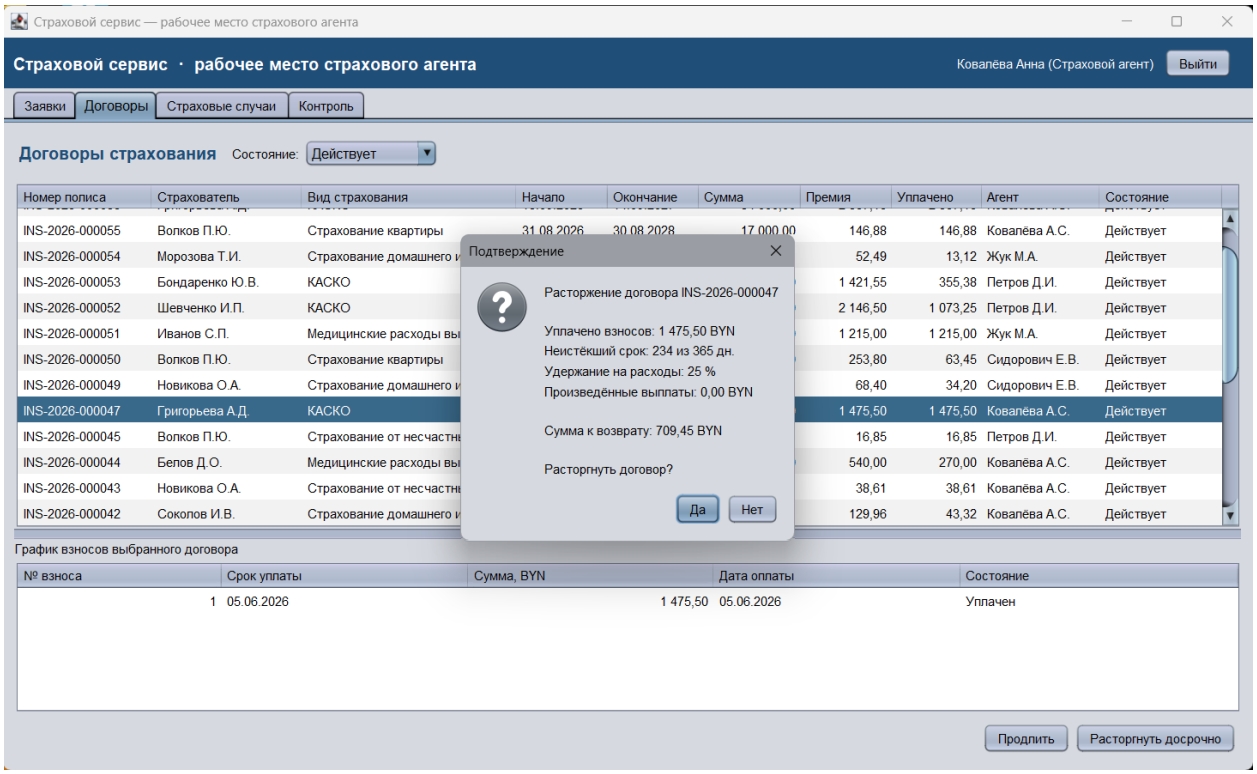


Рисунок 4.18 – Расчёт возврата при досрочном расторжении

4.5.11 Контроль просрочек и истекающих договоров (UC11). Вкладка «Контроль» (рисунок 4.19) содержит просроченные взносы с числом дней просрочки и договоры, срок действия которых заканчивается в ближайшие 30 дней, с телефонами клиентов для напоминания. Кнопка «Пересчитать состояния договоров» применяет правила смены состояний ко всем договорам.

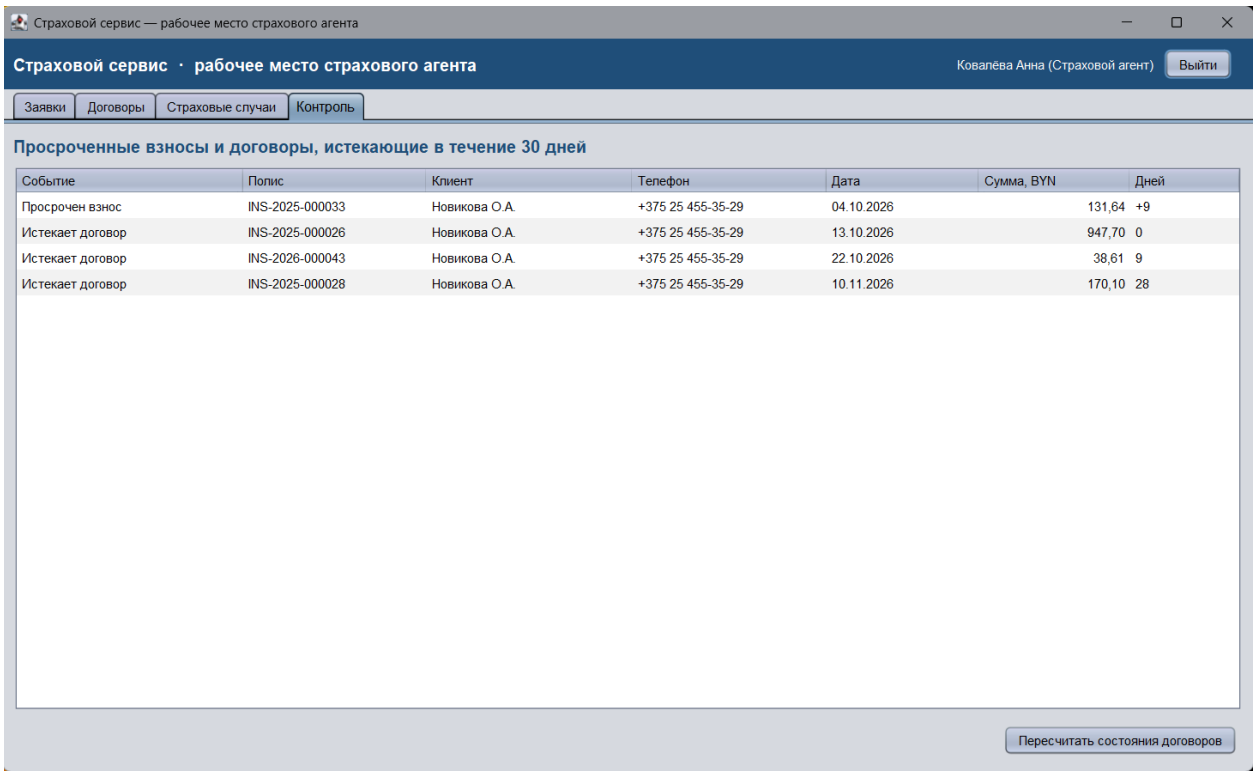


Рисунок 4.19 – Контроль просроченных взносов и истекающих договоров

4.5.12 Управление видами страхования и тарифами (UC13). Администратор выбирает вид страхования в таблице; его параметры переносятся в карточку, где можно изменить тариф, пределы суммы и срока (рисунок 4.20). Сервер проверяет, что тариф находится в интервале от 0 до 1, а минимальная сумма не больше максимальной.

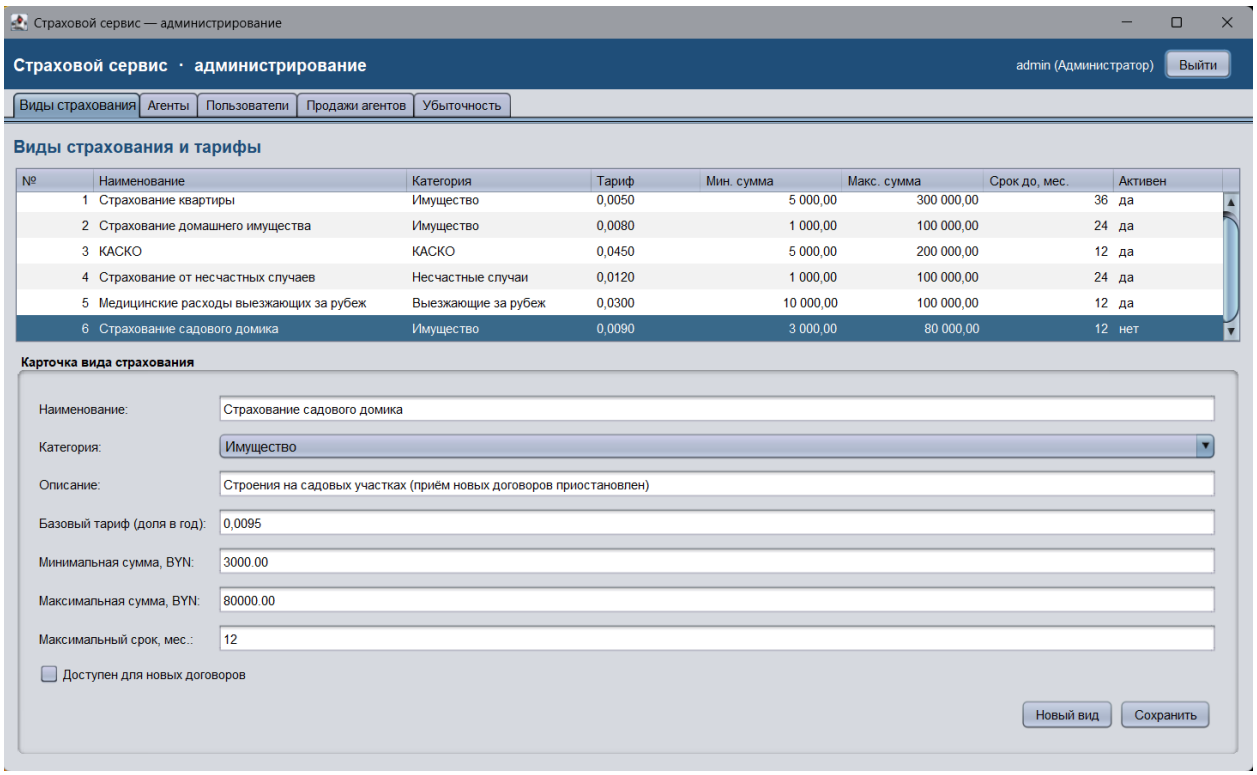


Рисунок 4.20 – Изменение тарифа вида страхования

4.5.13 Управление учётными записями (UC14). На вкладке «Агенты» администратор заполняет данные нового агента и создаёт учётную запись (рисунок 4.21). На вкладке «Пользователи» он блокирует учётную запись клиента (рисунок 4.22); после этого вход под ней невозможен (рисунок 4.25).

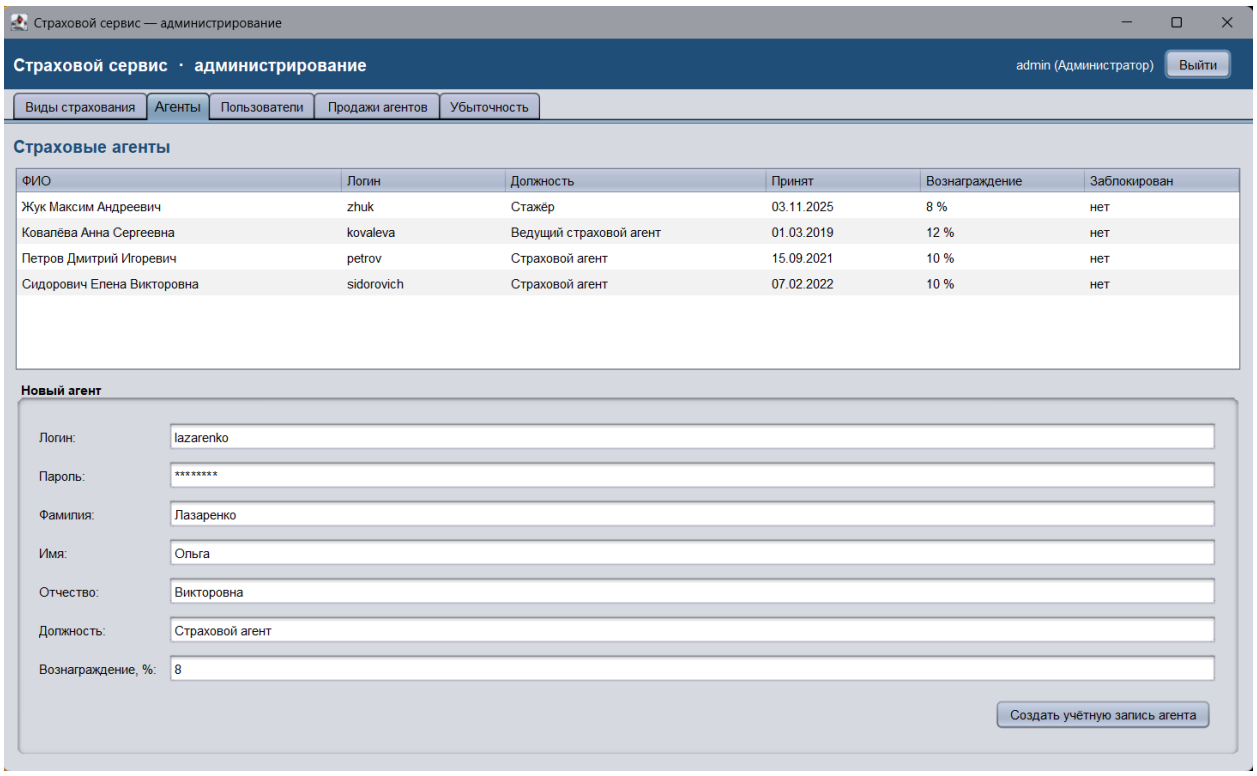


Рисунок 4.21 – Создание учётной записи агента

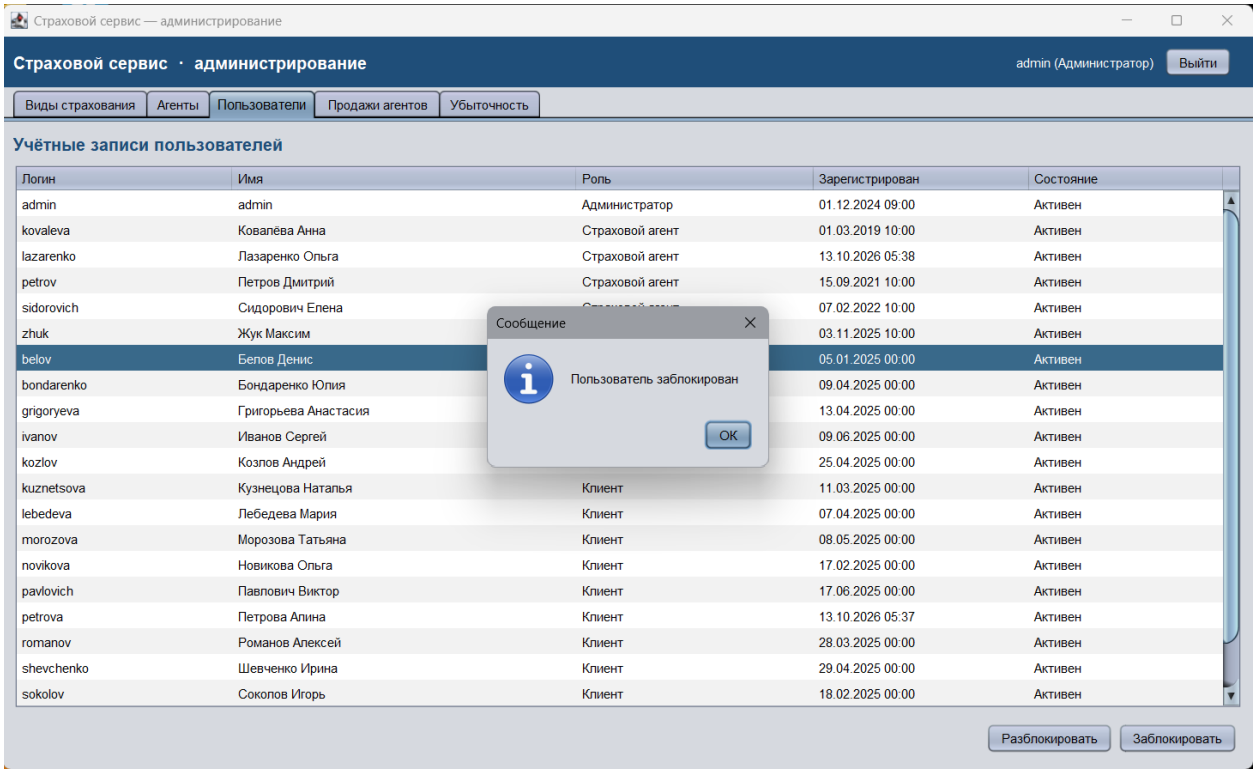


Рисунок 4.22 – Блокировка учётной записи пользователя

4.5.14 Отчёт о продажах агентов (UC15) и выгрузка в файл (UC17). На вкладке «Продажи агентов» администратор задаёт период с 01.01.2026 по

13.10.2026 и нажимает «Сформировать»: таблица и диаграмма показывают число договоров, сумму премий и вознаграждение каждого агента (рисунок 2.9). Кнопка «Выгрузить в файл» открывает окно сохранения (рисунок 4.23); отчёт сохраняется в текстовый файл, который содержит период, дату формирования и таблицу с теми же данными, что и окно отчёта.

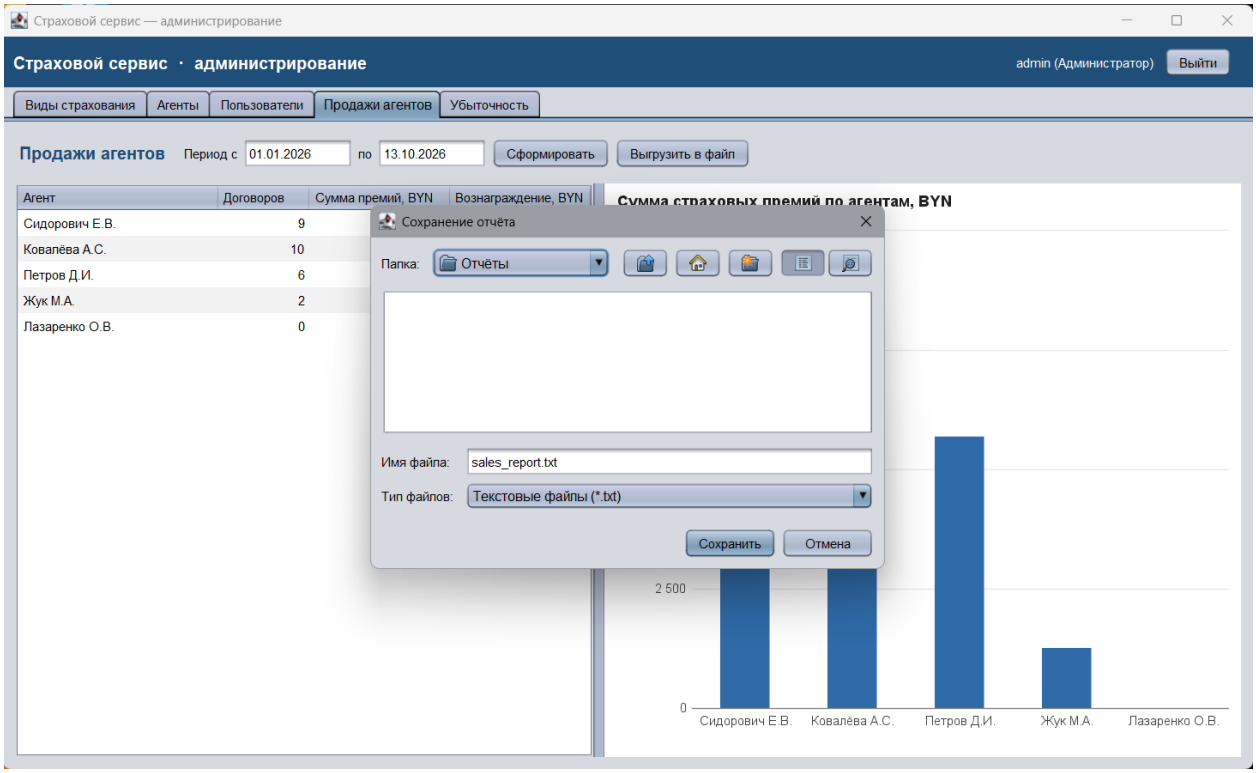


Рисунок 4.23 – Выгрузка отчёта в файл

4.5.15 Анализ убыточности (UC16). На вкладке «Убыточность» за период с 01.01.2025 строятся таблица собранных премий, выплат и коэффициента убыточности по видам страхования, столбчатая диаграмма премий и выплат и круговая диаграмма структуры премий (рисунок 4.24). Высокая убыточность КАСКО указывает на необходимость пересмотра тарифа этого вида.

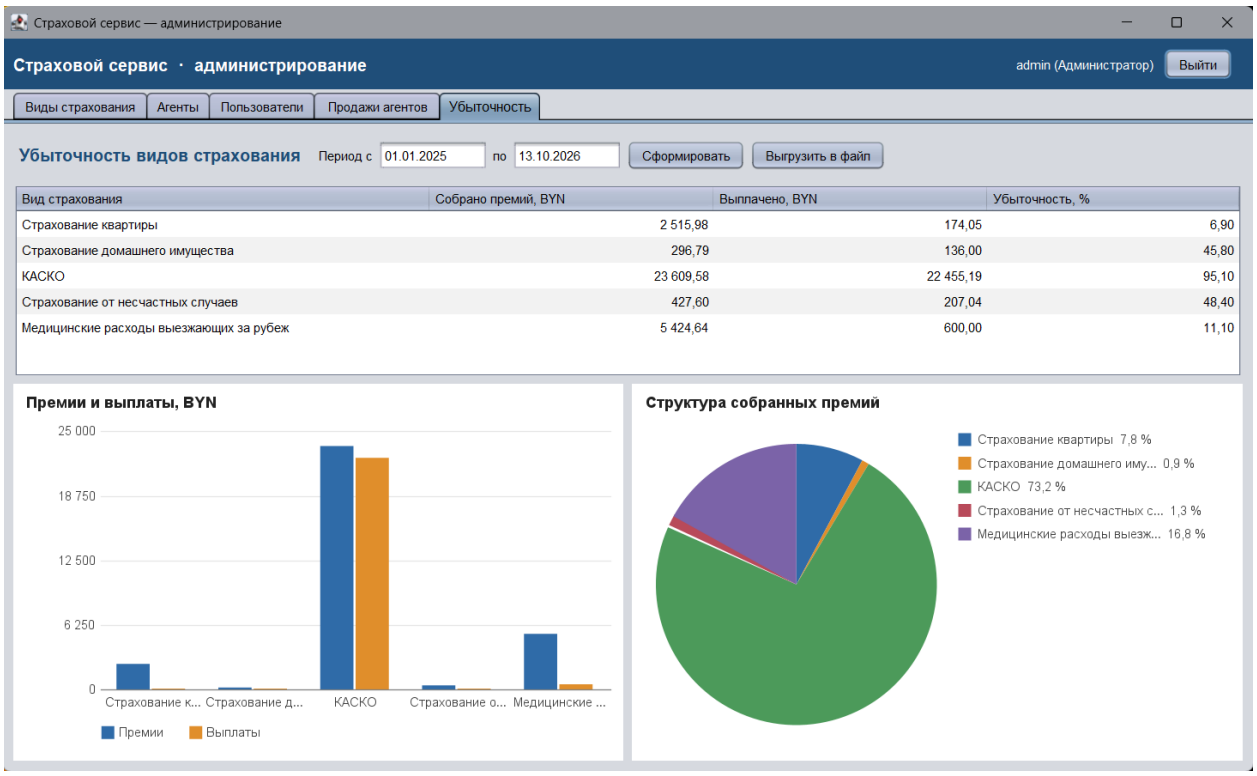


Рисунок 4.24 – Анализ убыточности видов страхования

4.5.16 Вход заблокированного пользователя. При попытке входа под заблокированной учётной записью сервер отказывает во входе (рисунок 4.25).

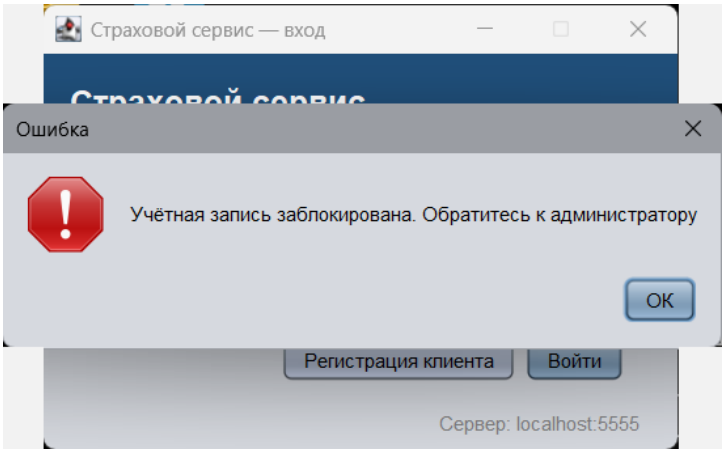


Рисунок 4.25 – Вход заблокированного пользователя

Соответствие вариантов использования и рисунков сквозного примера приведено в таблице 4.3. Все семнадцать вариантов использования выполнены успешно, что подтверждает соответствие программного средства функциональным требованиям подраздела 1.3.

Таблица 4.3 – Варианты использования в сквозном примере

Код	Вариант использования	Рисунок
UC1	Регистрация клиента	4.3
UC2	Авторизация	2.7, 4.2, 4.25
UC3	Расчёт стоимости страхования	2.10, 4.4
UC4	Подача заявки на страхование	4.5, 4.6
UC5	Рассмотрение заявки и оформление полиса	2.8, 4.7, 4.8
UC6	Внесение страхового взноса	4.10, 4.11
UC7	Подача заявления о страховом случае	4.12, 4.13
UC8	Урегулирование страхового случая	4.14, 4.15
UC9	Продление договора	4.16, 4.17
UC10	Досрочное расторжение договора	4.18
UC11	Контроль просрочек и истекающих договоров	4.19
UC12	Просмотр договоров и графика взносов	4.9, 4.11
UC13	Управление видами страхования и тарифами	4.20
UC14	Управление учётными записями	4.21, 4.22
UC15	Отчёт о продажах агентов	2.9
UC16	Анализ убыточности	4.24
UC17	Выгрузка отчёта в файл	4.23

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработано программное средство «Страховой сервис» для поддержки процесса оказания страховых услуг населению. Цель курсового проекта достигнута, все поставленные задачи решены.

Проведён анализ предметной области: описан процесс оказания страховых услуг физическим лицам, построены модели бизнес-процессов в нотации BPMN 2.0 и функциональная модель в нотации IDEF0 из четырёх уровней, сформулированы 17 функциональных требований и нефункциональные требования. Разработана информационная модель в нотации IDEF1X; база данных из девяти таблиц приведена к третьей нормальной форме, ссылочная целостность и допустимые значения обеспечиваются внешними ключами и ограничениями CHECK. Построены UML-диаграммы вариантов использования, состояний, последовательности, классов и развёртывания.

Спроектирована и реализована трёхуровневая архитектура: оконный клиент на Swing, многопоточный сервер с пулом потоков и пулом соединений с СУБД, база данных PostgreSQL. Вся бизнес-логика выполняется на сервере: расчёт страховой премии с семью поправочными коэффициентами, формирование графика взносов, проверка условий и расчёт страховой выплаты, расчёт возврата части премии при расторжении, автоматическая смена состояний договоров. Программное средство поддерживает три роли и семнадцать вариантов использования, двенадцать из которых реализуют нетривиальную бизнес-логику. В реализации использованы собственные иерархии классов, инкапсуляция, перегрузка и переопределение методов, сериализация, интерфейсы и абстрактные классы, статические методы, обработка исключений и шесть паттернов проектирования.

Объём исходного кода составляет около 5 500 строк в 99 классах и 1 300 строк тестов. Работоспособность подтверждена 29 модульными тестами, сквозным сценарием из 37 проверок, нагрузочным тестированием (200 одновременных клиентов без ошибок) и сквозным примером работы через пользовательский интерфейс. Программное средство работает в Windows 7 и выше при наличии среды выполнения Java 8 или новее; исходный код хранится в публичном репозитории GitHub.

Направления дальнейшего развития: подключение онлайн-оплаты взносов через платёжную систему, уведомления клиентов о сроках взносов по электронной почте, загрузка фотографий и документов к заявлениям о страховых случаях, журнал действий пользователей, переход на

неблокирующий ввод-вывод для обслуживания тысяч одновременных подключений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Дипломные проекты (работы). Общие требования : стандарт предприятия СТП 01-2024. – Минск : БГУИР, 2024. – 178 с.

[2] Гражданский кодекс Республики Беларусь [Электронный ресурс] : 7 дек. 1998 г., № 218-З // Национальный правовой Интернет-портал Республики Беларусь. – Режим доступа : https://pravo.by. – Дата доступа : 13.10.2026.

[3] О страховой деятельности [Электронный ресурс] : Указ Президента Респ. Беларусь, 25 авг. 2006 г., № 530 // Национальный правовой Интернет-портал Республики Беларусь. – Режим доступа : https://pravo.by. – Дата доступа : 13.10.2026.

[4] Белгосстрах [Электронный ресурс] : официальный сайт. – Режим доступа : https://bgs.by. – Дата доступа : 13.10.2026.

[5] Business Process Model and Notation (BPMN). Version 2.0.2 [Электронный ресурс] / Object Management Group. – 2013. – Режим доступа : https://www.omg.org/spec/BPMN/2.0.2/. – Дата доступа : 13.10.2026.

[6] Информационные технологии поддержки жизненного цикла продукции. Методология функционального моделирования : рекомендации по стандартизации Р 50.1.028–2001. – М. : Госстандарт России, 2001.

[7] IEEE Standard for Conceptual Modeling Language Syntax and Semantics for IDEF1X97 (IDEFobject) : IEEE Std 1320.2–1998. – New York : IEEE, 1998.

[8] Фаулер, М. UML. Основы. Краткое руководство по стандартному языку объектного моделирования / М. Фаулер. – 3-е изд. – СПб. : Символ-Плюс, 2004. – 192 с.

[9] Java Platform, Standard Edition 8. API Specification [Электронный ресурс] / Oracle. – Режим доступа : https://docs.oracle.com/javase/8/docs/api/. – Дата доступа : 13.10.2026.

[10] Java Concurrency на практике / Б. Гётц [и др.]. – СПб. : Питер, 2020. – 464 с.

[11] PostgreSQL 16 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. – Режим доступа : https://www.postgresql.org/docs/16/. – Дата доступа : 13.10.2026.

[12] Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма [и др.]. – СПб. : Питер, 2015. – 368 с.

[13] OMG Unified Modeling Language (OMG UML). Version 2.5.1 [Электронный ресурс] / Object Management Group. – 2017. – Режим доступа : https://www.omg.org/spec/UML/2.5.1. – Дата доступа : 13.10.2026.

[14] Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : ГОСТ 19.701–90. – Введ. 01.01.1992. – М. : Изд-во стандартов, 1991.

[15] Creating a GUI With Swing [Электронный ресурс] : The Java Tutorials / Oracle. – Режим доступа : https://docs.oracle.com/javase/tutorial/uiswing/. – Дата доступа : 13.10.2026.

[16] Шилдт, Г. Java. Полное руководство / Г. Шилдт. – 10-е изд. – М. : Диалектика, 2018. – 1488 с.

[17] Арнольд, К. Язык программирования Java / К. Арнольд, Дж. Гослинг, Д. Холмс. – 3-е изд. – М. : Вильямс, 2001. – 624 с.

[18] Эккель, Б. Философия Java / Б. Эккель. – 4-е изд. – СПб. : Питер, 2011. – 640 с.

[19] Блох, Дж. Java: эффективное программирование / Дж. Блох. – 3-е изд. – СПб. : Диалектика, 2019. – 464 с.

[20] Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – М. : Вильямс, 2005. – 1328 с.

[21] PostgreSQL JDBC Driver. Documentation [Электронный ресурс]. – Режим доступа : https://jdbc.postgresql.org/documentation/. – Дата доступа : 13.10.2026.

[22] Apache Maven Project [Электронный ресурс] / The Apache Software Foundation. – Режим доступа : https://maven.apache.org. – Дата доступа : 13.10.2026.

[23] JUnit 4 [Электронный ресурс]. – Режим доступа : https://junit.org/junit4/. – Дата доступа : 13.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)
Отчет о проверке на заимствования
в системе «Антиплагиат»

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программного кода алгоритмов, реализующих основную бизнес-логику

```
// Файл server/logic/PremiumCalculator.java
package by.bsuir.insurance.server.logic;

/**
 * Расчёт страховой премии: П = С × Т × Кв × Кс × Кбм × Кф × Кт.
 */
public class PremiumCalculator {
    private static final MathContext MC = MathContext.DECIMAL64;
    private static final BigDecimal TWELVE = new BigDecimal("12");

    /**
     * Полный расчёт по заявке.
     *
     * @param type         вид страхования
     * @param application  параметры страхования (сумма, срок, франшиза,
           описание объекта)
     * @param client       страхователь
     * @param startDate    дата начала действия договора
     * @param claimsIn2Years число страховых выплат клиенту за последние два
           года
     */
    public PremiumQuote calculate(InsuranceType type, Application
            application, Client client,
                                  LocalDate startDate, int claimsIn2Years)
                                        throws ValidationException {
        validate(type, application);
        int age = client.getAge(startDate);
        BigDecimal sum = application.getRequestedSum();

        PremiumQuote quote = new PremiumQuote();
        quote.setInsuredSum(Money.round(sum));
        quote.setBaseRate(type.getBaseRate());

        BigDecimal kAge = ageFactor(age);
        BigDecimal kTerm = termFactor(application.getTermMonths());
        BigDecimal kBonus = bonusMalusFactor(claimsIn2Years);
        BigDecimal kFranchise =
                franchiseFactor(application.getFranchisePercent());
        PremiumStrategy strategy = PremiumStrategyFactory.forType(type);
        BigDecimal kType = strategy.typeFactor(application, client, age);

        quote.addFactor("Кв (возраст " + age + ")", kAge);
        quote.addFactor("Кс (срок " + application.getTermMonths() + "
                мес.)", kTerm);
        quote.addFactor("Кбм (выплат за 2 года: " + claimsIn2Years + ")",
                kBonus);
        quote.addFactor("Кф (франшиза " + application.getFranchisePercent()
                + " %)", kFranchise);
        quote.addFactor(strategy.factorName(), kType);
```

```
    BigDecimal premium = sum.multiply(type.getBaseRate(), MC)
            .multiply(kAge, MC).multiply(kTerm, MC).multiply(kBonus, MC)
            .multiply(kFranchise, MC).multiply(kType, MC);
    quote.setPremium(Money.round(premium));
    quote.setFranchise(franchiseAmount(sum,
            application.getFranchisePercent()));
    return quote;
}

/** Упрощённый расчёт без истории клиента (перегрузка для калькулятора).
        */
public BigDecimal calculate(InsuranceType type, BigDecimal sum, int
        termMonths, int age) {
    BigDecimal p = sum.multiply(type.getBaseRate(),
            MC).multiply(ageFactor(age), MC)
            .multiply(termFactor(termMonths), MC);
    return Money.round(p);
}

public void validate(InsuranceType type, Application a) throws
        ValidationException {
    if (!type.isActive()) {
        throw new ValidationException("Вид страхования «" +
                type.getName() + "» недоступен для новых договоров");
    }
    BigDecimal sum = a.getRequestedSum();
    if (sum == null || sum.compareTo(type.getMinSum()) < 0 ||
            sum.compareTo(type.getMaxSum()) > 0) {
        throw new ValidationException("Страховая сумма должна быть от "
                + Money.format(type.getMinSum())
                + " до " + Money.format(type.getMaxSum()) + " BYN");
    }
    if (a.getTermMonths() < 1 || a.getTermMonths() >
            type.getMaxTermMonths()) {
        throw new ValidationException("Срок страхования должен быть от 1
                до " + type.getMaxTermMonths() + " мес.");
    }
    InstallmentScheduler.validate(a.getTermMonths(),
            a.getInstallments());
    int f = a.getFranchisePercent();
    if (f != 0 && f != 1 && f != 2 && f != 5) {
        throw new ValidationException("Допустимая франшиза: 0, 1, 2 или
                5 %");
    }
}

/** Возрастной коэффициент Кв. */
public static BigDecimal ageFactor(int age) {
    if (age < 25) {
        return new BigDecimal("1.20");
    }
    if (age > 60) {
        return new BigDecimal("1.35");
    }
    return new BigDecimal("1.00");
}
```

```
    /** Коэффициент срока Кс: доля года, для краткосрочных договоров
            надбавка 10 %. */
    public static BigDecimal termFactor(int termMonths) {
        BigDecimal k = new BigDecimal(termMonths).divide(TWELVE, MC);
        if (termMonths < 12) {
            k = k.multiply(new BigDecimal("1.10"), MC);
        }
        return k;
    }

    /** Коэффициент бонус-малус Кбм по числу выплат за два года. */
    public static BigDecimal bonusMalusFactor(int claims) {
        if (claims == 0) {
            return new BigDecimal("0.90");
        }
        return claims == 1 ? new BigDecimal("1.00") : new
                BigDecimal("1.25");
    }

    /** Коэффициент франшизы Кф. */
    public static BigDecimal franchiseFactor(int percent) {
        switch (percent) {
            case 0: return new BigDecimal("1.00");
            case 1: return new BigDecimal("0.95");
            case 2: return new BigDecimal("0.90");
            case 5: return new BigDecimal("0.80");
            default: throw new IllegalArgumentException("Допустимая
                    франшиза: 0, 1, 2 или 5 %");
        }
    }

    public static BigDecimal franchiseAmount(BigDecimal sum, int percent) {
        return Money.round(sum.multiply(new
                BigDecimal(percent)).divide(Money.HUNDRED, MC));
    }
}

// Файл server/logic/PremiumStrategy.java
package by.bsuir.insurance.server.logic;

/**
 * Стратегия расчёта поправочного коэффициента, специфичного для вида
        страхования
 * (паттерн «Стратегия»).
 */
public interface PremiumStrategy {

    /** Коэффициент вида страхования для заявки и страхователя. */
    BigDecimal typeFactor(Application application, Client client, int
            ageAtStart);

    /** Пояснение к коэффициенту для показа пользователю. */
    String factorName();
}
```

```
// Файл server/logic/AbstractPremiumStrategy.java
package by.bsuir.insurance.server.logic;

/**
 * Базовая стратегия: коэффициент равен 1, если описание объекта не содержит
 * ключевых слов повышенного риска; наследники задают свои ключевые слова и
       надбавки.
 */
public abstract class AbstractPremiumStrategy implements PremiumStrategy {

    @Override
    public BigDecimal typeFactor(Application application, Client client, int
            ageAtStart) {
        BigDecimal factor = extraFactor(application, client, ageAtStart);
        String description = application.getObjectDescription() == null
                ? "" : application.getObjectDescription().toLowerCase(Locale
                        .ROOT);
        for (String keyword : riskKeywords()) {
            if (description.contains(keyword)) {
                factor = factor.multiply(riskSurcharge());
                break;
            }
        }
        return factor;
    }

    /** Дополнительный множитель, зависящий от параметров договора
            (переопределяется). */
    protected BigDecimal extraFactor(Application application, Client client,
            int ageAtStart) {
        return BigDecimal.ONE;
    }

    /** Ключевые слова в описании объекта, означающие повышенный риск. */
    protected String[] riskKeywords() {
        return new String[0];
    }

    /** Надбавка за повышенный риск. */
    protected BigDecimal riskSurcharge() {
        return BigDecimal.ONE;
    }
}

// Файл server/logic/PropertyPremiumStrategy.java
package by.bsuir.insurance.server.logic;

/** Страхование имущества: скидка за крупную сумму, надбавка за деревянные
        строения. */
public class PropertyPremiumStrategy extends AbstractPremiumStrategy {
    private static final BigDecimal LARGE_SUM = new BigDecimal("100000");

    @Override
    protected BigDecimal extraFactor(Application application, Client client,
            int ageAtStart) {
        return application.getRequestedSum().compareTo(LARGE_SUM) > 0 ? new
```

```
                BigDecimal("0.95") : BigDecimal.ONE;
    }

    @Override
    protected String[] riskKeywords() {
        return new String[] {"деревян"};
    }

    @Override
    protected BigDecimal riskSurcharge() {
        return new BigDecimal("1.20");
    }

    @Override
    public String factorName() {
        return "Кт (сумма, материал стен)";
    }
}

// Файл server/logic/CascoPremiumStrategy.java
package by.bsuir.insurance.server.logic;

/** КАСКО: надбавка для молодых водителей. */
public class CascoPremiumStrategy extends AbstractPremiumStrategy {

    @Override
    protected BigDecimal extraFactor(Application application, Client client,
            int ageAtStart) {
        return ageAtStart < 25 ? new BigDecimal("1.15") : BigDecimal.ONE;
    }

    @Override
    public String factorName() {
        return "Кт (стаж водителя)";
    }
}

// Файл server/logic/AccidentPremiumStrategy.java
package by.bsuir.insurance.server.logic;

/** Страхование от несчастных случаев: надбавка за занятия спортом. */
public class AccidentPremiumStrategy extends AbstractPremiumStrategy {

    @Override
    protected String[] riskKeywords() {
        return new String[] {"спорт"};
    }

    @Override
    protected BigDecimal riskSurcharge() {
        return new BigDecimal("1.30");
    }

    @Override
    public String factorName() {
        return "Кт (занятия спортом)";
```

```
    }
}

// Файл server/logic/TravelPremiumStrategy.java
package by.bsuir.insurance.server.logic;

/** Медицинские расходы выезжающих за рубеж: надбавка за активный отдых. */
public class TravelPremiumStrategy extends AbstractPremiumStrategy {

    @Override
    protected String[] riskKeywords() {
        return new String[] {"горнолыж", "спорт"};
    }

    @Override
    protected BigDecimal riskSurcharge() {
        return new BigDecimal("1.50");
    }

    @Override
    public String factorName() {
        return "Кт (активный отдых)";
    }
}

// Файл server/logic/PremiumStrategyFactory.java
package by.bsuir.insurance.server.logic;

/** Фабрика стратегий расчёта премии (паттерн «Фабрика»). */
public final class PremiumStrategyFactory {
    private static final Map<InsuranceCategory, PremiumStrategy> STRATEGIES
            = new EnumMap<>(InsuranceCategory.class);

    static {
        STRATEGIES.put(InsuranceCategory.PROPERTY, new
                PropertyPremiumStrategy());
        STRATEGIES.put(InsuranceCategory.CASCO, new CascoPremiumStrategy());
        STRATEGIES.put(InsuranceCategory.ACCIDENT, new
                AccidentPremiumStrategy());
        STRATEGIES.put(InsuranceCategory.TRAVEL, new
                TravelPremiumStrategy());
    }

    private PremiumStrategyFactory() {
    }

    public static PremiumStrategy forType(InsuranceType type) {
        PremiumStrategy strategy = STRATEGIES.get(type.getCategory());
        if (strategy == null) {
            throw new IllegalArgumentException("Нет стратегии для категории
                    " + type.getCategory());
        }
        return strategy;
    }
}
```

```
// Файл server/logic/InstallmentScheduler.java
package by.bsuir.insurance.server.logic;

/** Формирование графика страховых взносов. */
public final class InstallmentScheduler {

    private InstallmentScheduler() {
    }

    public static void validate(int termMonths, int installments) throws
            ValidationException {
        if (installments != 1 && installments != 2 && installments != 4 &&
                installments != 12) {
            throw new ValidationException("Число взносов может быть 1, 2, 4
                    или 12");
        }
        if (installments > termMonths || termMonths % installments != 0) {
            throw new ValidationException("Срок " + termMonths + " мес.
                    нельзя разделить на " + installments + " равных
                    взносов");
        }
    }

    /**
     * Делит премию на равные взносы; копейки, оставшиеся от деления,
            добавляются к последнему взносу.
     * Первый взнос — в дату начала договора, следующие — через равные
            интервалы.
     */
    public static List<Payment> build(long policyId, BigDecimal premium, int
            installments,
                                      LocalDate startDate, int termMonths)
                                        throws ValidationException {
        validate(termMonths, installments);
        int stepMonths = termMonths / installments;
        BigDecimal base = premium.divide(new BigDecimal(installments), 2,
                RoundingMode.DOWN);
        List<Payment> list = new ArrayList<>();
        for (int i = 1; i <= installments; i++) {
            BigDecimal amount = i < installments
                    ? base : premium.subtract(base.multiply(new
                            BigDecimal(installments - 1)));
            list.add(new Payment(policyId, i, amount,
                    startDate.plusMonths((long) stepMonths * (i - 1))));
        }
        return list;
    }
}

// Файл server/logic/PayoutCalculator.java
package by.bsuir.insurance.server.logic;

/**
 * Алгоритм расчёта страховой выплаты (функция A44 модели IDEF0).
 */
public class PayoutCalculator {
```

```
/** Максимальный срок подачи заявления, дней. */
public static final int REPORT_DEADLINE_DAYS = 30;

/**
 * @param policy       договор
 * @param payments     график взносов договора
 * @param claim        страховой случай
 * @param damage       оценённый агентом размер ущерба
 * @param paidBefore   сумма выплат по договору до этого случая
 */
public PayoutDecision calculate(Policy policy, List<Payment> payments,
        Claim claim,
                                BigDecimal damage, BigDecimal
                                    paidBefore) {
    List<String> steps = new ArrayList<>();
    LocalDate incident = claim.getIncidentDate();

    // 1. Договор действовал на дату события
    steps.add("Период действия договора: " + policy.getStartDate() + " –
            " + policy.getEndDate());
    if (!policy.isActiveOn(incident)) {
        return PayoutDecision.reject("событие произошло вне периода
                действия договора", steps);
    }
    LocalDateTime terminatedAt = policy.getTerminatedAt();
    if (terminatedAt != null &&
            !incident.isBefore(terminatedAt.toLocalDate())) {
        return PayoutDecision.reject("договор был расторгнут до даты
                события", steps);
    }
    // 2. Нет просроченных взносов на дату события
    for (Payment p : payments) {
        if (p.isOverdue(incident)) {
            steps.add("Взнос № " + p.getInstallmentNo() + " со сроком "
                    + p.getDueDate() + " не был уплачен");
            return PayoutDecision.reject("на дату события имелись
                    просроченные страховые взносы", steps);
        }
    }
    steps.add("Просроченных взносов на дату события нет");
    // 3. Срок подачи заявления
    long days = ChronoUnit.DAYS.between(incident,
            claim.getReportDate());
    steps.add("Заявление подано через " + days + " дн. после события");
    if (days > REPORT_DEADLINE_DAYS) {
        return PayoutDecision.reject("заявление подано позднее " +
                REPORT_DEADLINE_DAYS + " дней с даты события", steps);
    }
    // 4. Франшиза
    BigDecimal franchise = Money.nz(policy.getFranchise());
    steps.add("Ущерб " + Money.format(damage) + ", франшиза " +
            Money.format(franchise));
    if (damage.compareTo(franchise) <= 0) {
        return PayoutDecision.reject("размер ущерба не превышает
                франшизу", steps);
    }
```

```
        BigDecimal payout = damage.subtract(franchise);
        // 5. Ограничение остатком страховой суммы
        BigDecimal remaining =
                policy.getInsuredSum().subtract(Money.nz(paidBefore));
        steps.add("Остаток страховой суммы: " + Money.format(remaining));
        if (remaining.signum() <= 0) {
            return PayoutDecision.reject("страховая сумма исчерпана
                    предыдущими выплатами", steps);
        }
        boolean exhausts = false;
        if (payout.compareTo(remaining) >= 0) {
            payout = remaining;
            exhausts = true;
            steps.add("Выплата ограничена остатком страховой суммы, договор
                    будет исполнен");
        }
        payout = Money.round(payout);
        steps.add("Сумма выплаты: " + Money.format(payout));
        return PayoutDecision.approve(payout, exhausts, steps);
    }
}

// Файл server/logic/RefundCalculator.java
package by.bsuir.insurance.server.logic;

/**
 * Возврат части премии при досрочном расторжении:
 * В = уплачено × (неистёкшие дни / все дни) × (1 − доля расходов) −
        выплаты, но не меньше 0.
 */
public class RefundCalculator {
    /** Доля расходов на ведение дела, удерживаемая страховщиком. */
    public static final BigDecimal EXPENSES_SHARE = new BigDecimal("0.25");

    public RefundCalculation calculate(Policy policy, BigDecimal paid,
            BigDecimal payouts, LocalDate terminationDate) {
        long total = ChronoUnit.DAYS.between(policy.getStartDate(),
                policy.getEndDate()) + 1;
        long remaining = ChronoUnit.DAYS.between(terminationDate,
                policy.getEndDate());
        if (terminationDate.isBefore(policy.getStartDate())) {
            remaining = total;
        }
        remaining = Math.max(0, Math.min(total, remaining));
        BigDecimal refund = Money.nz(paid)
                .multiply(new BigDecimal(remaining))
                .divide(new BigDecimal(total), MathContext.DECIMAL64)
                .multiply(BigDecimal.ONE.subtract(EXPENSES_SHARE))
                .subtract(Money.nz(payouts));
        if (refund.signum() < 0) {
            refund = BigDecimal.ZERO;
        }
        return new RefundCalculation(Money.nz(paid), total, remaining,
                EXPENSES_SHARE, Money.nz(payouts), Money.round(refund));
    }
}
```

```
// Файл server/logic/PolicyStatusRules.java
package by.bsuir.insurance.server.logic;

/** Правила автоматической смены состояний договора (см. диаграмму
        состояний). */
public final class PolicyStatusRules {
    /** Срок оплаты первого взноса после оформления, дней. */
    public static final int FIRST_PAYMENT_DAYS = 10;
    /** Допустимая просрочка очередного взноса до расторжения, дней. */
    public static final int MAX_OVERDUE_DAYS = 30;

    private PolicyStatusRules() {
    }

    /** Новое состояние договора на дату today (или текущее, если переход не
            нужен). */
    public static PolicyStatus evaluate(Policy policy, List<Payment>
            payments, LocalDate today) {
        PolicyStatus status = policy.getStatus();
        if (status == PolicyStatus.TERMINATED || status ==
                PolicyStatus.FULFILLED || status == PolicyStatus.EXPIRED) {
            return status;
        }
        if (status == PolicyStatus.WAITING_PAYMENT) {
            Payment first = payments.isEmpty() ? null : payments.get(0);
            if (first != null && first.isPaid()) {
                return PolicyStatus.ACTIVE;
            }
            LocalDate created = policy.getCreatedAt().toLocalDate();
            return ChronoUnit.DAYS.between(created, today) >
                    FIRST_PAYMENT_DAYS ? PolicyStatus.TERMINATED : status;
        }
        if (today.isAfter(policy.getEndDate())) {
            return PolicyStatus.EXPIRED;
        }
        long maxOverdue = 0;
        for (Payment p : payments) {
            if (p.isOverdue(today)) {
                maxOverdue = Math.max(maxOverdue,
                        ChronoUnit.DAYS.between(p.getDueDate(), today));
            }
        }
        if (maxOverdue > MAX_OVERDUE_DAYS) {
            return PolicyStatus.TERMINATED;
        }
        return maxOverdue > 0 ? PolicyStatus.SUSPENDED :
                PolicyStatus.ACTIVE;
    }
}

// Файл server/service/ApplicationService.java
package by.bsuir.insurance.server.service;

/** Расчёт стоимости, заявки и оформление договоров по заявкам. */
public class ApplicationService {
```

```
private final ApplicationDao applicationDao = new ApplicationDao();
private final InsuranceTypeDao typeDao = new InsuranceTypeDao();
private final ClientDao clientDao = new ClientDao();
private final ClaimDao claimDao = new ClaimDao();
private final PolicyDao policyDao = new PolicyDao();
private final PaymentDao paymentDao = new PaymentDao();
private final PremiumCalculator calculator = new PremiumCalculator();
private final ConnectionManager cm = ConnectionManager.getInstance();

/** Расчёт стоимости страхования для клиента (UC3). */
public PremiumQuote quote(long clientId, Application a, LocalDate
        startDate) throws Exception {
    return cm.withConnection(c -> quote(c, clientId, a, startDate));
}

private PremiumQuote quote(Connection c, long clientId, Application a,
        LocalDate startDate) throws Exception {
    InsuranceType type = typeDao.findById(c, a.getTypeId());
    if (type == null) {
        throw new ValidationException("Вид страхования не найден");
    }
    Client client = clientDao.findById(c, clientId);
    int claims = claimDao.countPaidClaims(c, clientId,
            startDate.minusYears(2), startDate);
    return calculator.calculate(type, a, client, startDate, claims);
}

/** Подача заявки клиентом (UC4). Параметры проверяются тем же
        калькулятором. */
public Application submit(long clientId, Application a) throws Exception
        {
    a.setClientId(clientId);
    a.setObjectDescription(Validator.required(a.getObjectDescription(),
            "Описание объекта страхования"));
    return cm.inTransaction(c -> {
        quote(c, clientId, a, LocalDate.now().plusDays(1));
        long id = applicationDao.insert(c, a);
        return applicationDao.findById(c, id);
    });
}

public ArrayList<Application> listByClient(long clientId) throws
        Exception {
    return new ArrayList<>(cm.withConnection(c ->
            applicationDao.listByClient(c, clientId)));
}

/** Новые заявки с предварительным расчётом премии. */
public ArrayList<Application> listNew() throws Exception {
    return cm.withConnection(c -> {
        List<Application> list = applicationDao.listByStatus(c,
                ApplicationStatus.NEW);
        for (Application a : list) {
            try {
                a.setEstimatedPremium(quote(c, a.getClientId(), a,
                        LocalDate.now().plusDays(1)).getPremium());
```

```
            } catch (ValidationException e) {
                a.setEstimatedPremium(null);
            }
        }
        return new ArrayList<>(list);
    });
}

/**
 * Одобрение заявки и оформление договора (UC5): в одной транзакции
        рассчитывается премия,
 * создаются договор и график взносов, заявка переводится в статус
        «Одобрена».
 */
public Policy approve(long agentId, long applicationId, LocalDate
        startDate) throws Exception {
    if (startDate == null || startDate.isBefore(LocalDate.now())) {
        throw new ValidationException("Дата начала действия договора не
                может быть в прошлом");
    }
    return cm.inTransaction(c -> {
        Application a = applicationDao.lockById(c, applicationId);
        if (a == null) {
            throw new ValidationException("Заявка не найдена");
        }
        if (a.getStatus() != ApplicationStatus.NEW) {
            throw new BusinessRuleException("Заявка уже рассмотрена: " +
                    a.getStatus().getTitle());
        }
        PremiumQuote q = quote(c, a.getClientId(), a, startDate);
        Policy p = new Policy();
        p.setApplicationId(a.getId());
        p.setClientId(a.getClientId());
        p.setAgentId(agentId);
        p.setTypeId(a.getTypeId());
        p.setInsuredSum(q.getInsuredSum());
        p.setPremium(q.getPremium());
        p.setFranchise(q.getFranchise());
        p.setStartDate(startDate);
        p.setEndDate(startDate.plusMonths(a.getTermMonths()).minusDays(1
                ));
        long policyId = policyDao.insert(c, p);
        List<Payment> schedule = InstallmentScheduler.build(policyId,
                q.getPremium(), a.getInstallments(),
                startDate, a.getTermMonths());
        paymentDao.insertAll(c, schedule);
        applicationDao.decide(c, a.getId(), ApplicationStatus.APPROVED,
                agentId,
                "Оформлен договор " + p.getPolicyNumber());
        return policyDao.findById(c, policyId);
    });
}

public Application reject(long agentId, long applicationId, String
        comment) throws Exception {
    String reason = Validator.required(comment, "Причина отказа");
```

```
        return cm.inTransaction(c -> {
            Application a = applicationDao.lockById(c, applicationId);
            if (a == null) {
                throw new ValidationException("Заявка не найдена");
            }
            if (a.getStatus() != ApplicationStatus.NEW) {
                throw new BusinessRuleException("Заявка уже рассмотрена: " +
                        a.getStatus().getTitle());
            }
            applicationDao.decide(c, applicationId,
                    ApplicationStatus.REJECTED, agentId, reason);
            return applicationDao.findById(c, applicationId);
        });
    }
}

// Файл server/service/PolicyService.java
package by.bsuir.insurance.server.service;

/** Сопровождение договоров: взносы, продление, расторжение, контроль. */
public class PolicyService {
    private static final Logger LOG =
            Logger.getLogger(PolicyService.class.getName());
    /** За сколько дней до окончания договор попадает в список истекающих и
            может быть продлён. */
    public static final int EXPIRING_DAYS = 30;

    private final PolicyDao policyDao = new PolicyDao();
    private final PaymentDao paymentDao = new PaymentDao();
    private final ClaimDao claimDao = new ClaimDao();
    private final ClientDao clientDao = new ClientDao();
    private final InsuranceTypeDao typeDao = new InsuranceTypeDao();
    private final ApplicationDao applicationDao = new ApplicationDao();
    private final PremiumCalculator premiumCalculator = new
            PremiumCalculator();
    private final RefundCalculator refundCalculator = new
            RefundCalculator();
    private final ConnectionManager cm = ConnectionManager.getInstance();

    public ArrayList<Policy> listByClient(long clientId) throws Exception {
        return new ArrayList<>(cm.withConnection(c ->
                policyDao.listByClient(c, clientId)));
    }

    public ArrayList<Policy> listAll(PolicyStatus status) throws Exception {
        return new ArrayList<>(cm.withConnection(c -> policyDao.listAll(c,
                status)));
    }

    /** График взносов; клиент может просматривать только свои договоры. */
    public ArrayList<Payment> payments(User user, long policyId) throws
            Exception {
        return cm.withConnection(c -> {
            Policy p = policyDao.findById(c, policyId);
            checkAccess(user, p);
            return new ArrayList<>(paymentDao.listByPolicy(c, policyId));
```

```
    });
}

/** Внесение клиентом ближайшего неуплаченного взноса (UC6). */
public Policy payInstallment(User user, long policyId) throws Exception
        {
    LocalDate today = LocalDate.now();
    return cm.inTransaction(c -> {
        Policy p = policyDao.lockById(c, policyId);
        checkAccess(user, p);
        if (p.getStatus() != PolicyStatus.WAITING_PAYMENT &&
                p.getStatus() != PolicyStatus.ACTIVE
                && p.getStatus() != PolicyStatus.SUSPENDED) {
            throw new BusinessRuleException("По договору в статусе «" +
                    p.getStatus().getTitle()
                    + "» взносы не принимаются");
        }
        Payment next = null;
        for (Payment pm : paymentDao.listByPolicy(c, policyId)) {
            if (!pm.isPaid()) {
                next = pm;
                break;
            }
        }
        if (next == null) {
            throw new BusinessRuleException("Все взносы по договору уже
                    уплачены");
        }
        paymentDao.markPaid(c, policyId, next.getInstallmentNo(),
                today);
        applyRules(c, policyDao.findById(c, policyId), today);
        return policyDao.findById(c, policyId);
    });
}

/**
 * Продление договора (UC9): создаётся новый договор того же клиента и
        вида на тот же срок,
 * начинающийся на следующий день после окончания текущего; премия
        рассчитывается заново.
 */
public Policy renew(long agentId, long policyId) throws Exception {
    return cm.inTransaction(c -> {
        Policy old = policyDao.lockById(c, policyId);
        if (old == null) {
            throw new ValidationException("Договор не найден");
        }
        LocalDate today = LocalDate.now();
        boolean expiringSoon = (old.getStatus() == PolicyStatus.ACTIVE)
                && !old.getEndDate().isAfter(today.plusDays(EXPIRING_DAY
                        S));
        if (old.getStatus() != PolicyStatus.EXPIRED && !expiringSoon) {
            throw new BusinessRuleException("Продлить можно истёкший
                    договор или действующий договор, "
                    + "срок которого заканчивается в течение " +
                            EXPIRING_DAYS + " дней");
```

```
            }
            if (policyDao.hasRenewal(c, old)) {
                throw new BusinessRuleException("Договор уже продлён");
            }
            int term = (int) ChronoUnit.MONTHS.between(old.getStartDate(),
                    old.getEndDate().plusDays(1));
            LocalDate start = old.getEndDate().plusDays(1);
            if (start.isBefore(today.plusDays(1))) {
                start = today.plusDays(1);
            }
            Application params = new Application();
            params.setTypeId(old.getTypeId());
            params.setRequestedSum(old.getInsuredSum());
            params.setTermMonths(term);
            params.setFranchisePercent(old.getFranchise().signum() == 0 ? 0
                    : old.getFranchise().multiply(new
                            java.math.BigDecimal(100))
                    .divide(old.getInsuredSum(), 0,
                            java.math.RoundingMode.HALF_UP).intValue());
            params.setInstallments(1);
            Application source = old.getApplicationId() == null ? null :
                    applicationDao.findById(c, old.getApplicationId());
            params.setObjectDescription(source == null ? "" :
                    source.getObjectDescription());
            if (source != null && term % source.getInstallments() == 0) {
                params.setInstallments(source.getInstallments());
            }
            int claims = claimDao.countPaidClaims(c, old.getClientId(),
                    start.minusYears(2), start);
            PremiumQuote q = premiumCalculator.calculate(typeDao.findById(c,
                    old.getTypeId()), params,
                    clientDao.findById(c, old.getClientId()), start,
                            claims);
            Policy p = new Policy();
            p.setClientId(old.getClientId());
            p.setAgentId(agentId);
            p.setTypeId(old.getTypeId());
            p.setInsuredSum(q.getInsuredSum());
            p.setPremium(q.getPremium());
            p.setFranchise(q.getFranchise());
            p.setStartDate(start);
            p.setEndDate(start.plusMonths(term).minusDays(1));
            long id = policyDao.insert(c, p);
            paymentDao.insertAll(c, InstallmentScheduler.build(id,
                    q.getPremium(), params.getInstallments(), start, term));
            return policyDao.findById(c, id);
        });
    }

    /** Предварительный расчёт возврата при расторжении. */
    public RefundCalculation previewRefund(long policyId, LocalDate date)
            throws Exception {
        return cm.withConnection(c -> refund(c,
                checkTerminable(policyDao.findById(c, policyId)), date));
    }
```

```
private RefundCalculation refund(Connection c, Policy p, LocalDate date)
        throws Exception {
    return refundCalculator.calculate(p, p.getPaidAmount(),
            claimDao.sumPayouts(c, p.getId()), date);
}

/** Досрочное расторжение с возвратом части премии (UC10). */
public Policy terminate(long policyId) throws Exception {
    LocalDate today = LocalDate.now();
    return cm.inTransaction(c -> {
        Policy p = checkTerminable(policyDao.lockById(c, policyId));
        RefundCalculation r = refund(c, p, today);
        paymentDao.deleteUnpaidAfter(c, policyId, today);
        policyDao.terminate(c, policyId, LocalDateTime.now(),
                r.getRefund());
        return policyDao.findById(c, policyId);
    });
}

private Policy checkTerminable(Policy p) throws Exception {
    if (p == null) {
        throw new ValidationException("Договор не найден");
    }
    if (p.getStatus() != PolicyStatus.ACTIVE && p.getStatus() !=
            PolicyStatus.SUSPENDED
            && p.getStatus() != PolicyStatus.WAITING_PAYMENT) {
        throw new BusinessRuleException("Расторгнуть можно только
                действующий, приостановленный "
                + "или ожидающий оплаты договор");
    }
    return p;
}

/** Список контроля (UC11): просроченные взносы и договоры, истекающие в
        ближайшие 30 дней. */
public ArrayList<ControlItem> controlList() throws Exception {
    LocalDate today = LocalDate.now();
    return cm.withConnection(c -> {
        ArrayList<ControlItem> items = new
                ArrayList<>(paymentDao.listOverdue(c, today));
        for (Policy p : policyDao.listExpiring(c, today, EXPIRING_DAYS))
                {
            if (policyDao.hasRenewal(c, p)) {
                continue;
            }
            items.add(new ControlItem(ControlItem.Kind.EXPIRING_POLICY,
                    p.getId(), p.getPolicyNumber(),
                    p.getClientName(), clientDao.findById(c,
                            p.getClientId()).getPhone(), p.getEndDate(),
                    p.getPremium(), ChronoUnit.DAYS.between(today,
                            p.getEndDate())));
        }
        return items;
    });
}
```

```
    /** Пересчёт состояний всех изменяемых договоров на текущую дату.
            Возвращает число изменённых. */
    public int refreshStatuses() throws Exception {
        LocalDate today = LocalDate.now();
        int changed = cm.inTransaction(c -> {
            int n = 0;
            for (Policy p : policyDao.listChangeable(c)) {
                if (applyRules(c, p, today)) {
                    n++;
                }
            }
            return n;
        });
        LOG.info("Пересчёт состояний договоров: изменено " + changed);
        return changed;
    }

    private boolean applyRules(Connection c, Policy p, LocalDate today)
            throws Exception {
        List<Payment> payments = paymentDao.listByPolicy(c, p.getId());
        PolicyStatus next = PolicyStatusRules.evaluate(p, payments, today);
        if (next != p.getStatus()) {
            policyDao.updateStatus(c, p.getId(), next);
            return true;
        }
        return false;
    }

    static void checkAccess(User user, Policy p) throws Exception {
        if (p == null) {
            throw new ValidationException("Договор не найден");
        }
        if (user.getRole() == Role.CLIENT && p.getClientId() !=
                user.getPersonId()) {
            throw new AuthException("Нет доступа к чужому договору");
        }
    }
}

// Файл server/service/ClaimService.java
package by.bsuir.insurance.server.service;

/** Заявления о страховых случаях и их урегулирование. */
public class ClaimService {
    private final ClaimDao claimDao = new ClaimDao();
    private final PolicyDao policyDao = new PolicyDao();
    private final PaymentDao paymentDao = new PaymentDao();
    private final PayoutCalculator calculator = new PayoutCalculator();
    private final ConnectionManager cm = ConnectionManager.getInstance();

    /** Заявление клиента о страховом случае (UC7). */
    public Claim report(User user, Claim claim) throws Exception {
        Validator.notFuture(claim.getIncidentDate(), "Дата события");
        Validator.positive(claim.getDamageAmount(), "Размер ущерба");
        claim.setDescription(Validator.required(claim.getDescription(),
                "Описание события"));
```

```
    claim.setReportDate(LocalDate.now());
    return cm.inTransaction(c -> {
        Policy p = policyDao.findById(c, claim.getPolicyId());
        PolicyService.checkAccess(user, p);
        if (!p.getStatus().allowsClaims()) {
            throw new BusinessRuleException("По договору в статусе «" +
                    p.getStatus().getTitle()
                    + "» заявления не принимаются");
        }
        return claimDao.findById(c, claimDao.insert(c, claim));
    });
}

public ArrayList<Claim> listByClient(long clientId) throws Exception {
    return new ArrayList<>(cm.withConnection(c ->
            claimDao.listByClient(c, clientId)));
}

public ArrayList<Claim> listPending() throws Exception {
    return new ArrayList<>(cm.withConnection(c ->
            claimDao.listByStatus(c, ClaimStatus.REPORTED)));
}

public ArrayList<Claim> listAll() throws Exception {
    return new ArrayList<>(cm.withConnection(claimDao::listAll));
}

/** Предварительный расчёт выплаты для показа агенту. */
public PayoutDecision preview(long claimId, BigDecimal assessedDamage)
        throws Exception {
    return cm.withConnection(c -> decide(c, claimDao.findById(c,
            claimId), assessedDamage));
}

/**
 * Урегулирование страхового случая (UC8): расчёт выплаты, фиксация
        решения и,
 * если страховая сумма исчерпана, перевод договора в состояние
        «Исполнен».
 */
public Claim settle(long agentId, long claimId, BigDecimal
        assessedDamage, String comment) throws Exception {
    return cm.inTransaction(c -> {
        Claim claim = claimDao.lockById(c, claimId);
        PayoutDecision d = decide(c, claim, assessedDamage);
        String text = (comment == null || comment.trim().isEmpty()) ?
                d.getReason() : comment.trim() + ". " + d.getReason();
        claimDao.decide(c, claimId, d.isApproved() ?
                ClaimStatus.APPROVED : ClaimStatus.REJECTED,
                Money.round(assessedDamage), d.getAmount(), agentId,
                        text);
        if (d.isExhaustsSum()) {
            policyDao.updateStatus(c, claim.getPolicyId(),
                    PolicyStatus.FULFILLED);
        }
        return claimDao.findById(c, claimId);
```

```
        });
    }

    private PayoutDecision decide(Connection c, Claim claim, BigDecimal
            assessedDamage) throws Exception {
        if (claim == null) {
            throw new ValidationException("Страховой случай не найден");
        }
        if (claim.getStatus() != ClaimStatus.REPORTED) {
            throw new BusinessRuleException("Решение по страховому случаю
                    уже принято");
        }
        Validator.positive(assessedDamage, "Оценённый ущерб");
        Policy p = policyDao.lockById(c, claim.getPolicyId());
        return calculator.calculate(p, paymentDao.listByPolicy(c,
                p.getId()), claim, assessedDamage,
                claimDao.sumPayouts(c, p.getId()));
    }
}
```

ПРИЛОЖЕНИЕ В

(обязательное)

Листинг скрипта генерации базы данных

```
-- Файл database/01_schema.sql
-- ========================================================================
-- ПС «Страховой сервис»: скрипт генерации базы данных (PostgreSQL 16)
-- Создание: psql -U postgres -f 01_schema.sql
-- ========================================================================
DROP DATABASE IF EXISTS insurance WITH (FORCE);
CREATE DATABASE insurance ENCODING 'UTF8' TEMPLATE template0;
\connect insurance

-- Роли пользователей
CREATE TABLE roles (
    id   SERIAL PRIMARY KEY,
    name VARCHAR(20) NOT NULL UNIQUE
);

-- Учётные записи
CREATE TABLE users (
    id            SERIAL PRIMARY KEY,
    login         VARCHAR(50)  NOT NULL UNIQUE,
    password_hash CHAR(64)     NOT NULL,
    salt          CHAR(32)     NOT NULL,
    role_id       INTEGER      NOT NULL REFERENCES roles (id) ON DELETE
            RESTRICT,
    is_blocked    BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMP    NOT NULL DEFAULT now(),
    CONSTRAINT chk_users_login CHECK (length(login) >= 3)
);

-- Клиенты (страхователи)
CREATE TABLE clients (
    id          SERIAL PRIMARY KEY,
    user_id     INTEGER      NOT NULL UNIQUE REFERENCES users (id) ON DELETE
            RESTRICT,
    last_name   VARCHAR(50)  NOT NULL,
    first_name  VARCHAR(50)  NOT NULL,
    patronymic  VARCHAR(50),
    birth_date  DATE         NOT NULL,
    passport_no VARCHAR(20)  NOT NULL UNIQUE,
    phone       VARCHAR(20)  NOT NULL,
    address     VARCHAR(200) NOT NULL,
    CONSTRAINT chk_clients_birth CHECK (birth_date > DATE '1900-01-01')
);

-- Страховые агенты
CREATE TABLE agents (
    id              SERIAL PRIMARY KEY,
    user_id         INTEGER      NOT NULL UNIQUE REFERENCES users (id) ON
            DELETE RESTRICT,
    last_name       VARCHAR(50)  NOT NULL,
    first_name      VARCHAR(50)  NOT NULL,
```

```
    patronymic      VARCHAR(50),
    position        VARCHAR(60)  NOT NULL,
    hire_date       DATE         NOT NULL,
    commission_rate NUMERIC(5,4) NOT NULL DEFAULT 0.1000,
    CONSTRAINT chk_agents_commission CHECK (commission_rate >= 0 AND
            commission_rate <= 0.5)
);

-- Виды страхования и тарифы
CREATE TABLE insurance_types (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(80)   NOT NULL UNIQUE,
    category        VARCHAR(20)   NOT NULL,
    description     TEXT,
    base_rate       NUMERIC(6,4)  NOT NULL,
    min_sum         NUMERIC(12,2) NOT NULL,
    max_sum         NUMERIC(12,2) NOT NULL,
    max_term_months INTEGER       NOT NULL,
    is_active       BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT chk_types_category CHECK (category IN ('PROPERTY', 'CASCO',
            'ACCIDENT', 'TRAVEL')),
    CONSTRAINT chk_types_rate CHECK (base_rate > 0 AND base_rate < 1),
    CONSTRAINT chk_types_sums CHECK (min_sum > 0 AND max_sum >= min_sum),
    CONSTRAINT chk_types_term CHECK (max_term_months BETWEEN 1 AND 60)
);

-- Заявки на страхование
CREATE TABLE applications (
    id                  SERIAL PRIMARY KEY,
    client_id           INTEGER       NOT NULL REFERENCES clients (id) ON
            DELETE RESTRICT,
    type_id             INTEGER       NOT NULL REFERENCES insurance_types
            (id) ON DELETE RESTRICT,
    agent_id            INTEGER       REFERENCES agents (id) ON DELETE
            RESTRICT,
    requested_sum       NUMERIC(12,2) NOT NULL,
    term_months         INTEGER       NOT NULL,
    franchise_percent   INTEGER       NOT NULL DEFAULT 0,
    installments        INTEGER       NOT NULL DEFAULT 1,
    object_description  TEXT          NOT NULL,
    status              VARCHAR(20)   NOT NULL DEFAULT 'NEW',
    decision_comment    TEXT,
    created_at          TIMESTAMP     NOT NULL DEFAULT now(),
    decided_at          TIMESTAMP,
    CONSTRAINT chk_app_status CHECK (status IN ('NEW', 'APPROVED',
            'REJECTED')),
    CONSTRAINT chk_app_sum CHECK (requested_sum > 0),
    CONSTRAINT chk_app_term CHECK (term_months > 0),
    CONSTRAINT chk_app_franchise CHECK (franchise_percent IN (0, 1, 2, 5)),
    CONSTRAINT chk_app_installments CHECK (installments IN (1, 2, 4, 12))
);

-- Договоры страхования (полисы)
CREATE TABLE policies (
    id              SERIAL PRIMARY KEY,
    policy_number   VARCHAR(20)   NOT NULL UNIQUE,
```

```
    application_id INTEGER       UNIQUE REFERENCES applications (id) ON
            DELETE RESTRICT,
    client_id      INTEGER       NOT NULL REFERENCES clients (id) ON DELETE
            RESTRICT,
    agent_id       INTEGER       NOT NULL REFERENCES agents (id) ON DELETE
            RESTRICT,
    type_id        INTEGER       NOT NULL REFERENCES insurance_types (id) ON
            DELETE RESTRICT,
    insured_sum    NUMERIC(12,2) NOT NULL,
    premium        NUMERIC(12,2) NOT NULL,
    franchise      NUMERIC(12,2) NOT NULL DEFAULT 0,
    start_date     DATE          NOT NULL,
    end_date       DATE          NOT NULL,
    status         VARCHAR(20)   NOT NULL DEFAULT 'WAITING_PAYMENT',
    created_at     TIMESTAMP     NOT NULL DEFAULT now(),
    terminated_at  TIMESTAMP,
    refund_amount  NUMERIC(12,2),
    CONSTRAINT chk_pol_status CHECK (status IN
        ('WAITING_PAYMENT', 'ACTIVE', 'SUSPENDED', 'EXPIRED', 'TERMINATED',
                'FULFILLED')),
    CONSTRAINT chk_pol_sums CHECK (insured_sum > 0 AND premium > 0 AND
            franchise >= 0),
    CONSTRAINT chk_pol_dates CHECK (end_date > start_date)
);

-- Страховые взносы (зависимая сущность, идентифицирующая связь с договором)
CREATE TABLE payments (
    policy_id      INTEGER       NOT NULL REFERENCES policies (id) ON DELETE
            CASCADE,
    installment_no INTEGER       NOT NULL,
    amount         NUMERIC(12,2) NOT NULL,
    due_date       DATE          NOT NULL,
    paid_date      DATE,
    PRIMARY KEY (policy_id, installment_no),
    CONSTRAINT chk_pay_amount CHECK (amount > 0),
    CONSTRAINT chk_pay_no CHECK (installment_no > 0)
);

-- Страховые случаи
CREATE TABLE claims (
    id               SERIAL PRIMARY KEY,
    policy_id        INTEGER       NOT NULL REFERENCES policies (id) ON
            DELETE RESTRICT,
    incident_date    DATE          NOT NULL,
    report_date      DATE          NOT NULL,
    description      TEXT          NOT NULL,
    damage_amount    NUMERIC(12,2) NOT NULL,
    assessed_damage  NUMERIC(12,2),
    status           VARCHAR(20)   NOT NULL DEFAULT 'REPORTED',
    payout_amount    NUMERIC(12,2) NOT NULL DEFAULT 0,
    agent_id         INTEGER       REFERENCES agents (id) ON DELETE
            RESTRICT,
    decision_comment TEXT,
    decided_at       TIMESTAMP,
    CONSTRAINT chk_claim_status CHECK (status IN ('REPORTED', 'APPROVED',
            'REJECTED')),
```

```
    CONSTRAINT chk_claim_amounts CHECK (damage_amount > 0 AND payout_amount
            >= 0),
    CONSTRAINT chk_claim_dates CHECK (report_date >= incident_date)
);

-- Индексы для частых выборок
CREATE INDEX idx_app_status   ON applications (status);
CREATE INDEX idx_pol_client   ON policies (client_id);
CREATE INDEX idx_pol_status   ON policies (status);
CREATE INDEX idx_pay_due      ON payments (due_date) WHERE paid_date IS
        NULL;
CREATE INDEX idx_claim_policy ON claims (policy_id);

INSERT INTO roles (name) VALUES ('CLIENT'), ('AGENT'), ('ADMIN');
```

Обозначение	Наименование	Дополнительные сведения
	Текстовые документы	
БГУИР КП 1-40 01 02-08 015 ПЗ	Пояснительная записка	115 с.
	Отчет о проверке на заимство-	Приложение А
	вания в системе "Антиплагиат"	к ПЗ
	Графические документы	
ГУИР.425707.015 Д2	IDEF0-модель процессов	Формат А4
	предметной области	
ГУИР.425707.015 Д3	Схема алгоритма расчёта	Формат А4
	страховой выплаты	
ГУИР.425707.015 Д5	Плакаты, отражающие	Формат А4
	результаты курсового	3 листа
	проектирования:	
	Лист 1 – Диаграмма классов ПС	
	Лист 2 – Модели представления ПС	
	Лист 3 – Интерфейс ПС	
	Прочие документы	
	Исходный код программного	Репозиторий
	средства	GitHub

Перв.примен.

Справ.№

Подп. и дата

Инв.№ дубл.

Взам.инв.№

Подп.и дата

Инв.№ подл.

Изм.	Лист	№ докум.	Подп.	Дата

БГУИР КП 1-40 01 02-08 015 Д1

Разраб.	Балащенко			
Пров.	Сторожев			
Т.контр.				
Н.контр.				
Утв.				

Программное средство для поддержки процесса оказания страховых услуг населению

Ведомость документов

Лит.	Лист	Листов
Т	115	115

Кафедра ЭИ, группа 414351